



Universidad de Extremadura

Centro Universitario de Mérida

Grado en Ingeniería Informática

Trabajo Fin de Grado

Estudio, implementación y perfilado de FlashAttention en GPU mediante CUDA, Triton y PyTorch

Ángel Moreno Domínguez

Mérida, septiembre de 2026





Universidad de Extremadura

Centro Universitario de Mérida

Grado en Ingeniería Informática

Trabajo Fin de Grado

Estudio, implementación y perfilado de FlashAttention en GPU mediante CUDA, Triton y PyTorch

Autor: Ángel Moreno Domínguez
Fdo:

Directores: Mercedes Eugenia Paoletti Ávila
Juan Mario Haut Hurtado
Fdo:

Mérida, septiembre de 2026

Agradecimientos

Quiero dar las gracias a mi familia, cuyo apoyo silencioso y paciencia han sido la base de cada paso que he dado, y a mis amigos, que me recordaron que existe vida más allá de la pantalla.

Estoy profundamente agradecido a Manuel Ujaldón y Mercedes. Desde el principio confiaron en mí y vieron potencial donde yo solo veía incertidumbre. Consideraron que sería una buena idea que participara en una edición de PUMPS+AI, una experiencia que amplió mi visión del campo y me dio una confianza que no estaba seguro de tener.

Aunque a veces el panorama general pueda parecer sombrío, veo luz en el futuro, y buena parte de esa luz procede de las personas que me han acompañado durante el camino.

Quiero agradecer a Francisco Chávez las muchas horas que dedicó a resolver los problemas de la cola de Slurm. También estoy agradecido a COMPUTAEX y a Javier por aceptarme durante un mes completo de prácticas y por hacerme sentir, desde el primer día, como un miembro valorado del equipo. A Miguel Ángel le debo un agradecimiento especial por su apoyo durante una situación personal difícil hace dos años; su amabilidad llegó en el momento en que más la necesitaba.

También quiero dar las gracias a Javier, cuyas clases de EC disfruté sinceramente y que, sin que yo fuera consciente de ello en aquel momento, acabaron orientando la dirección de este Trabajo Fin de Grado.

Estoy profundamente agradecido a Mario por aceptar la dirección de este trabajo, así como por la orientación y la confianza que ha depositado en mí.

Este trabajo refleja muchas horas de estudio, esfuerzo y perseverancia.

Resumen

El rendimiento de una aplicación no depende únicamente de la capacidad del hardware disponible, sino también de la forma en la que el programa consigue aprovecharlo. Esta cuestión resulta especialmente importante en las GPU, donde alcanzar un alto rendimiento requiere tener en cuenta aspectos como la jerarquía de memoria, la ocupancia, el paralelismo o el coste de las distintas instrucciones.

Este Trabajo Fin de Grado estudia la implementación y optimización de FlashAttention para una GPU NVIDIA con arquitectura Ampere. Se desarrollan dos implementaciones del mecanismo de atención no causal, una en CUDA y otra en Triton, ambas validadas numéricamente frente a SDPA. En la implementación CUDA se analizan distintas decisiones de bajo nivel relacionadas con el uso de registros y memoria compartida, el número de *warps*, el número de bloques residentes, el uso de múltiples buffers, el *loop unrolling* y las políticas de caché. También se estudia el cálculo de la función exponencial, desarrollando una aproximación polinómica adaptada al posterior uso de BF16.

La búsqueda de configuraciones se automatiza mediante Optuna y el comportamiento de los kernels se analiza utilizando NVIDIA Nsight Compute y el código SASS generado. Finalmente, las implementaciones desarrolladas se comparan con Torch SDPA y cuDNN SDPA para diferentes longitudes de secuencia, estudiando tanto su rendimiento como su precisión numérica.

El trabajo muestra cómo decisiones de implementación aparentemente pequeñas pueden tener un efecto significativo en el aprovechamiento final del hardware.

Palabras clave **-** FlashAttention, CUDA, Triton, GPU, Ampere, SDPA, optimización, rendimiento

Abstract

The performance of an application depends not only on the capabilities of the available hardware, but also on how efficiently the program is able to exploit it. This is particularly important for GPUs, where achieving high performance requires considering aspects such as the memory hierarchy, occupancy, parallelism, and the cost of different instructions.

This Bachelor's Thesis studies the implementation and optimization of FlashAttention for an NVIDIA GPU based on the Ampere architecture. Two implementations of the non-causal attention mechanism are developed, one in CUDA and another in Triton, both numerically validated against SDPA. The CUDA implementation analyzes several low-level design decisions related to register and shared-memory usage, the number of *warps*, the number of resident blocks, the use of multiple buffers, *loop unrolling*, and cache policies. The computation of the exponential function is also studied, developing a polynomial approximation adapted to the subsequent use of BF16.

The search for suitable configurations is automated using Optuna, while kernel behavior is analyzed using NVIDIA Nsight Compute and the generated SASS code. Finally, the developed implementations are compared with Torch SDPA and cuDNN SDPA for different sequence lengths, considering both performance and numerical accuracy.

The results show how seemingly small implementation decisions can have a significant effect on the final utilization of the available hardware.

Keywords **-** FlashAttention, CUDA, Triton, GPU, Ampere, SDPA, optimization, performance

Índice general

Índice de figuras **VII**

Índice de tablas **X**

1. Motivación y objetivos **1**
1.1. Introducción . . . 1
1.2. Motivación . . . 3
1.3. Objetivos . . . 3
1.4. Originalidad del TFG . . . 4
1.5. Organización del documento . . . 4

2. Trabajo relacionado / Conceptos básicos **5**
2.1. Trabajos relacionados . . . 5
2.1.1. Writing Speed-of-Light Flash Attention for 5090 in CUDA C++ 5
2.1.2. Flash Attention From Scratch . . . 6
2.1.3. GEMM . . . 6
2.1.4. MLC.ai: Modern GPU Programming for MLSys . . . 7
2.1.5. FlashAttention . . . 7
2.1.6. FlashAttention-2 . . . 7
2.2. Conceptos básicos . . . 8
2.2.1. Acceso coalescente . . . 8
2.2.2. Conflictos de banco . . . 8
2.2.3. CTA . . . 9
2.2.4. Double buffering . . . 9
2.2.5. Hiperparámetro . . . 9
2.2.6. ILP y TLP . . . 9
2.2.7. L1 . . . 9
2.2.8. L2 . . . 10
2.2.9. Lane . . . 10
2.2.10. Latencia y throughput . . . 10
2.2.11. Loop unrolling . . . 10
2.2.12. Memoria compartida . . . 11

2.2.13. Ocupancia . . . 11
2.2.14. Política de eviction . . . 12
2.2.15. Predicación . . . 13
2.2.16. Registros . . . 13
2.2.17. SASS y PTX . . . 13
2.2.18. Spills . . . 13
2.2.19. Streaming Multiprocessor . . . 13
2.2.20. Swizzle . . . 14
2.2.21. Tile . . . 14
2.2.22. TPE multivariante . . . 14
2.2.23. ULP . . . 14
2.2.24. Warp . . . 14
2.2.25. Warp divergence . . . 15
2.2.26. Wavefront . . . 15
2.3. Herramientas . . . 15
2.3.1. CUDA 13.0 & C++20 . . . 15
2.3.2. NVCC . . . 16
2.3.3. CMake . . . 16
2.3.4. uv . . . 16
2.3.5. Pytest . . . 16
2.3.6. Triton . . . 16
2.3.7. NCU . . . 16
2.3.8. Optuna . . . 17

3. Metodología de desarrollo **18**
3.1. Algoritmo general FlashAttention-2 . . . 18
3.2. Instrucción `mma.sync` . . . 21
3.3. Decisiones de paralelismo y warp coarsening . . . 23
3.4. Uso de registros y SRAM . . . 24
3.5. Cargado Q_r . . . 27
3.6. Traspuesta en registros. . . . 30
3.7. Instrucción `ldmatrix.x4`: carga de un tile 16×16 de SRAM con layout `mma.sync` . . . 32
3.8. Instrucción `cp.async` . . . 32
3.9. Doble buffer SMEM-RF . . . 33
3.10. K_i/V_i y SRAM . . . 33
3.10.1. SRAM layout naive vs swizzled . . . 34
3.11. Multibuffer en SRAM . . . 35
3.12. Loop unrolling y partial loop unrolling . . . 36
3.13. Cálculo del row-sum y del row-max . . . 36

3.14. Cálculo de la exponencial . . . 37
3.14.1. Fusionar el scale con la exponencial . . . 37
3.14.2. Cálculo de $e^{\text{scale(score}-\text{row_max)}}$ . . . 38
3.14.3. Reducción de rango . . . 39
3.14.4. Sollya . . . 39
3.14.5. Downcast de FP32 a BF16 . . . 40
3.14.6. Cálculo de 2^n . . . 41
3.14.7. Validez . . . 43
3.14.8. Análisis SASS: `__expf` frente a la aproximación por polinomio . 44
3.15. Downcast a P . . . 45
3.16. Multiplicar PV . . . 46
3.17. División por recíproco de O . . . 46
3.18. Volcado de O en memoria principal . . . 46
3.19. Optimización de hiperparámetros mediante Optuna . . . 46
3.19.1. Poda del espacio de búsqueda . . . 47

4. Análisis de resultados **50**
4.1. Cualidades del sistema . . . 50
4.2. Comandos NCU . . . 50
4.2.1. Por qué utilizamos NCU para medir el rendimiento . . . 51
4.3. Longitud de secuencia 8192 . . . 53
4.3.1. Ciclos, frecuencia y tiempo estimado . . . 53
4.3.2. Throughput pipelines . . . 54
4.3.3. Ocupancia . . . 59
4.3.4. Cantidad de instrucciones ejecutadas . . . 60
4.3.5. Memoria . . . 63
4.3.6. Throughput, sincronizaciones, warps activos . . . 64
4.3.7. Conclusión y análisis para longitud de secuencia 8192 . . . 67
4.4. Longitud de secuencia 16384 . . . 68
4.4.1. Ciclos, frecuencia y tiempo estimado . . . 68
4.4.2. Ocupancia . . . 69
4.4.3. Cantidad de instrucciones ejecutadas . . . 70
4.4.4. Memoria . . . 73
4.4.5. Throughput, sincronizaciones, warps activos . . . 75
4.4.6. Conclusión y análisis para longitud de secuencia 16384 . . . 76
4.5. Longitud de secuencia 32768 . . . 78
4.5.1. Ciclos, frecuencia y tiempo estimado . . . 78
4.5.2. Ocupancia . . . 79
4.5.3. Cantidad de instrucciones ejecutadas . . . 80
4.5.4. Memoria . . . 83

4.5.5. Throughput, sincronizaciones, warps activos 85
4.5.6. Conclusión y análisis para longitud de secuencia 32768 86
4.6. Longitud de secuencia 65536 87
4.6.1. Ciclos, frecuencia y tiempo estimado 87
4.6.2. Throughput pipelines 88
4.6.3. Ocupancia 92
4.6.4. Cantidad de instrucciones ejecutadas 93
4.6.5. Memoria 95
4.6.6. Throughput, sincronizaciones, warps activos 97
4.6.7. Conclusión y análisis de los kernels 98

5. Conclusiones **100**

Anexos **103**

A. Correspondencia entre magnitudes y métricas de NVIDIA Nsight Compute **104**
A.1. Tiempo y configuración de lanzamiento 105
A.2. Ocupancia 106
A.3. Instrucciones ejecutadas 106
A.4. Register spilling 107
A.5. DRAM y caché L2 107
A.6. Memoria compartida y políticas de caché 108
A.7. Emisión de instrucciones y motivos de stall 109

B. Qué quiere decir la fórmula de la atención **110**

C. NVIDIA Blackwell (SM100 y SM103) **112**
C.1. Una nota sobre Blackwell 114

D. Evitar cómputos innecesarios **115**

E. Caché L2 **117**

F. Repositorio del proyecto **118**

Bibliografía **119**

Índice de figuras

3.1. Layout A (M16×K16) para la instrucción `mma.sync`. 22
3.2. Mapping de warps: cada warp carga un tile Q y opera sobre varios tiles K_i. 24
3.3. Mapping de registros para elementos de Q (m16k128) usando layout A `mma.sync`. 28
3.4. Traspuesta en registros: idea recursiva. 31
3.5. Mapeado de lanes y fragmentos en la instrucción `ldmatrix.x4` para la carga de un tile 16×16 desde SRAM a registros con el layout requerido por `mma.sync`. 33
3.6. Comparación entre un layout naive y un layout swizzled en SRAM. El swizzle modifica la distribución de los elementos entre bancos para evitar conflictos de banco en el patrón de acceso considerado. 35
3.7. Cronograma de un multibuffer de dos etapas en SRAM. Tras la precarga inicial, una etapa puede reutilizarse para cargar los siguientes tiles K_i y V_i mientras se procesan los datos almacenados en la otra. En la imagen, solo aparecen las precargas de K para simplificar, pero en la práctica, un buffer lo forma un par (K_i, V_i) 36

4.1. Throughput de los pipelines de ejecución para CUDA con 8K, expresado como porcentaje del máximo sostenido durante los ciclos transcurridos. 54
4.2. Throughput de los pipelines de ejecución para cuDNN con 8K, expresado como porcentaje del máximo sostenido durante los ciclos transcurridos. 55
4.3. Throughput de los pipelines de ejecución para Triton con 8K, expresado como porcentaje del máximo sostenido durante los ciclos transcurridos. 56
4.4. Factor de crecimiento del número de ciclos al aumentar la longitud de secuencia de 8192 a 16384. 68
4.5. Factor de crecimiento del número de ciclos al aumentar la longitud de secuencia de 16384 a 32768. 78
4.6. Factor de crecimiento del número de ciclos al aumentar la longitud de secuencia de 32768 a 65536. 87
4.7. Throughput de los pipelines de ejecución para CUDA con 64K. 88
4.8. Throughput de los pipelines de ejecución para cuDNN con 64K. 89
4.9. Throughput de los pipelines de ejecución para SDPA con 64K. 90

4.10. Throughput de los pipelines de ejecución para Triton con 64K. 91
4.11. Sobrecoste temporal de cada implementación con respecto a la implementación más rápida para cada longitud de secuencia. Un valor de 0 % indica la implementación con menor tiempo de ejecución para dicha longitud. 98

5.1. Diagrama de decisión para la elección entre Torch, cuDNN, Triton y CUDA en función de los requisitos del proyecto. 101

Índice de tablas

3.1. Uso esperado de registros por thread. 27
3.2. Distribución de la diferencia en ULP entre `__expf()` y `poly2_exp` después de convertir los resultados a BF16. 40
3.3. Error de la implementación CUDA respecto a Torch SDPA para las distintas longitudes de secuencia. 43

4.1. Frecuencia del SM, ciclos transcurridos y tiempo de ejecución para una longitud de secuencia de 8192. 53
4.2. Tipos de operaciones ejecutadas por los pipelines de la NVIDIA RTX 3090 (Ampere, SM 8.6). 57
4.3. Tamaño del grid y número de warps lanzados para una longitud de secuencia de 8192. 59
4.4. Uso de registros y SRAM por bloque para una longitud de secuencia de 8192. 59
4.5. Ocupancia teórica, ocupancia alcanzada y número medio de warps activos por SM para una longitud de secuencia de 8192. 59
4.6. Número de instrucciones ejecutadas para una longitud de secuencia de 8192. 60
4.7. Número de instrucciones de tipo `SHFL` y porcentaje sobre el total para una longitud de secuencia de 8192. 61
4.8. Número de instrucciones de carga/almacenamiento y porcentaje sobre el total para una longitud de secuencia de 8192. 61
4.9. Número de instrucciones de punto flotante y porcentaje sobre el total para una longitud de secuencia de 8192. 61
4.10. Número de instrucciones Tensor Core (`HMMA`) y porcentaje sobre el total para una longitud de secuencia de 8192. 62
4.11. Número de instrucciones exponenciales (`MUFU.EX2`) y porcentaje sobre el total para una longitud de secuencia de 8192. 62
4.12. Número de instrucciones enteras y porcentaje sobre el total para una longitud de secuencia de 8192. 62
4.13. Volumen de datos leído y escrito en DRAM para una longitud de secuencia de 8192. 63

4.14. Volumen de datos leído y escrito en la caché L2 y tasa de aciertos para una longitud de secuencia de 8192. 63
4.15. Número de wavefronts teóricos y ejecutados en memoria compartida, conflictos de banco y proporción de conflictos para una longitud de secuencia de 8192. 64
4.16. Número de sectores accedidos en L2 mediante las políticas *evict first* y *evict last* para una longitud de secuencia de 8192. 64
4.17. Porcentaje de ciclos en los que se emitió una instrucción para una longitud de secuencia de 8192. En NCU, esta métrica corresponde a `smsp__issue_active.avg.pct_of_peak_sustained_active`. 64
4.18. Número medio de warp-ciclos de *stall* por instrucción emitida para diferentes causas de *stall*, junto con el porcentaje de ciclos con emisión, para una longitud de secuencia de 8192. 65
4.19. Porcentaje de warp-ciclos activos asociados a diferentes causas de *stall* para una longitud de secuencia de 8192. 66
4.20. Frecuencia del SM, ciclos transcurridos y tiempo de ejecución para una longitud de secuencia de 16384. 68
4.21. Tamaño del grid y número de warps lanzados para una longitud de secuencia de 16384. 69
4.22. Uso de registros y SRAM por bloque para una longitud de secuencia de 16384. 69
4.23. Ocupancia teórica, ocupancia alcanzada y número medio de warps activos por SM para una longitud de secuencia de 16384. 70
4.24. Número de instrucciones ejecutadas para una longitud de secuencia de 16384. 70
4.25. Número de instrucciones de tipo `SHFL` y porcentaje sobre el total para una longitud de secuencia de 16384. 71
4.26. Número de instrucciones de carga/almacenamiento y porcentaje sobre el total para una longitud de secuencia de 16384. 71
4.27. Número de instrucciones de punto flotante y porcentaje sobre el total para una longitud de secuencia de 16384. 71
4.28. Número de instrucciones Tensor Core (`HMMA`) y porcentaje sobre el total para una longitud de secuencia de 16384. 72
4.29. Número de instrucciones exponenciales (`MUFU.EX2`) y porcentaje sobre el total para una longitud de secuencia de 16384. 72
4.30. Número de instrucciones enteras y porcentaje sobre el total para una longitud de secuencia de 16384. 72
4.31. Accesos a memoria local asociados a *register spilling* para una longitud de secuencia de 16384. 73
4.32. Volumen de datos leído y escrito en DRAM para una longitud de secuencia de 16384. 73

4.33. Volumen de datos leído y escrito en la caché L2 y tasa de aciertos para una longitud de secuencia de 16384. 74
4.34. Número de wavefronts teóricos y ejecutados en memoria compartida, conflictos de banco y proporción de conflictos para una longitud de secuencia de 16384. 74
4.35. Número de sectores accedidos en L2 mediante las políticas *evict first* y *evict last* para una longitud de secuencia de 16384. 75
4.36. Porcentaje de ciclos con emisión de instrucciones (16384) 75
4.37. Porcentaje de warp-ciclos activos asociados a diferentes causas de *stall* para una longitud de secuencia de 16384. 76
4.38. Frecuencia del SM, ciclos transcurridos y tiempo de ejecución para una longitud de secuencia de 32768. 78
4.39. Tamaño del grid y número de warps lanzados para una longitud de secuencia de 32768. 79
4.40. Uso de registros y SRAM por bloque para una longitud de secuencia de 32768. 79
4.41. Ocupancia teórica, ocupancia alcanzada y número medio de warps activos por SM para una longitud de secuencia de 32768. 79
4.42. Número de instrucciones ejecutadas para una longitud de secuencia de 32768. 80
4.43. Número de instrucciones de tipo `SHFL` y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384. 80
4.44. Número de instrucciones de carga/almacenamiento y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384. 81
4.45. Número de instrucciones de punto flotante y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384. 81
4.46. Número de instrucciones Tensor Core (`HMMA`) y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384. 81
4.47. Número de instrucciones exponenciales (`MUFU.EX2`) y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384. 82
4.48. Número de instrucciones enteras y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384. 82
4.49. Volumen de datos leído y escrito en DRAM para una longitud de secuencia de 32768. 83
4.50. Aumento porcentual del volumen de datos leído y escrito en DRAM al pasar de una longitud de secuencia de 16384 a 32768. 83

4.51. Accesos a memoria local para una longitud de secuencia de 32768. . . . 84
4.52. Volumen de datos leído y escrito en la caché L2 y tasa de aciertos para una longitud de secuencia de 32768. 84
4.53. Porcentaje de ciclos en los que se emitió una instrucción para longitudes de secuencia de 32768 y 16384. En NCU, esta métrica corresponde a `smsp__issue_active.avg.pct_of_peak_sustained_active`. 85
4.54. Porcentaje de warp-ciclos activos asociados a diferentes causas de *stall* para una longitud de secuencia de 32768. 85
4.55. Frecuencia del SM, ciclos transcurridos y tiempo de ejecución para una longitud de secuencia de 65536. 87
4.56. Tamaño del grid y número de warps lanzados para una longitud de secuencia de 65536. 92
4.57. Uso de registros y SRAM por bloque para una longitud de secuencia de 65536. 92
4.58. Ocupancia teórica, ocupancia alcanzada y número medio de warps activos por SM para una longitud de secuencia de 65536. 92
4.59. Número de instrucciones ejecutadas para una longitud de secuencia de 65536. 93
4.60. Número de instrucciones de tipo `SHFL` y porcentaje sobre el total para una longitud de secuencia de 65536. 93
4.61. Número de instrucciones de carga/almacenamiento y porcentaje sobre el total para una longitud de secuencia de 65536. 93
4.62. Número de instrucciones de punto flotante y porcentaje sobre el total para una longitud de secuencia de 65536. 94
4.63. Número de instrucciones Tensor Core (`HMMA`) y porcentaje sobre el total para una longitud de secuencia de 65536. 94
4.64. Número de instrucciones exponenciales (`MUFU.EX2`) y porcentaje sobre el total para una longitud de secuencia de 65536. 94
4.65. Número de instrucciones enteras y porcentaje sobre el total para una longitud de secuencia de 65536. 95
4.66. Volumen de datos leído y escrito en DRAM para una longitud de secuencia de 65536. 95
4.67. Incremento porcentual del volumen total transferido a DRAM al duplicar la longitud de secuencia. 96
4.68. Accesos a memoria local asociados a register spilling para una longitud de secuencia de 65536. 96
4.69. Volumen de datos leído y escrito en la caché L2 y tasa de aciertos para una longitud de secuencia de 65536. 96
4.70. Porcentaje de ciclos en los que se emitió una instrucción para una longitud de secuencia de 65536, comparado con 32768. En NCU, esta métrica corresponde a `smsp__issue_active.avg.pct_of_peak_sustained_active`. 97

4.71. Porcentaje de warp-ciclos activos asociados a diferentes causas de *stall* para una longitud de secuencia de 65536. 97

5.1. Comparación de los tiempos de ejecución (ms) de las distintas implementaciones en función de la longitud de secuencia. Se destaca en negrita el menor tiempo para cada longitud. 100

A.1. Correspondencia entre las magnitudes de tiempo, lanzamiento y recursos y las métricas de NCU. 105
A.2. Correspondencia entre las magnitudes de ocupancia y las métricas de NCU. 106
A.3. Correspondencia entre las categorías de instrucciones estudiadas y las métricas SASS de NCU. 106
A.4. Métricas utilizadas para identificar accesos asociados al register spilling. 107
A.5. Correspondencia entre las magnitudes de tráfico de memoria y las métricas de NCU. 107
A.6. Métricas utilizadas para estudiar memoria compartida y políticas de eviction de L2. 108
A.7. Métricas utilizadas para estudiar la emisión de instrucciones y los principales motivos de stall. 109

Capítulo 1

Motivación y objetivos

1.1. Introducción

En computación de altas prestaciones hay dos maneras de conseguir un mayor rendimiento. La primera es mejorando el hardware: un hardware más rápido realiza las operaciones con mayor rapidez. La segunda es optimizando el programa: si las operaciones utilizan un 20 % de la capacidad de cómputo disponible y se puede incrementar ese porcentaje, se consigue un mayor rendimiento. Sin embargo, aunque a primera vista puede parecer que esta segunda opción es gratuita, la realidad es que aumentar el porcentaje de uso del hardware suele requerir mayores costes de ingeniería.

El motivo es que estructurar un programa para obtener un mayor rendimiento es, en sí mismo, una labor de ingeniería. Un programa es rápido porque utiliza mejor el hardware disponible, por lo que es indispensable conocer el hardware del que se dispone y el coste de las decisiones que se toman. Coste significa latencia, *throughput*, qué operaciones son críticas de optimizar —porque se utilizan con mayor frecuencia—, tiempo de investigación del hardware y coste de implementación. Coste también significa portabilidad del programa. Cuanto mayores sean las diferencias arquitecturales entre las máquinas, más difícil resulta establecer una equivalencia que permita un porteo sencillo.

El principal problema es que todos estos requisitos aparecen de forma simultánea: simultáneamente, es necesario conocer en detalle el hardware sobre el que se va a trabajar, el coste de las decisiones tomadas, los algoritmos que se van a implementar y estructurar el código de tal manera que permita dar soporte a cambios de funcionamiento a medida que avance el desarrollo y se introduzcan nuevos requisitos.

Las GPU (*Graphics Processing Units*), o tarjetas gráficas, presentan este mismo abanico de problemas. Provenientes del mundo de los gráficos y los videojuegos, donde los costes de optimizar un programa pueden marcar directamente la diferencia entre un juego que funciona y uno que no, se han popularizado en muchos otros ámbitos, especialmente como consecuencia del auge de la inteligencia artificial. Sin embargo, esto no significa que su utilidad se limite a este campo; al contrario, su uso puede resultar recomendable siempre que el volumen de datos a tratar sea masivo y el problema

permita aprovechar su elevado paralelismo.

Programar tarjetas gráficas presenta un coste importante, pero puede marcar la diferencia entre poder procesar un gran volumen de datos o directamente no poder hacerlo, por lo que, en muchos casos, ese coste resulta asumible. Este coste se debe, en parte, a que una CPU funciona de manera distinta a una GPU —aunque muchas de las optimizaciones también pueden beneficiar a ambas—, por lo que es necesario familiarizarse con su modelo de ejecución. A esto se añade la existencia de numerosas GPU de distintas generaciones y fabricantes, con diferencias en sus arquitecturas, capacidades y modelos de programación.

El entrenamiento de modelos de inteligencia artificial modernos requiere grandes cantidades de capacidad de cómputo, por lo que el coste asociado al uso de aceleradores constituye un factor relevante. En este contexto, mejorar la eficiencia de las operaciones fundamentales permite aumentar el rendimiento obtenido de un mismo dispositivo y reducir el tiempo de cómputo necesario para completar una determinada carga de trabajo.

El rendimiento de muchas aplicaciones modernas no está limitado únicamente por la capacidad de cómputo del procesador, sino también por la velocidad a la que los datos pueden moverse entre los distintos niveles de memoria. Este problema se conoce como *memory wall* [1]: la capacidad de cómputo ha aumentado a un ritmo superior al rendimiento del subsistema de memoria, haciendo que el movimiento de datos pueda convertirse en un factor limitante.

El mecanismo de atención utilizado por la arquitectura *Transformer* [2] presenta precisamente este tipo de problema. Una implementación ingenua materializa en memoria matrices intermedias de gran tamaño, generando un elevado tráfico de memoria. FlashAttention [3] reorganiza el cálculo para evitar la materialización de determinadas matrices intermedias y reducir los accesos a memoria.

El núcleo de este trabajo se centra precisamente en optimizar este mecanismo para la arquitectura NVIDIA Ampere. El mecanismo de atención es especialmente interesante porque permite mostrar muchos de los costes asociados a este tipo de programas: no basta con disponer de una gran capacidad de cómputo, sino que también es necesario organizar el algoritmo de manera que los datos puedan suministrarse eficientemente a las unidades de cálculo.

En nuestra implementación hacemos uso de las multiplicaciones matriciales aceleradas disponibles en Ampere, balanceamos la ocupancia, exploramos distintas instrucciones y optimizamos el cálculo de la función exponencial mediante una aproximación polinómica.

También implementamos una versión en Triton, que reduce considerablemente los costes de ingeniería asociados a la programación a bajo nivel. A lo largo de la memoria nos centramos principalmente en la optimización en CUDA para comprender mejor los costes mencionados anteriormente.

1.2. Motivación

La motivación principal es conocer cómo optimizar kernels de alto rendimiento en arquitecturas GPU actuales. Si bien es cierto que NVIDIA proporciona implementaciones altamente optimizadas de algunos de los kernels más utilizados, como la GEMM o el propio mecanismo de atención, no siempre existe una implementación específica para el problema que queremos resolver, especialmente cuando se pretende desarrollar una operación personalizada o fusionar varias operaciones en un mismo kernel.

Por ello, este Trabajo de Fin de Grado no se limita a comparar tiempos de ejecución y concluir que una versión es más rápida que otra. Se exploran cuestiones microarquitecturales con el objetivo de adquirir conocimientos que permitan, ante un problema futuro:

1. Decidir si dicha solución puede adaptarse eficientemente a una tarjeta gráfica.
2. Decidir cómo implementar esa misma solución para conseguir un alto rendimiento en la GPU.

La motivación puede resumirse como: **aprender principios de optimización de kernels que sigan siendo útiles cuando no exista una implementación de biblioteca que resuelva directamente el problema**.

1.3. Objetivos

Los objetivos de este Trabajo de Fin de Grado son:

O1. Implementar en CUDA el algoritmo FlashAttention forward no causal para unas longitudes de secuencia y dimensión de cabeza determinadas.

O2. Justificar las decisiones tomadas durante la implementación de O1.

O3. Implementar en Triton el algoritmo FlashAttention forward no causal para unas longitudes de secuencia y dimensión de cabeza determinadas.

O4. Comparar las diferencias de rendimiento entre las versiones CUDA, Triton, Torch SDPA y cuDNN SDPA y dar una explicación a los distintos resultados obtenidos.

Un objetivo adicional es mostrar al lector una comparación de la dificultad de programación entre CUDA y Triton. CUDA es un modelo de programación de bajo nivel que hace necesario conocer la arquitectura de la GPU al detalle. Triton, en cambio, abstrae gran parte del funcionamiento de las GPUs. Comprender un kernel Triton requiere un conocimiento menos detallado de determinados aspectos microarquitecturales de la GPU, mientras que un kernel CUDA requiere conocer no solo el algoritmo estudiado, sino factores complejos como la vida de los registros, distintas instrucciones a nivel de warp/CTA o el uso de los distintos niveles de memoria.

CUDA proporciona mayor libertad y permite aplicar optimizaciones de bajo nivel en etapas avanzadas del desarrollo. Triton, en cambio, proporciona menor libertad, pero permite obtener implementaciones de alto rendimiento con un menor tiempo de desarrollo.

Aunque la "dificultad de programación" es un concepto difícil de cuantificar objetivamente, la comparación de ambas implementaciones permitirá al lector valorar las diferencias en complejidad de desarrollo y control sobre el hardware.

1.4. Originalidad del TFG

La originalidad de este Trabajo de Fin de Grado no es el algoritmo "FlashAttention", sino cómo se implementa, optimiza y analiza:

1. En primer lugar, se implementa el mecanismo de atención en CUDA y Triton, y se compara con Torch SDPA y con cuDNN SDPA mediante NCU.

2. En segundo lugar, la implementación CUDA explica varias decisiones arquitecturales y optimizaciones de bajo nivel: número de buffers y el tamaño de los mismos, traspuesta de una matriz en registros, aproximación polinómica de la función exponencial con conteo de instrucciones y throughput de SASS, unrolling parcial y estudio de la ocupancia del kernel.

3. En tercer lugar, se hace una búsqueda de parámetros con Optuna. Entre los trabajos consultados para este TFG no se ha encontrado una metodología de búsqueda de parámetros similar aplicada a este tipo de kernels.

1.5. Organización del documento

El documento se estructura de la siguiente manera. El Capítulo 1 presenta la motivación del trabajo y sus objetivos. El Capítulo 2 introduce los trabajos relacionados, los conceptos necesarios para comprender la implementación y las principales herramientas utilizadas.

El Capítulo 3 describe el algoritmo FlashAttention, el desarrollo de las implementaciones CUDA y Triton, las optimizaciones aplicadas y la búsqueda de hiperparámetros. El Capítulo 4 presenta el análisis de rendimiento mediante NVIDIA Nsight Compute. Finalmente, el Capítulo 5 presenta las conclusiones obtenidas durante el desarrollo del trabajo, las limitaciones encontradas y posibles líneas de mejora.

Capítulo 2

Trabajo relacionado / Conceptos básicos

2.1. Trabajos relacionados

2.1.1. Writing Speed-of-Light Flash Attention for 5090 in CUDA C++

Thien Tran implementa una versión iterativa del kernel de atención en una RTX 5090 utilizando instrucciones Ampere [4].

Fija como parámetros:

Parámetro	**Valor**
Longitud de batch	1
Número de cabezas	8
Longitud de la consulta	4096
Longitud de la clave y de los valores	8192

Mientras que los kernels aquí descritos emplean:

Parámetro	**Valor**
Longitud de batch	1
Número de cabezas	1
Longitud de consulta, clave y valores	$\{8192, 16384, 32768, 65536\}$

A nivel del cargado de Q_r:

- Nuestro kernel CUDA carga el tile de Q_r directamente desde memoria principal y realiza una transposición en registros.
- El suyo transfiere Q_r a SRAM y de SRAM a registros.

A nivel de doble buffer:

- En nuestro kernel, un buffer lo forman K_i y V_i a la vez y hay posibilidad de multibuffer.
- En su kernel, hay tres buffers formados por dos tiles de K_i y uno de V_i, sin posibilidad de multibuffer.

Nuestro kernel CUDA computa una aproximación polinómica de la operación trascendental.

Una nota interesante es que Thien Tran carga Q_r en SRAM y después en registros. Acto seguido inicia las transferencias de K_i, K_{i+1} y V_i. Por tanto, no expone paralelismo entre la carga de Q_r y las cargas de K_i y V_i.

Repetir los experimentos a día de hoy en una RTX 5090 puede no dar resultados similares. El propio autor especifica que, en su versión de Torch, el backend de SDPA era FlashAttention-2 para Ampere.

2.1.2. Flash Attention From Scratch

Sonny Li implementa una versión iterativa del kernel de atención en una RTX 3090 y en una Ampere A100 [5]. Su implementación comparte varias ideas con la desarrollada en este trabajo, pero existen algunas diferencias importantes. En nuestro caso, el número de buffers se trata como un hiperparámetro, la función exponencial se sustituye por una aproximación polinómica, las matrices Q_r y O se trasponen en registros, la búsqueda de configuraciones se realiza mediante Optuna y el grado de *unrolling* también se ajusta de forma variable. Estas decisiones permiten explorar automáticamente distintas configuraciones del kernel en función de la longitud de secuencia y del comportamiento observado en cada caso.

2.1.3. GEMM

Armbruster presenta una guía extensa sobre cómo implementar una GEMM eficiente en CUDA sobre arquitecturas Ampere [6]. Describe técnicas como las cargas vectorizadas, el uso de instrucciones `mma`, doble buffer, *swizzling*, *padding* y `ldmatrix`, entre otras.

Dado que el kernel de atención contiene dos multiplicaciones matriciales, QK^T y PV, saber implementar una multiplicación de matrices eficiente es primordial. Muchas

de las técnicas empleadas en este trabajo para organizar los datos, moverlos entre memoria y registros y alimentar los Tensor Cores parten directamente de los mismos principios utilizados en una GEMM optimizada.

2.1.4. MLC.ai: Modern GPU Programming for MLSys

Este recurso de MLC.ai, basado en material de los cursos de Machine Learning Systems de Carnegie Mellon University, es uno de los recursos más completos para comprender la arquitectura de GPUs modernas [7]. Además de centrarse en Blackwell, explica conceptos que tienen continuidad con arquitecturas anteriores como Ampere y utiliza numerosos diagramas interactivos para mostrar la organización del hardware y el movimiento de los datos.

El recorrido finaliza con la implementación de FlashAttention-4 para una GPU con *compute capability* `SM_100`, aplicando técnicas como TMA, Tensor Memory, *warp specialization* y operaciones matriciales específicas de Blackwell.

Su lectura resulta especialmente recomendable como complemento a los conceptos tratados en esta memoria, debido a la claridad de sus diagramas y a la forma progresiva en la que relaciona la arquitectura de la GPU con la implementación de kernels de alto rendimiento.

2.1.5. FlashAttention

FlashAttention introduce una implementación exacta del mecanismo de atención diseñada teniendo en cuenta el coste de movimiento de datos entre los distintos niveles de memoria de la GPU [3]. La idea principal consiste en dividir las matrices Q, K y V en bloques que puedan procesarse utilizando memoria *on-chip*, evitando materializar completamente la matriz de atención QK^T en memoria global.

Para ello, el algoritmo procesa los bloques de K y V de forma iterativa y mantiene los valores necesarios para calcular el *softmax* mediante una actualización incremental del máximo y de la suma de cada fila. De esta forma se reduce considerablemente el tráfico con memoria global sin aproximar el resultado de la atención.

Esta organización constituye la base del kernel desarrollado en este trabajo: los bloques de Q permanecen durante varias iteraciones mientras se recorren los correspondientes bloques de K y V, intentando maximizar el reúso de los datos almacenados en registros y memoria compartida.

2.1.6. FlashAttention-2

FlashAttention-2 mantiene el mismo principio de reducción de accesos a memoria, pero modifica la distribución del trabajo para aumentar el paralelismo y reducir operaciones auxiliares que no utilizan Tensor Cores [8]. Una de sus principales aportaciones consiste en mejorar el reparto de la computación entre bloques y *warps*, reduciendo

sincronizaciones y haciendo que una mayor proporción del tiempo se dedique a las multiplicaciones matriciales.

También modifica la forma de paralelizar el cálculo de la atención, distribuyendo el trabajo de forma más eficiente entre los *warps* de un CTA. Estas ideas son especialmente relevantes para este trabajo, donde el número de *warps*, el reparto de las filas de Q_r y otras decisiones de paralelismo se tratan como parámetros que afectan directamente al rendimiento del kernel.

La implementación desarrollada parte del esquema de FlashAttention-2, pero añade decisiones propias de optimización, como un número variable de buffers, diferentes grados de *unrolling*, una aproximación polinómica de la exponencial y la búsqueda automática de configuraciones mediante Optuna.

2.2. Conceptos básicos

Este apartado recoge los conceptos y términos que se utilizarán a lo largo del trabajo. Se organiza en forma de glosario y los conceptos se ordenan alfabéticamente. Se espera que, en caso de duda durante la lectura del texto, el lector pueda consultar los conceptos aquí descritos y, en caso de necesitar más información, acudir a la documentación oficial correspondiente.

2.2.1. Acceso coalescente

Un acceso coalescente ocurre cuando los threads de un warp acceden a posiciones contiguas de memoria global, permitiendo agrupar los accesos en el menor número posible de transacciones a DRAM [9]. En dispositivos con compute capability 6.0 o superior, estas transacciones tienen una granularidad de 32 bytes [10].

Esto reduce el número de transferencias necesarias hacia memoria principal y permite amortizar la latencia asociada al acceso a las celdas DRAM, aprovechando la transferencia eficiente de posiciones consecutivas.

2.2.2. Conflictos de banco

La memoria compartida se divide en 32 bancos, de forma que palabras consecutivas de 32 bits se asignan a bancos consecutivos. Cuando varios lanes de un mismo warp acceden a direcciones distintas pertenecientes al mismo banco, se produce un *bank conflict* (conflicto de banco) y los accesos se serializan. Si acceden a bancos diferentes, los accesos pueden realizarse en paralelo. En el caso de una lectura de la misma dirección, el dato puede distribuirse mediante *broadcast* sin producir un conflicto de banco [11].

Los conflictos de banco se producen a nivel de warp.

2.2.3. CTA

Un *Cooperative Thread Array* (CTA) o bloque es un conjunto de threads que cooperan durante la ejecución de un kernel.

Todos los threads pertenecientes a un CTA se ejecutan en un único Streaming Multiprocessor y comparten los recursos asignados al bloque, entre ellos registros y memoria compartida [12]. Por tanto, un CTA solo puede ser asignado a un SM si este dispone de recursos suficientes para alojarlo completamente; un CTA no puede dividirse entre varios SM.

El número de CTAs que pueden residir simultáneamente en un SM depende de los recursos requeridos por cada CTA y de los recursos disponibles en el SM [13].

2.2.4. Double buffering

El *double buffering* es una técnica que utiliza dos buffers para solapar transferencia de datos y cómputo. Mientras se realizan operaciones sobre los datos almacenados en un buffer, el segundo puede utilizarse para cargar los datos necesarios para la siguiente iteración. Una vez finalizadas ambas operaciones, los buffers intercambian sus funciones [14].

2.2.5. Hiperparámetro

Un hiperparámetro es un parámetro de configuración cuyo valor se selecciona antes de ejecutar una implementación concreta. En este trabajo son hiperparámetros, entre otros, el número de warps, el número de buffers y el grado de *unrolling*.

2.2.6. ILP y TLP

ILP (*Instruction-Level Parallelism*) es el paralelismo entre instrucciones independientes de un mismo flujo de ejecución [15].

TLP (*Thread-Level Parallelism*) es el paralelismo obtenido mediante la ejecución concurrente de múltiples threads o warps [15].

2.2.7. L1

La caché L1 es el primer nivel de caché de datos accesible desde un SM. En Ampere `SM_86` forma parte de una memoria unificada compartida con la memoria compartida. Las líneas de caché tienen un tamaño de 128 bytes y se dividen en cuatro sectores de 32 bytes, que constituyen la granularidad mínima de transferencia [16].

PTX permite indicar distintas políticas de caché para las cargas. En particular, `.ca` permite utilizar L1 y L2, mientras que `.cg` evita la L1 y utiliza L2 y niveles posteriores [17].

2.2.8. L2

La caché L2 es un nivel de caché compartido por todos los SM de la GPU y se sitúa entre estos y la memoria principal. A diferencia de la L1, no pertenece a un SM concreto, por lo que puede almacenar datos reutilizados por distintos bloques y SM. Al igual que la L1, utiliza líneas de caché de 128 bytes divididas en cuatro sectores de 32 bytes [16].

2.2.9. Lane

Un *lane* es la posición que ocupa un thread dentro de un warp. Los lanes de un warp están numerados de 0 a 31 [18].

2.2.10. Latencia y throughput

La latencia es el tiempo necesario para completar una operación. El *throughput* indica cuántas operaciones pueden iniciarse o completarse por unidad de tiempo [15].

Una operación puede tener una latencia elevada y, al mismo tiempo, un throughput alto si existe suficiente paralelismo.

2.2.11. Loop unrolling

El *loop unrolling* es una optimización que consiste en reducir o eliminar las iteraciones explícitas de un bucle mediante la repetición de las instrucciones que forman su cuerpo. En CUDA, el compilador puede realizar esta transformación automáticamente para bucles pequeños cuyo número de iteraciones sea conocido en tiempo de compilación, o puede solicitarse mediante `#pragma unroll` [19, 20].

Esta optimización presenta varias ventajas:

1. Reduce las instrucciones asociadas al control del bucle, como la actualización del contador, la comprobación de la condición y el salto al inicio de la siguiente iteración.

2. Puede transformar índices de arrays conocidos en tiempo de compilación en accesos a registros independientes. Esto es especialmente relevante en CUDA, ya que los arrays cuyo índice no puede determinarse como constante pueden ser almacenados en memoria local en lugar de registros [21].

3. Expone simultáneamente al compilador las instrucciones pertenecientes a varias iteraciones, aumentando sus posibilidades de reordenarlas y explotar paralelismo a nivel de instrucción.

Sin embargo, un mayor nivel de desenrollado incrementa el tamaño del código generado y puede aumentar la presión sobre la caché de instrucciones y sobre los registros. Por este motivo, el grado óptimo de *unrolling* constituye un compromiso que depende del kernel y de la arquitectura.

2.2.12. Memoria compartida

La memoria compartida (*shared memory*) es una memoria ubicada físicamente dentro del Streaming Multiprocessor y accesible por todos los threads de un mismo CTA. Está destinada al intercambio y reutilización de datos entre los threads del bloque [11].

A diferencia de una caché, su contenido es gestionado explícitamente por el programador, por lo que NVIDIA la describe como una memoria *scratchpad* gestionada por el usuario.

En la arquitectura `SM_86` de la RTX 3090, cada SM dispone de hasta 100 KiB de memoria compartida. Sin embargo, un único CTA puede utilizar como máximo 99 KiB, ya que CUDA reserva 1 KiB por bloque [22].

2.2.13. Ocupancia

La ocupancia es el número de warps que se pueden asignar a un SM sobre el total permitido en esa GPU [13]. Para `SM_86`, en un SM caben 16 CTAs, cada uno con 1024 threads máximo, haciendo un total de 1536 threads (48 warps). La ocupancia también se ve afectada por la SRAM alojada por cada CTA [23].

Una mayor cantidad de warps asignados a un SM permite intercalar las operaciones entre los mismos. Si un warp ejecuta una operación costosa y tiene una dependencia real de datos, el Streaming Multiprocessor continúa ejecutando esa operación en segundo plano, mientras un nuevo warp, con una nueva instrucción, es encolado, consiguiendo paralelismo a nivel de instrucciones.

Sin embargo, es difícil saber con qué ocupancia rinde mejor un kernel. Puede ser que el kernel esté limitado por la memoria y que se beneficie de mayor ocupancia para enmascarar las latencias; puede ser también que un único warp sature ya los pipelines que actualmente formen el cuello de botella del kernel (como las instrucciones de multiplicación matricial, que se procesan en el pipeline de Tensor Cores [16]) y que no compense aumentar el número de warps, o que simplemente sí lo haga pero que no exista una manera directa de cuantificar cuánto mejor va a ser nuestro kernel.

Si aumentamos el número de warps por CTA, pero el tiempo medio de espera en barreras es mayor, entonces no compensa una mayor ocupancia.

Si reducimos la ocupancia pero, a cambio, necesitamos más accesos a SMEM porque no podemos reutilizar los datos, podemos perder rendimiento. También puede ocurrir que el número de registros empleados por thread sea un factor limitante y que haya que acceder a DRAM para conseguir una mayor ocupancia. Esos accesos a memoria pueden compensar la ocupancia, pero también perjudicar el rendimiento.

Trade-offs relacionados con la ocupancia

A continuación, una serie de trade-offs con la ocupancia.

- **Reducir registros mediante recómputo.**

Es una de las alternativas con menor impacto sobre el rendimiento de una aplicación. El warp recalcula una operación múltiples veces para favorecer que la latencia de operaciones más caras se compense con los warps nuevos.

- **Reducir los registros mediante un particionado de trabajo.**

  Es difícil de calibrar. Si un warp realiza cierta cantidad de trabajo y podemos reducirla sin aumentar la sincronización entre los threads, entonces la cantidad de registros se reduce proporcionalmente y obtenemos nuevos niveles de paralelismo.

  Si el warp realiza cierta cantidad de trabajo y podemos reducirla pero a costa de aumentar la sincronización entre los warps, entonces la pregunta es si ese nivel añadido de paralelismo compensa la sincronización; ganaremos rendimiento si lo hace, si no, lo perderemos aun habiendo aumentado la ocupancia y el paralelismo.

- **Forzar spills a DRAM.**

  Es una de las alternativas menos recomendables. Fuerza que los registros sobrantes para conseguir la ocupancia sean volcados a memoria principal y sean recuperados de la misma cada vez que se usen. En pocas ocasiones un kernel aumenta su rendimiento si aumenta la ocupancia a costa de DRAM spills.

- **Forzar spills a SRAM.**

  Es una opción relativamente novedosa (CUDA 13.0). Los registros se vuelcan a SRAM, que tiene menor latencia, en vez de a DRAM.

- **Aumentar el número de warps por CTA.**

  Puede ser beneficioso porque puede reutilizar los datos de SRAM a nivel de bloque. Si no hay `__syncthreads()`, entonces el beneficio es prácticamente gratuito.

  Si hay `__syncthreads()`, entonces puede reducirse el rendimiento. Cuantos más warps llegan a la barrera, menos hay disponibles para ejecutar.

  Si se compensa el número de warps aumentando el número de CTAs, menor es el coste de sincronización de los mismos, a costa de requerir más SRAM. Más CTAs por SM en vez de más warps por SM también realizan más accesos a memoria.

2.2.14. Política de eviction

Una política de *eviction* indica al sistema de caché qué datos deberían conservarse o expulsarse con mayor prioridad [17].

En este trabajo se utilizan las políticas `evict_first` y `evict_last`, que favorecen respectivamente la expulsión temprana o la conservación de los datos en caché.

2.2.15. Predicación

La predicación permite ejecutar una instrucción de forma condicional sin introducir necesariamente un salto de control. Una instrucción predicada solo produce efecto en los lanes cuyo registro de predicado asociado es verdadero [24].

2.2.16. Registros

Los registros son memoria ubicada dentro del Streaming Multiprocessor y tienen alcance local a cada thread. El compilador gestiona su asignación y los utiliza para almacenar variables locales durante la ejecución de un kernel [25].

El fichero de registros está formado por registros de 32 bits. En la arquitectura `SM_86` utilizada por la RTX 3090, cada Streaming Multiprocessor dispone de 65536 registros de 32 bits, equivalentes a 256 KiB [22].

El número de registros utilizado por cada thread es determinado durante la compilación del kernel. Este consumo afecta al número de warps y CTAs que pueden residir simultáneamente en un SM y, por tanto, puede limitar la ocupancia.

2.2.17. SASS y PTX

El ensamblador de NVIDIA es SASS, que es el resultado de compilar una representación intermedia conocida como PTX. Esto permite a NVIDIA esconder los detalles cambiantes de la implementación de la GPU, mientras que permite al programador trabajar sobre una representación intermedia. Si el binario CUDA incluye un código PTX, esto permite a NVIDIA hacer un JIT-compilation ajustado a las peculiaridades del hardware. Es importante remarcar que PTX no es lo que ejecuta la GPU. Una instrucción PTX puede compilarse a varias instrucciones SASS [17].

2.2.18. Spills

Se dice que hay spills cuando el compilador agota el espacio de registros de un thread y tiene que volcar contenidos de los mismos forzosamente a DRAM o SRAM para dar espacio a otras variables [21, 26].

2.2.19. Streaming Multiprocessor

El *Streaming Multiprocessor* (SM) es la unidad de procesamiento encargada de ejecutar los CTAs o *thread blocks*. Todos los threads pertenecientes a un mismo CTA se ejecutan en un único SM [12].

Cada SM dispone de una cantidad limitada de recursos, entre ellos registros y memoria compartida, así como límites sobre el número de warps y CTAs que pueden residir simultáneamente en él. El número de CTAs y warps residentes depende tanto de estos límites arquitectónicos como de los recursos consumidos por el kernel [15].

2.2.20. Swizzle

Un *swizzle* es una permutación del mapeo entre índices lógicos y direcciones físicas con el objetivo de distribuir accesos que producirían conflictos de banco entre distintos bancos de memoria compartida. Un patrón habitual es el *XOR swizzle* [27].

2.2.21. Tile

Un *tile* o tesela es un fragmento de una matriz o conjunto de datos que se procesa como una unidad. Puede almacenarse temporalmente en registros o memoria compartida durante su procesamiento [9].

2.2.22. TPE multivariante

El *Tree-structured Parzen Estimator* (TPE) es un método de optimización utilizado por Optuna para seleccionar nuevas configuraciones de hiperparámetros a partir de los resultados obtenidos en evaluaciones anteriores. En lugar de recorrer exhaustivamente todas las combinaciones posibles, construye modelos probabilísticos que distinguen entre las configuraciones que han producido buenos resultados y el resto, favoreciendo posteriormente el muestreo de regiones prometedoras del espacio de búsqueda [28].

En su versión multivariante, los hiperparámetros no se modelan de forma completamente independiente, sino conjuntamente. Esto permite capturar relaciones entre ellos. En el contexto de este trabajo resulta especialmente útil, ya que parámetros como el número de *warps*, el número de buffers o el grado de *unrolling* pueden interactuar entre sí y no necesariamente tienen un efecto independiente sobre el rendimiento del kernel.

2.2.23. ULP

ULP (*Unit in the Last Place*) representa la distancia entre dos números de punto flotante representables consecutivos alrededor de un determinado valor [29].

Por ejemplo, en BF16 el número representable inmediatamente superior a 1 es 1,0078125, obtenido al incrementar el último bit de la mantisa. Ese incremento es lo que corresponde a 1 ULP.

2.2.24. Warp

Un *warp* es un grupo lógico de 32 threads que constituye la unidad básica de planificación y ejecución SIMT en una GPU NVIDIA. Los threads de un mismo warp ejecutan una instrucción común sobre los lanes activos, aunque pueden divergir y seguir caminos de ejecución distintos.

En las arquitecturas CUDA actuales, el tamaño de un warp es de 32 threads [18].

2.2.25. Warp divergence

La *warp divergence* ocurre cuando distintos lanes de un mismo warp siguen caminos de control diferentes. En ese caso, el warp ejecuta los distintos caminos necesarios, activando únicamente los lanes que corresponden a cada uno de ellos [18].

2.2.26. Wavefront

Un *wavefront* es la máxima unidad de trabajo de una petición de memoria que puede procesarse en paralelo durante un ciclo. En memoria compartida, un acceso sin conflictos puede transferir hasta 128 bytes por *wavefront*, correspondientes a 32 bancos de 4 bytes cada uno. Si una petición requiere varios *wavefronts*, estos se procesan de forma serializada [16].

2.3. Herramientas

2.3.1. CUDA 13.0 & C++20

Este proyecto necesita una versión CUDA 13.0 o superior, y ha sido desarrollado usando CUDA 13.0.

El motivo es la directiva PTX `.pragma "enable_smem_spilling";`. En la iteración final del algoritmo, cuando se emplea Optuna para calibrar los distintos parámetros de la configuración, podemos forzar los spills a SRAM en vez de a DRAM. Esta característica se introduce en CUDA 13.0 [26].

La representación de determinadas constantes de tipo `float` mediante su patrón de bits en un `uint32_t` proviene del diseño inicial de la implementación. En C++17 no era posible utilizar valores de punto flotante como *non-type template parameters*, por lo que se optó por representar la constante como un entero de 32 bits (*bitcast*).

En la versión actual del proyecto se utiliza C++20, que sí permite parámetros de plantilla de punto flotante. Sin embargo, se ha mantenido la opción *bitcast*, utilizando `std::bit_cast` para obtener el patrón de bits del `float` de forma explícita:

```
std::bit_cast<uint32_t>(0.08838834764831845f) // 1/sqrt(128)
```

Dentro del kernel, podemos reinterpretar esos bits como un `float` con `__uint_as_float()`.

2.3.2. NVCC

El compilador utilizado es NVCC. Se asigna automáticamente por CMake. Los flags utilizados han sido `-O3` y `-Xptxas=-v` para ver los registros consumidos, SRAM y *spills*. Este último flag ayuda a identificar si el compilador falla al desenrollar un bucle, si se hacen accesos no pretendidos a DRAM, etc.

2.3.3. CMake

El gestor de proyecto utilizado ha sido CMake. Permite una integración sencilla con Torch y con Google Test.

Se utilizan *presets*, uno para cada longitud de secuencia. De esta manera, es posible tener una configuración distinta para cada variación en los datos de entrada.

2.3.4. uv

`uv` es el gestor de paquetes de Python. Destaca por su rapidez y sencillez de uso.

2.3.5. Pytest

Pytest es el entorno de pruebas utilizado para Python y, conforme ha avanzado el proyecto, también para CUDA. Se utilizan tests de correctness y tests no funcionales para medir el rendimiento mediante NCU, principalmente.

2.3.6. Triton

Triton es un *Domain Specific Language* (DSL) basado en Python para programar GPUs de última generación. Permite desarrollar código portable entre GPUs de distintas familias y/o marcas [30] con reescrituras mínimas. Es de código abierto y ha sido desarrollado principalmente por OpenAI [31]. Contiene un autotuner propio [32] y ofrece una integración directa con Python.

2.3.7. NCU

NVIDIA Nsight Compute (NCU) es el profiler empleado para analizar el rendimiento de los kernels CUDA. Permite obtener métricas relacionadas con la ocupancia, la actividad de los warps, el uso de las unidades de ejecución y el comportamiento de la jerarquía de memoria.

En este proyecto se utiliza principalmente para identificar cuellos de botella y estudiar el efecto de las distintas optimizaciones aplicadas al kernel. También permite analizar aspectos como los accesos a memoria compartida, los conflictos de banco y el número de wavefronts generados.

2.3.8. Optuna

Optuna es el autotuner empleado para ajustar los parámetros del kernel CUDA. Permite explorar automáticamente distintas configuraciones del kernel y seleccionar aquellas que ofrecen un mejor rendimiento.

En este proyecto se utiliza para ajustar parámetros como el número de warps, el número de CTAs por SM, el número de buffers, el tipo de acceso a memoria y el grado de unrolling.

Capítulo 3

Metodología de desarrollo

3.1. Algoritmo general FlashAttention-2

Sean las matrices

$$Q, K, V \in \mathbb{R}^{N \times d}, \tag{3.1}$$

donde N es la longitud de secuencia y d la dimensión de cada cabeza de atención.

Las matrices se dividen a lo largo de la dimensión de secuencia en bloques de B_r filas. Se define

$$T_r = \frac{N}{B_r}. \tag{3.2}$$

Por tanto,

$$Q_1, \ldots, Q_{T_r}, \qquad K_1, \ldots, K_{T_r}, \qquad V_1, \ldots, V_{T_r}, \tag{3.3}$$

con

$$Q_i, K_i, V_i \in \mathbb{R}^{B_r \times d}. \tag{3.4}$$

A su vez, cada uno de estos bloques se divide a lo largo de la dimensión de cabeza d en subbloques de B_c columnas. Se define

$$T_c = \frac{d}{B_c}. \tag{3.5}$$

Por ejemplo, cada bloque K_i se divide como

$$K_i = \begin{bmatrix} K_{i,1} & K_{i,2} & \cdots & K_{i,T_c} \end{bmatrix}, \tag{3.6}$$

donde

$$K_{i,j} \in \mathbb{R}^{B_r \times B_c}. \tag{3.7}$$

De forma análoga,

$$Q_{i,j}, V_{i,j} \in \mathbb{R}^{B_r \times B_c}. \tag{3.8}$$

De esta forma, el producto entre un bloque Q_r y un bloque K_i puede calcularse acumulando los productos de sus subbloques:

$$Q_r K_i^T = \sum_{j=1}^{T_c} Q_{r,j} K_{i,j}^T. \tag{3.9}$$

A partir de estas expresiones, el pseudocódigo del algoritmo se muestra en la página siguiente.

Algorithm 1 Algoritmo general FlashAttention-2

Require: $Q, K, V \in \mathbb{R}^{N \times d}$, B_r, B_c
Ensure: $O = \text{softmax}(QK^T/sqrt(d))V$

$T_r \leftarrow N/B_r$ ▷ Número de bloques en la dimensión de secuencia
$T_c \leftarrow d/B_c$ ▷ Número de subbloques en la dimensión de cabeza
for $r = 1, \ldots, T_r$ **do**
  Cargar Q_r desde DRAM a memoria on-chip ▷ $Q_r \in \mathbb{R}^{B_r \times d}$
  $O_r^{(0)} \leftarrow 0$ ▷ Acumulador de la salida
  $l_r^{(0)} \leftarrow 0$ ▷ Denominador acumulado del softmax
  $m_r^{(0)} \leftarrow -\infty$ ▷ Máximo actual de cada fila
  for $i = 1, \ldots, T_r$ **do**
    Cargar K_i y V_i desde DRAM a memoria on-chip ▷ $K_i, V_i \in \mathbb{R}^{B_r \times d}$
    $S_{r,i} \leftarrow 0$ ▷ $S_{r,i} \in \mathbb{R}^{B_r \times B_r}$
    for $j = 1, \ldots, T_c$ **do**
      Obtener $Q_{r,j}$ ▷ $Q_{r,j} \in \mathbb{R}^{B_r \times B_c}$
      Obtener $K_{i,j}$ ▷ $K_{i,j} \in \mathbb{R}^{B_r \times B_c}$
      $S_{r,i} \leftarrow S_{r,i} + Q_{r,j}K_{i,j}^T$
      ▷ Acumular el producto a lo largo de d
    end for
    $S_{r,i} \leftarrow \frac{1}{\sqrt{d}} S_{r,i}$
    $m_r^{(i)} \leftarrow \text{máx}\left(m_r^{(i-1)}, \text{rowmax}(S_{r,i})\right)$
    ▷ Actualizar el máximo de cada fila
    $\alpha_r^{(i)} \leftarrow \exp\left(m_r^{(i-1)} - m_r^{(i)}\right)$
    ▷ Factor de reescalado de resultados anteriores
    $P_{r,i} \leftarrow \exp\left(S_{r,i} - m_r^{(i)}\right)$
    ▷ Calcular las exponenciales del bloque actual
    ▷ La resta de $m_r^{(i)}$ se realiza de forma independiente para cada fila
    $l_r^{(i)} \leftarrow \alpha_r^{(i)} l_r^{(i-1)} + \text{rowsum}\left(P_{r,i}\right)$
    ▷ Actualizar el denominador acumulado
    $O_r^{(i)} \leftarrow \alpha_r^{(i)} O_r^{(i-1)} + P_{r,i}V_i$
    ▷ Actualizar el acumulador de salida
  end for
  for $k = 0, \ldots, B_r - 1$ **do**
    $O_r[k,:] \leftarrow \dfrac{O_r^{(T_r)}[k,:]}{l_r^{(T_r)}[k]}$
    ▷ Normalizar cada fila
  end for
  Escribir O_r en DRAM ▷ Bloque correspondiente de la matriz de salida
end for
return O

El procedimiento se repite para todos los bloques de Q, obteniendo finalmente

$$O = \text{softmax}(QK^T/sqrt(d))V. \tag{3.10}$$

La formulación anterior esconde los detalles de la GPU. Describe qué operaciones deben realizarse, pero no cómo deben ejecutarse sobre la arquitectura.

Por este motivo, es necesario desarrollar los parámetros de la implementación para estudiar cómo encajan en la GPU y qué configuración aprovecha mejor los recursos disponibles.

A lo largo de los siguientes apartados se describen las principales instrucciones y aspectos de hardware necesarios para entender la implementación del kernel.

3.2. Instrucción `mma.sync`

La familia de instrucciones PTX encargadas de acelerar la multiplicación de matrices en Ampere se denomina **mma.sync**.

Dado que los parámetros que utilizamos son BF16, la instrucción empleada es:

`mma.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32` [17]

Esta instrucción multiplica una matriz A en `bf16` de dimensiones 16×16 por una matriz B en `bf16` de dimensiones 16×8, suma una matriz C en `fp32` de dimensiones 16×8 y sobrescribe el resultado en C, también en `fp32`.

$$\underbrace{A_{16\times16}}_{\texttt{bf16}}\ \underbrace{B_{16\times8}}_{\texttt{bf16}} + \underbrace{C_{16\times8}}_{\texttt{fp32}} \longrightarrow \underbrace{C_{16\times8}}_{\texttt{fp32}} \tag{3.11}$$

Operando	Dimensiones	Tipo
A	16×16	`bf16`
B	16×8	`bf16`
C	16×8	`fp32`

No existe en Ampere una instrucción que permita multiplicar matrices de mayores dimensiones.

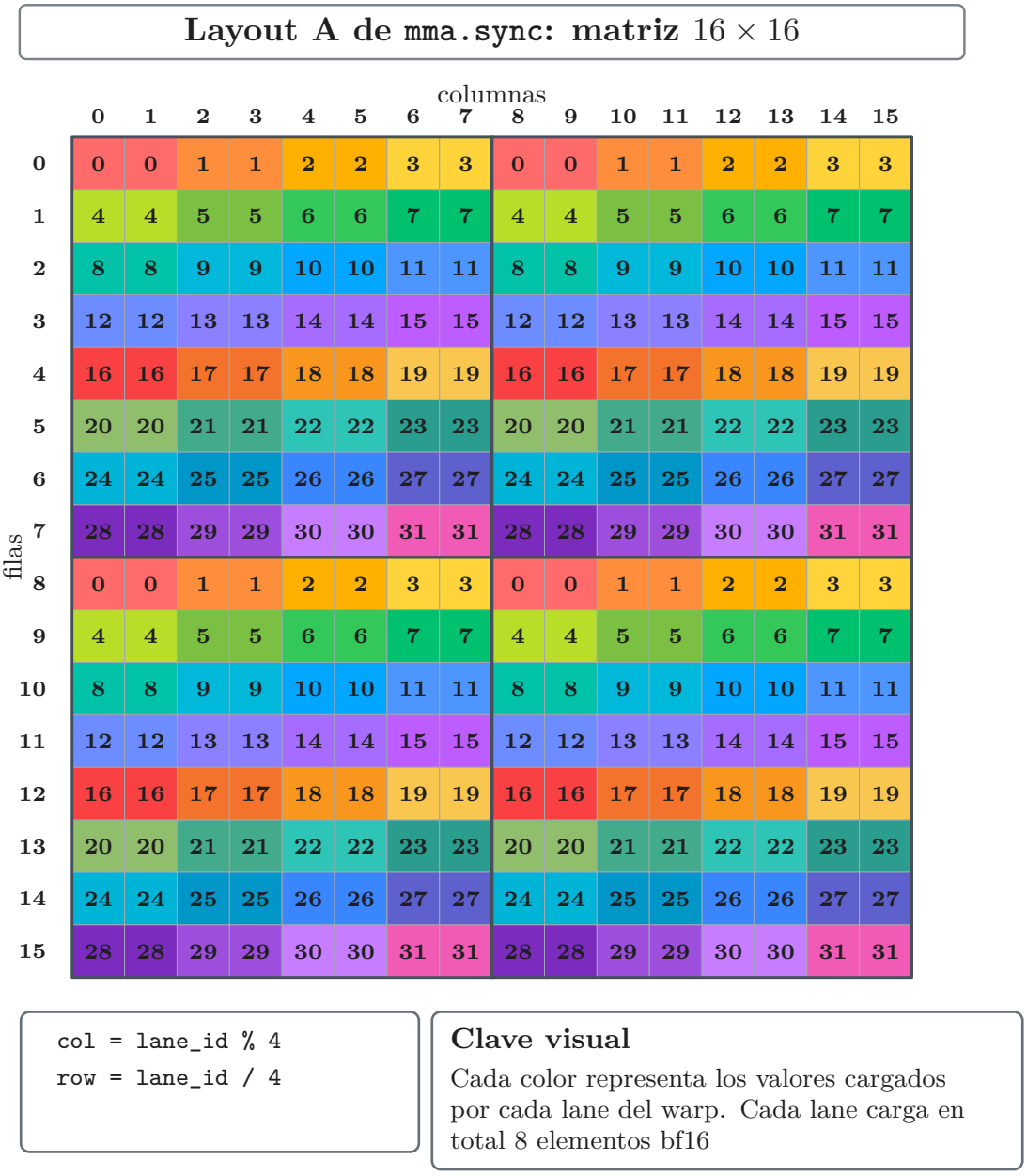


Figura 3.1. Layout A (M16×K16) para la instrucción `mma.sync`.

La distribución descrita se muestra en la Figura 3.1.

El resultado de esta instrucción es problemático porque, si se quiere aprovechar la instrucción, no se puede establecer $B_r = 8$, $B_c = 16$ y calcular directamente

$$\sum_j Q_i K_{i,j}^T, \tag{3.12}$$

porque el resultado S tendría dimensiones $M16N8$.

Si S tiene dimensiones $M16N8$, no se puede utilizar directamente para multiplicar los elementos $V_{i,j}$ de dimensiones $M16K8$, porque no existe una instrucción de la familia que permita realizar esa operación.

En la implementación se fijan $B_r = B_c = 16$. Sin embargo, para este caso, la dimensión B_c se divide en dos, de modo que cada subbloque $K_{i,j} \in \mathbb{R}^{16\times16}$ se divide en dos subtiles 16×8, manteniendo dos acumuladores S independientes, ambos de dimensiones $M16N8$.

- El primer acumulador contiene el resultado de

$$S_{\text{par}} = \sum_{\substack{j \\ j \text{ par}}} Q_i K_{i,j}^T. \tag{3.13}$$

- El segundo acumulador contiene el resultado de

$$S_{\text{impar}} = \sum_{\substack{j \\ j \text{ impar}}} Q_i K_{i,j}^T. \tag{3.14}$$

Finalmente, ambas matrices se concatenan:

$$S = \begin{bmatrix} S_{\text{par}} & S_{\text{impar}} \end{bmatrix}, \tag{3.15}$$

obteniendo una matriz S de dimensiones $M16K16$, que sí puede emplearse posteriormente para multiplicar V.

$$S_{\text{par}}\,(M16N8) \quad \textit{concat} \quad S_{\text{impar}}\,(M16N8) \longrightarrow S\,(M16K16) \longrightarrow S\,V \tag{3.16}$$

3.3. Decisiones de paralelismo y warp coarsening

Se paraleliza a través de la longitud de secuencia.

Paralelizar a través de la dimensión de cabeza tiene como ventaja disponer de más cómputo matricial por separado, pero presenta varios costes:

- Es necesario sincronizar (SRAM) los resultados parciales de la matriz S.
- Se produce trabajo desigual por parte de los threads durante el cálculo de la matriz P, porque solo un warp debe calcular las exponenciales, restas, máximos, etc.
- Es necesario sincronizar la matriz P para que los otros warps puedan obtenerla y computar $P_i \cdot V_{i,j}$.
- Finalmente, se necesita otra sincronización para combinar los resultados parciales de $P \cdot V_{i,j}$.
- Además, dependiendo de la granularidad, $Q_{i,j}$ y $O_{i,j}$ podrían no cumplir el cupo de 32 bytes por carga/almacenamiento, que es el número que determina NVIDIA para accesos coalesced [10].

Para una dimensión de cabeza (d) de 128, se considera que los costes de paralelizar a través de la dimensión de cabeza exceden los beneficios del paralelismo añadido.

El número de rows que procesa un warp se fija en 16, porque es la dimensión que tiene la matriz A (`m16k16`) de `mma.sync`.

Procesar un múltiplo de esas 16 filas multiplica el número de registros empleados por warp por la misma cantidad y reduce el nivel de paralelismo.

Aumentar el nivel de paralelismo implica que, si el hardware mejora, aumenta el ancho de banda de la DRAM y aumenta el número de SMs, se puede aprovechar más.

Por estos motivos, se considera que 16 rows es el número que maximiza el paralelismo y la ocupancia para esta implementación.

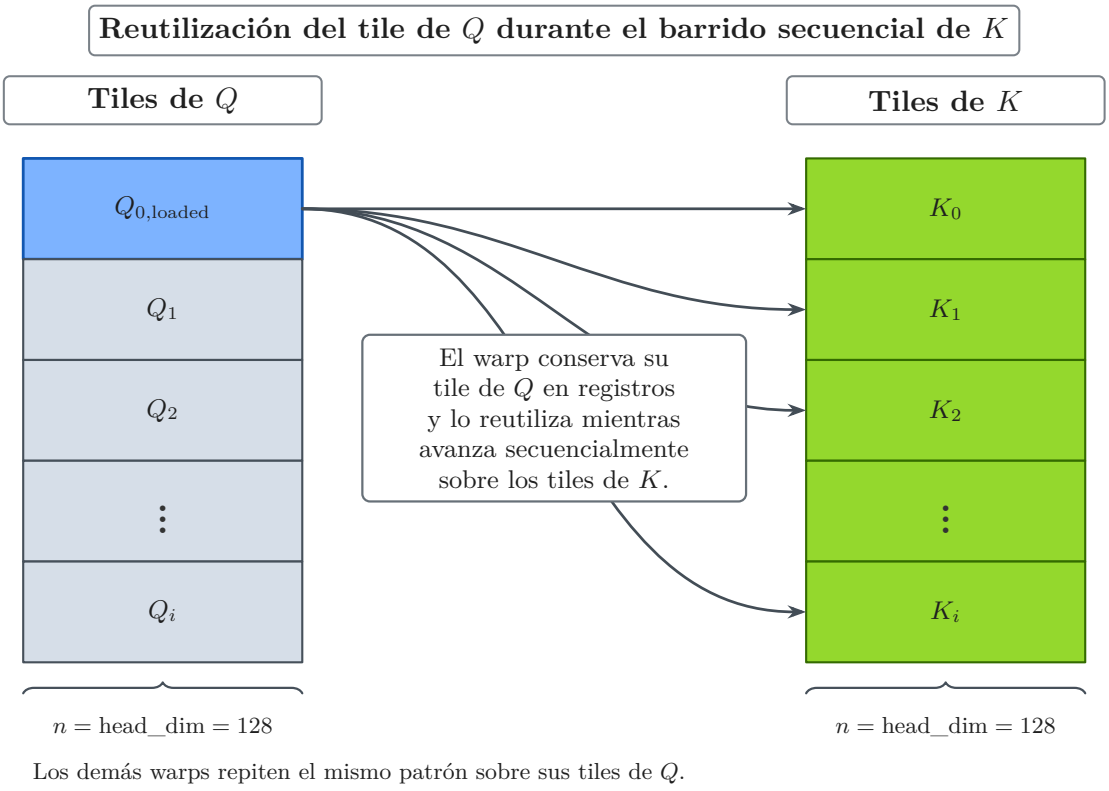


Figura 3.2. Mapping de warps: cada warp carga un tile Q y opera sobre varios tiles K_i.

El reparto descrito se muestra en la Figura 3.2.

3.4. Uso de registros y SRAM

El uso de registros y SRAM del kernel por los componentes principales es el siguiente.

Tile Q_r

El tile $Q_r \in \mathbb{R}^{16\times128}$ del tipo `bfloat16` requiere:

$$16 \times 128 \text{ bfloat16} \times \frac{1 \text{ registro}}{2 \text{ bfloat16}} = \underline{\textbf{1024 registros}}. \tag{3.17}$$

Por thread:

$$1024 \text{ registros} \times \frac{1 \text{ warp}}{32 \text{ threads}} = \underline{\textbf{32 registros/thread}}. \tag{3.18}$$

El tile Q se carga directamente desde la memoria principal a registros.

Tiles K_i y V_i

Los tiles $K_i, V_i \in \mathbb{R}^{16\times128}$, del tipo `bfloat16`, se cargan en SRAM y, a continuación, en registros.

$$2 \times 16 \times 128 \text{ bfloat16} \times \frac{2 \text{ bytes}}{1 \text{ bfloat16}} = \underline{\textbf{8192 bytes}}. \tag{3.19}$$

El uso de `ldmatrix.x4` carga dos tiles de $16 \times 8 \times 2$.
El doble buffer SRAM-registros requiere

$$16 \times 8 \times 2 \times 2 \tag{3.20}$$

elementos de espacio en registros.

$$16 \times 8 \times 2 \times 2 \text{ elementos} \times \frac{1 \text{ registro}}{2 \text{ bfloat16}} = \underline{\textbf{256 registros}}. \tag{3.21}$$

Por thread:

$$256 \text{ registros} \times \frac{1 \text{ warp}}{32 \text{ threads}} = \underline{\textbf{8 registros/thread}}. \tag{3.22}$$

En la práctica, cuando terminamos de procesar K_i, V_i puede reutilizar ese mismo espacio en registros.

Matriz de scores $S_{r,i}$

La matriz de scores $S_{r,i}$, resultado de $Q_r K_i^T$, requiere:

$$16 \times 16 \text{ FP32} \times \frac{1 \text{ registro}}{1 \text{ FP32}} \times \frac{1 \text{ warp}}{32 \text{ threads}} = \underline{\textbf{8 registros/thread}}. \tag{3.23}$$

Matriz $P_{r,i}$

La matriz $P_{r,i}$ requiere:

$$16 \times 16 \text{ bfloat16} \times \frac{1 \text{ registro}}{2 \text{ bfloat16}} \times \frac{1 \text{ warp}}{32 \text{ threads}} = \underline{\textbf{4 registros/thread}}. \tag{3.24}$$

En la práctica, se deben reutilizar los registros de $S_{r,i}$.

Máximo por row

Dado el formato de $A \in \mathbb{R}^{16\times16}$, cada thread almacena 4 registros correspondientes a tiles de 8×8.

El máximo por row y por thread ocupa entonces **2 registros**.

El máximo por row anterior es idéntico al máximo por row y thread, que son **2 registros**.

Row denominator

El row denominator ocupa **2 registros**.

El row denominator anterior ocupa también **2 registros**.

Output accumulator

El output accumulator requiere:

$$16 \times 128 \text{ elementos FP32} \times \frac{1 \text{ registro}}{1 \text{ FP32}} \times \frac{1 \text{ warp}}{32 \text{ threads}} = \textbf{64 registros/thread}. \quad (3.25)$$

Uso total esperado de registros

El uso esperado de los registros es mantener Q, output accumulator, máximo por row, máximo por row anterior, row denominator, row denominator anterior, tiles de K/V y matriz S en registros.

La matriz P se espera que utilice registros de S. La matriz S consume registros adicionales porque no puede utilizar los del output accumulator.

Componente	Registros/thread
Q_r	32
O_r	64
$S_{r,i}$	8
K/V	8
$m_r^{(i)}/m_r^{(i-1)}$	2
$l_r^{(i)}/l_r^{(i-1)}$	2
`scale_factor`	2
`row_sum`	2
Total	**120**

Tabla 3.1. Uso esperado de registros por thread.

La cifra de 120 registros se interpreta de la siguiente manera:

Si se utilizan menos de 120 registros, datos que son reutilizados múltiples veces, como O_r y Q_r, deben ser almacenados en memoria local/SRAM y recargados cuando se usen, un proceso que puede acabar resultando costoso.

El compilador puede decidir optimizar operaciones y almacenar el resultado en registros, utilizando más de 120 registros por thread. Este tipo de operaciones pueden ser de indexación, por ejemplo.

Es pronto para afirmar que una cuenta de registros menor o mayor que 120 aumente o disminuya el rendimiento. Una menor cantidad de registros implica más operaciones de memoria, pero también una mayor ocupancia. En ese caso, la SM puede intercalar esas operaciones de memoria con instrucciones procedentes de otros warps residentes en la máquina.

Más registros pueden reducir el número de warps que residen en un SM y, con ello, reducir la variedad de operaciones, dificultando esconder la latencia si es necesario.

3.5. Cargado Q_r

Cargar Q_r presenta numerosas opciones. Q_r se reutiliza durante cada iteración sobre los bloques K_i y V_i. Es lógico que el factor que más se reutiliza se almacene en la memoria de mayor velocidad, es decir, los registros. Hay múltiples maneras de cargar Q_r, cada una con sus ventajas e inconvenientes.

El problema consiste, por tanto, en decidir cómo trasladar Q_r desde memoria global hasta los registros y en qué formato realizar dicha carga.

La manera más directa de cargar Q_r es la siguiente:

Dividir Q_r en tiles $Q_{r,j}$ de 16×16 elementos. Dentro de cada warp, los lanes se

dividen en grupos de 4 lanes para formar 8 grupos de 4 lanes cada uno. Cada lane carga entonces 2 BF16 por registro por cuadrante de 8×8 directamente desde memoria DRAM, como se muestra en la Figura 3.3.

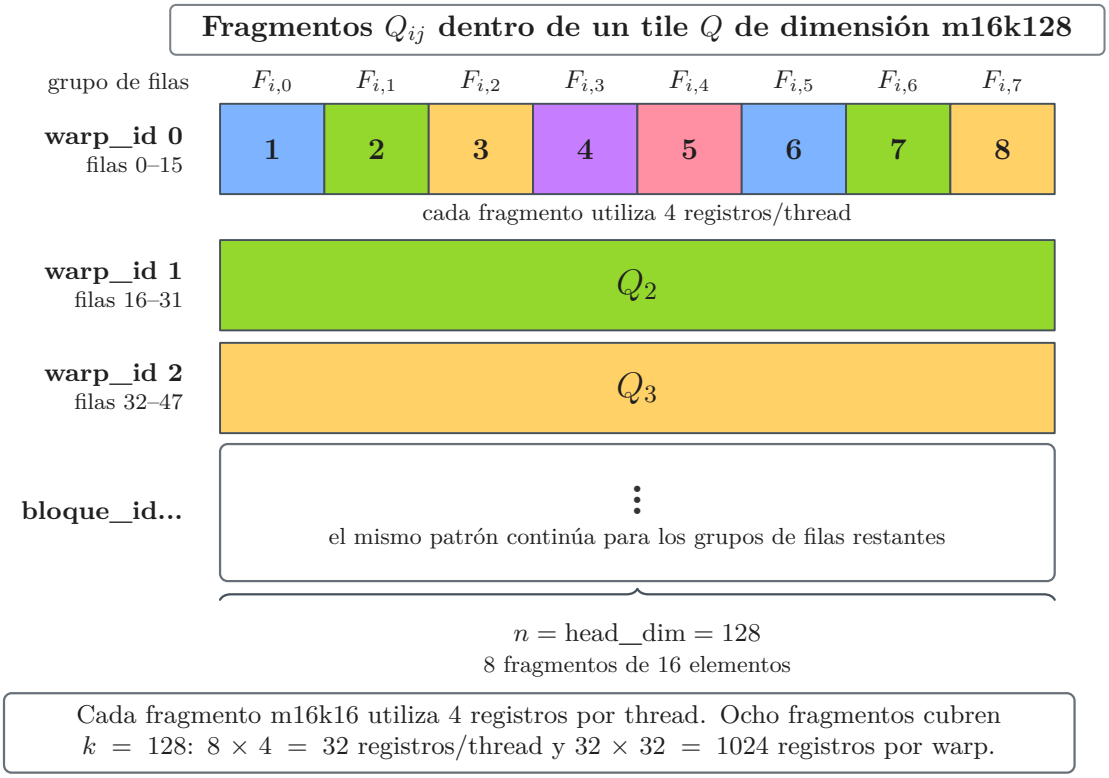


Figura 3.3. Mapping de registros para elementos de Q (m16k128) usando layout A `mma.sync`.

for $x = 0$ to 1 **do**
  for $y = 0$ to 1 **do**
    Cargar cuadrante $Q_{r,j}^{(y,x)} \in \mathbb{R}^{8\times8}$
  end for
end for

Esta distribución permite obtener en registros el formato necesario para las operaciones `mma.sync` posteriores, pero acceder de esta manera a memoria principal no es recomendable. Los elementos de cada columna están separados por

$$2 \text{ bytes/BF16} \times 128 \text{ BF16} = 256 \text{ bytes}. \tag{3.26}$$

Acceder a 8 rows carga 8 líneas de caché de 128 bytes, de los cuales solo se usan fragmentos de 16 bytes. Es decir,

$$\frac{16 \text{ bytes}}{128 \text{ bytes}} = \frac{1}{8} = 12{,}5\,\%. \tag{3.27}$$

Únicamente se utiliza el 12,5 % de cada línea de caché, lo que supone una tasa de aprovechamiento baja.

Se puede pensar que la línea de caché sigue en memoria al cargar el siguiente bloque de 8×8, como es probable que ocurra en CPU, pero sin conocer la política de reemplazo ni la contención de la misma, no se puede afirmar con seguridad. Es funcional, sin embargo.

Una alternativa para evitar este patrón de acceso consiste en cargar primero Q_r en SRAM utilizando cargas de 16 bytes y, posteriormente, de SRAM a RF.

El espacio total de Q_r es

$$16 \times 128 \text{ bfloat16} \times \frac{2 \text{ bytes}}{1 \text{ bfloat16}} = 4096 \text{ bytes}. \tag{3.28}$$

Esta alternativa, sin embargo, introduce otros costes cuya magnitud no puede determinarse únicamente a partir de la información disponible. Las preguntas sin resolver son las siguientes:

1. ¿Cuál es la latencia L2-SRAM, cuál es la latencia SRAM-RF, cuál es la latencia L2-L1 y cuál es la latencia L1-RF? Sin una tabla de latencias, no podemos afirmar que un acceso L2-SRAM + SRAM-RF sea más barato que un acceso L2-L1 + L1-RF.

2. Si dos warps encolados por dos warp schedulers acceden a la SRAM simultáneamente, ¿los accesos se serializan? NVIDIA define conflictos de banco a nivel de warp, pero no explica qué ocurre si hay accesos concurrentes a la SRAM.

3. Si cargamos Q_r en memoria compartida, una posibilidad es reservar un buffer específico para Q_r, lo que incrementa el consumo de SRAM del CTA. Otra alternativa consiste en reutilizar la misma región de memoria compartida utilizada posteriormente por los buffers de K_i y V_i. En este caso, Q_r se carga primero y, una vez consumido, el bucle puede comenzar a cargar los correspondientes K_i y V_i desplazando el `buffer_offset` en una posición. Esta estrategia permite evitar reservar 4 KiB adicionales durante toda la vida del CTA, pero complica la gestión de los buffers y de sus offsets, al menos durante la primera iteración.

4. No sabemos el throughput de la L1 ni qué efecto tienen las cargas vectorizadas.

5. No sabemos la latencia L2-L1 + L1-RF frente a L2-RF (*uncached loads*).

Aunque no sea posible cuantificar con precisión todos estos costes, sí pueden identificarse algunos inconvenientes de utilizar SRAM para Q_r.

Si estamos utilizando la SRAM para K y V, cuantos más CTAs residan en un SM, mayor puede ser la contención sobre la memoria compartida. Cargar también Q_r a través de SRAM aumenta la presión sobre este recurso y, además, puede requerir reservar 4 KiB adicionales por CTA durante toda su ejecución, lo que puede reducir la ocupancia.

Una alternativa es utilizar la L1 para las cargas de Q_r, descargando así parte de la presión sobre la SRAM. La pregunta principal es si, teniendo en cuenta las instrucciones

adicionales necesarias para realizar la permutación mariposa de los registros, esta estrategia puede superar el rendimiento de la carga a través de SRAM.

Existe, además, una tercera posibilidad: no utilizar siquiera la L1 para Q_r. Mediante una carga como `__ldcg()` se puede cargar Q_r desde memoria global a registros utilizando la política de caché global y haciendo bypass de la L1. De esta forma, se evita tanto reservar SRAM adicional para Q_r como la posible latencia añadida de descargar y cargar los datos en L1 antes de llegar a los registros.

Esta posibilidad resulta especialmente interesante porque Q_r se carga una única vez y, una vez en registros, se reutiliza durante el procesamiento de los distintos tiles de K y V. Sin embargo, los elementos cargados directamente en registros no se encuentran inicialmente en el layout requerido por `mma.sync`, por lo que es necesario realizar una permutación de los registros mediante una traspuesta.

Por tanto, existen varias posibilidades para la carga de Q_r: utilizar SRAM como almacenamiento intermedio, utilizar la L1 o realizar una carga directa a registros evitando la L1. La elección entre ellas depende del coste de las cargas, la presión sobre SRAM y L1, la ocupancia y el coste adicional de reorganizar los registros.

La siguiente sección describe la permutación utilizada para realizar esta reorganización una vez que Q_r se encuentra en registros.

3.6. Traspuesta en registros.

$$32 \text{ threads/warp} \times 16 \text{ bytes/thread} = 512 \text{ bytes/warp} \tag{3.29}$$

por instrucción de carga vectorizada.

$$1 \times 128 \text{ BF16} \times 2 \text{ bytes/BF16} = 256 \text{ bytes.} \tag{3.30}$$

Cada grupo de 16 lanes puede cargar un row de Q_i utilizando cargas vectorizadas de 16 bytes.

Cada lane se presenta con 32 registros de 32 bits. Para cada lane_i, el registro j es el elemento que debería tener el lane_j en la posición i. Es decir, una traspuesta en registros.

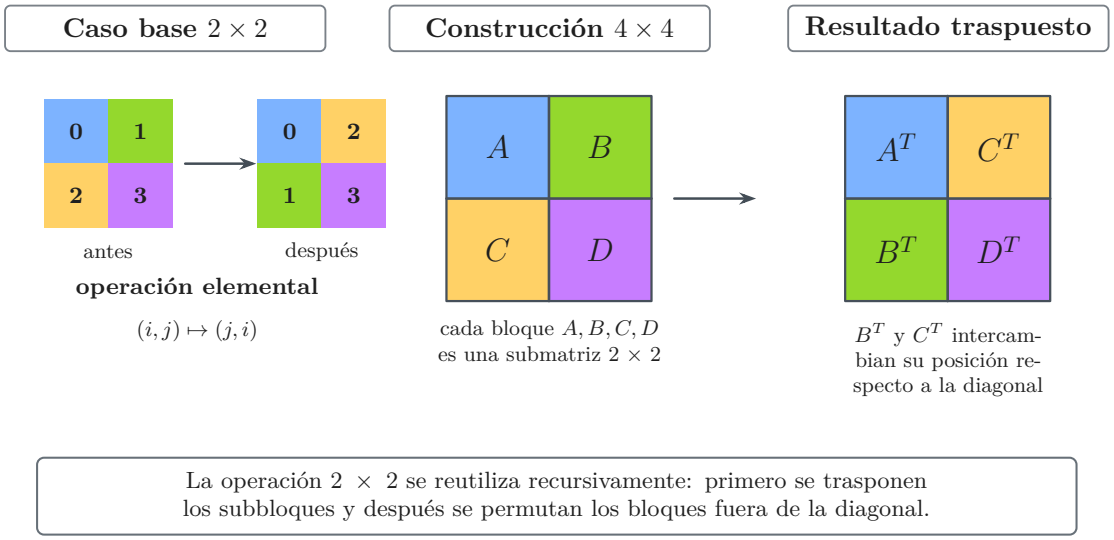


Figura 3.4. Traspuesta en registros: idea recursiva.

La idea de la traspuesta se muestra en la Figura 3.4.

Algorithm 2 Permutación mariposa de los registros

```
1: for d ∈ {1, 2, 4, 8, 16} do                          ▷ Seleccionar tamaño del bloque
2:     upper ← (lane_id & d) ≠ 0
3:     for base = 0, 2d, 4d, ..., 32 − 2d do            ▷ Seleccionar bloque de 2d registros
4:         for offset = 0, ..., d − 1 do                ▷ Seleccionar registros dentro del bloque
5:             j0 ← base + offset
6:             j1 ← j0 + d
7:             if upper then
8:                 send ← reg[j0]
9:             else
10:                send ← reg[j1]
11:            end if
12:            received ← shfl_xor(send, d)
13:            if upper then
14:                reg[j0] ← received
15:            else
16:                reg[j1] ← received
17:            end if
18:        end for
19:    end for
20: end for
```

Como visualizar el algoritmo es complicado, el pseudocódigo se muestra en el Algoritmo 2. Se debe tener en cuenta que estos tres bucles únicamente determinan qué registros intercambia cada lane con otro lane del warp.

La desventaja de operar de esta manera es que los registros no se intercambian

mediante un simple renombrado, como puede ocurrir dentro de una CPU, sino que el intercambio entre lanes requiere instrucciones `shfl`. Estas instrucciones tienen un throughput relativamente bajo frente a muchas operaciones aritméticas. Para dispositivos con *compute capability* 8.6/8.9, NVIDIA documenta un throughput máximo teórico de 32 resultados por ciclo y por multiprocesador para las operaciones de *warp shuffle* [33].

3.7. Instrucción `ldmatrix.x4`: carga de un tile 16×16 de SRAM con layout `mma.sync`

Esta instrucción nos permite cargar un tile 16×16 en registros desde la SRAM [17]. La carga se divide en cuatro tiles de 8×8.

Cada tile de 8×8 contiene 8 rows de 8 elementos `bfloat16`. Como cada `bfloat16` ocupa 2 bytes, cada row contiene

$$8 \text{ elementos} \times \frac{2 \text{ bytes}}{1 \text{ bfloat16}} = 16 \text{ bytes}. \tag{3.31}$$

Por tanto, las 8 rows del tile requieren

$$8 \text{ rows} \times 16 \text{ bytes/row} = 128 \text{ bytes}. \tag{3.32}$$

Como la instrucción accede a los chunks de 8×8 de forma secuencial, permite cargar el tile completo de 16×16 en un total de 4 wavefronts, siguiendo el orden descrito en la Figura 3.5.

Los datos se cargan directamente en los registros con el layout esperado para un tile A por la instrucción `mma.sync` descrita anteriormente.

Esta instrucción se utiliza para cargar los tiles $K_{i,j}$ y $V_{i,j}$ desde SRAM a registros.

3.8. Instrucción `cp.async`

La instrucción `cp.async` permite al hardware transferir datos desde memoria principal a SRAM sin pasar por L1 ni por registros [14, 17].

NVIDIA detalla que esta instrucción mejora el rendimiento cuando se utiliza en transferencias de datos cuyo destino es SRAM y, además, hace posible la técnica de múltiple buffer.

En esta implementación se utiliza la siguiente variante:

```
cp.async.cg.shared.global.L2::cache_hint.L2::256B
```

La modalidad `.cg` permite evitar la L1, mientras que `L2::cache_hint.L2::256B` proporciona un *cache hint* de 256 bytes para L2.

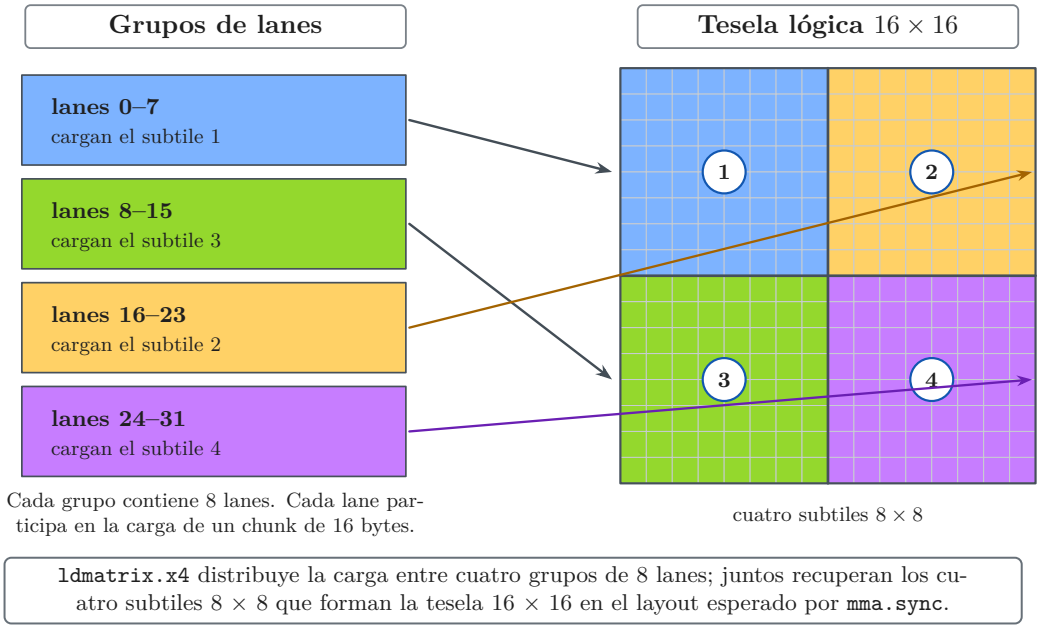


Figura 3.5. Mapeado de lanes y fragmentos en la instrucción `ldmatrix.x4` para la carga de un tile 16 × 16 desde SRAM a registros con el layout requerido por `mma.sync`.

Adicionalmente, la instrucción soporta un parámetro `policy`, que permite especificar la política de reemplazo que debe seguir el elemento cargado.

Para los elementos de Q y el tile de salida O se utiliza la política `evict_first`, porque sus datos no se reutilizan entre warps. En cambio, para las matrices K y V se utiliza la política `evict_last`, porque sus datos se reutilizan entre CTAs.

En el caso concreto de `cp.async`, únicamente se especifica la política `evict_last`; esta instrucción no se utiliza para cargar Q.

3.9. Doble buffer SMEM-RF

NVIDIA documenta que es posible realizar un doble buffer SMEM-RF [34]. En el kernel CUDA se establece un doble buffer de 2 tiles de 16 × 16.

3.10. K_i/V_i y SRAM

Mientras que Q_r se utiliza únicamente por warp, K_i y V_i se reutilizan por todos los warps. Si tenemos más warps por CTA, cargar K_i y V_i en SRAM permite reducir los accesos a L2 de manera proporcional al número de warps.

Actualmente, un buffer lo forman como mínimo K_i y V_i. K_{i+1} y V_{i+1} forman, en conjunto, 8 KiB de memoria. Se cargan cooperativamente entre los warps.

El número de buffers empleados en la estrategia multibuffer se determina mediante una búsqueda de parámetros.

3.10.1. SRAM layout naive vs swizzled

Un layout naive ocasiona conflictos de banco. Un conflicto de banco ocurre cuando dos threads de un mismo warp acceden a distintas direcciones de memoria almacenadas en el mismo banco. Esto ocasiona que los accesos se serialicen.

NCU define un wavefront como la máxima cantidad de bytes que un warp puede transferir a/desde la memoria SRAM. Actualmente son 128 bytes.

Si los 32 threads de un warp acceden a datos distintos almacenados en un mismo banco, por ejemplo, elementos de 4 bytes, la SRAM solo puede suministrar 4 bytes por ciclo desde dicho banco. Por tanto, la transferencia necesita 32 ciclos.

El ancho de banda aprovechado respecto a un acceso con conflictos es:

$$\frac{4 \text{ bytes/ciclo} \times 32 \text{ ciclos}}{128 \text{ bytes/ciclo} \times 32 \text{ ciclos}} = \frac{4}{128} = \frac{1}{32}. \tag{3.33}$$

Por tanto, únicamente se aprovecha

$$\frac{1}{32} = 3{,}125\,\% \tag{3.34}$$

del ancho de banda disponible.

Por este motivo, los accesos a SRAM deben procurar distribuirse entre bancos distintos.

Un layout *swizzled* introduce una mayor complejidad en el cálculo de las direcciones de memoria a cambio de distribuir los accesos entre los distintos bancos y eliminar los conflictos de banco para el patrón de acceso considerado.

Los buffers K_i y V_i se cargan utilizando un layout *swizzled*. A diferencia del ejemplo mostrado en la Figura 3.6, cada bloque considerado por el swizzle tiene un tamaño de 16 bytes, ocupando cuatro bancos de SRAM.

Este tamaño se utiliza para aprovechar la instrucción `ldmatrix.x4`. De esta forma, no se producen conflictos de banco entre los bloques correspondientes a los tiles de 8×8.

Como cada fila contiene 256 bytes, se divide en

$$\frac{256 \text{ bytes}}{16 \text{ bytes/bloque}} = 16 \text{ bloques}. \tag{3.35}$$

Por tanto, existen 16 columnas de bloques de 16 bytes. La mitad de los lanes del warp se encarga de cargar K_i, mientras que la otra mitad carga V_i. El patrón de swizzle se repite cada 8 columnas y, de la misma forma, vuelve a repetirse cada 8 filas.

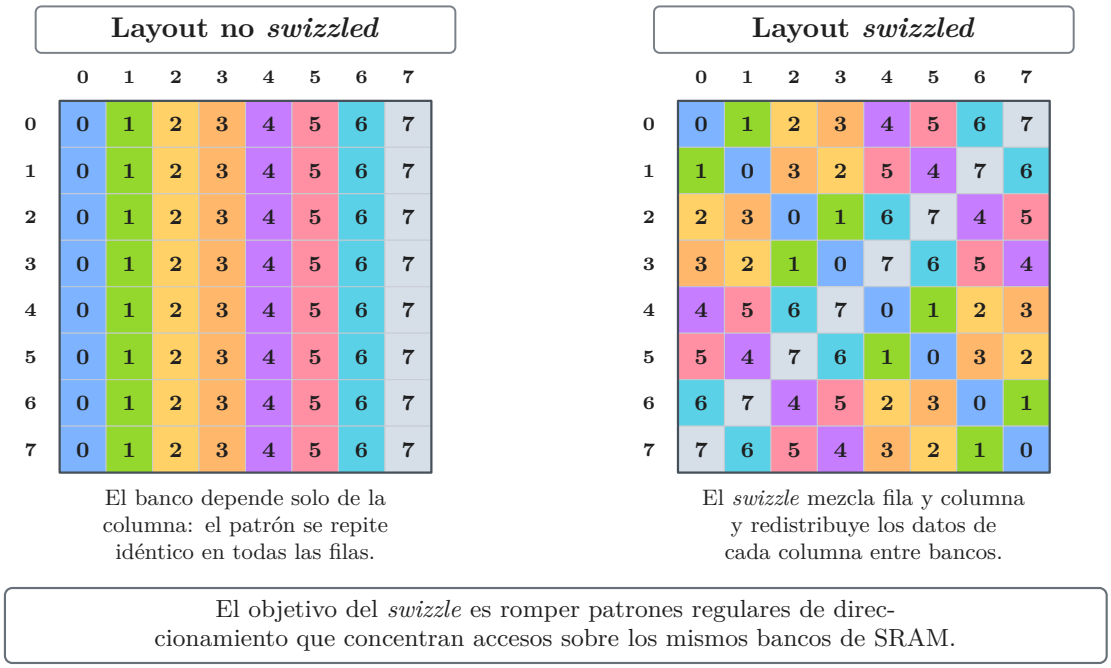


Figura 3.6. Comparación entre un layout naive y un layout swizzled en SRAM. El swizzle modifica la distribución de los elementos entre bancos para evitar conflictos de banco en el patrón de acceso considerado.

3.11. Multibuffer en SRAM

Los warps de un CTA cooperan para cargar los elementos de un multibuffer. La pregunta principal al utilizar múltiples buffers es: ¿qué tamaño debe tener cada etapa, o qué número de etapas debe tener el multibuffer, para maximizar la actividad del consumidor?

Actualmente, cada etapa contiene los buffers K_i y V_i correspondientes a un valor de i, y un CTA puede mantener varias etapas cargadas simultáneamente. El número de etapas que maximiza el rendimiento del consumidor se determina mediante una optimización de parámetros en la Sección 3.19.

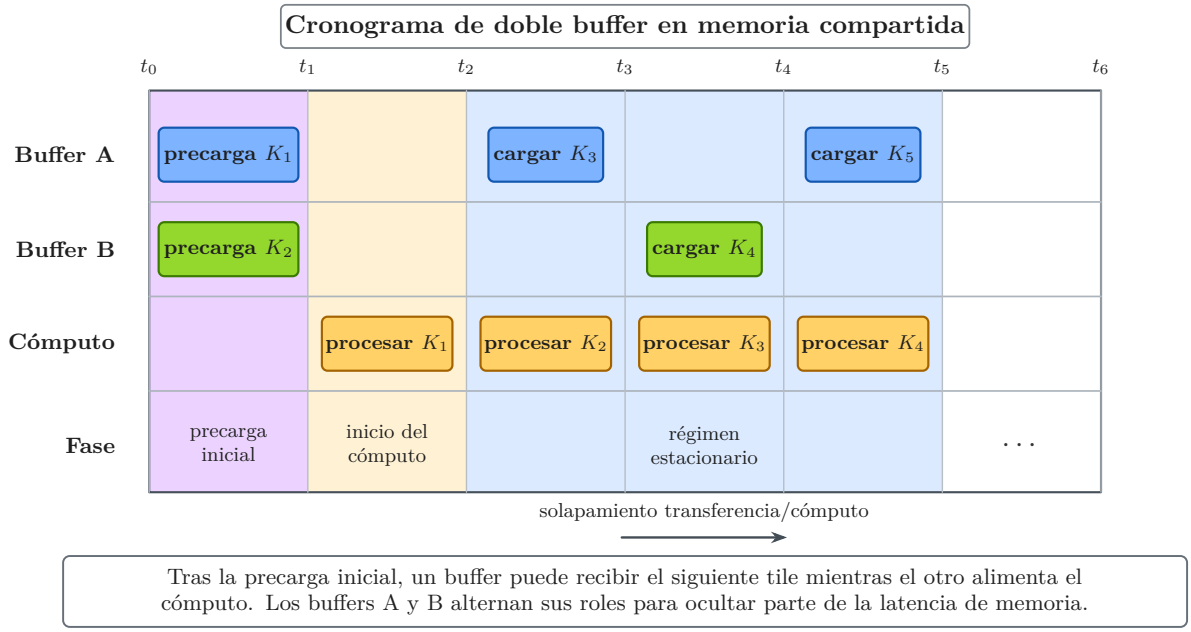


Figura 3.7. Cronograma de un multibuffer de dos etapas en SRAM. Tras la precarga inicial, una etapa puede reutilizarse para cargar los siguientes tiles K_i y V_i mientras se procesan los datos almacenados en la otra. En la imagen, solo aparecen las precargas de K para simplificar, pero en la práctica, un buffer lo forma un par (K_i, V_i)

El funcionamiento del multibuffer se muestra en la Figura 3.7.

3.12. Loop unrolling y partial loop unrolling

Loop unrolling es una técnica que convierte un bucle `for` de n iteraciones en una secuencia de instrucciones repetidas n veces. Las ventajas son: reducir o eliminar completamente los saltos como consecuencia del bucle y reorganizar las instrucciones pertenecientes originalmente a distintas iteraciones para aprovechar mejor las pipelines. El problema es que un loop unrolling excesivo puede producir fallos en la memoria caché de instrucciones, eliminando la ventaja de rendimiento. Cuánto desenrollar un bucle es una tarea de optimización [20].

Loop unrolling completo también ayuda a CUDA a identificar variables de tipo arreglo como registros, siempre y cuando se accedan mediante constantes.

3.13. Cálculo del row-sum y del row-max

Estas operaciones necesitan calcular el máximo y la suma de datos que se encuentran en registros de distintos threads del warp. Para calcular la suma se utiliza el operador suma, mientras que para calcular el máximo se utiliza la función `fmaxf`.

Para intercambiar valores entre los registros de distintos lanes del mismo warp, se utiliza la instrucción `shfl_xor_sync` con `width = 4`, agrupando los threads en grupos

de 4 lanes.

3.14. Cálculo de la exponencial

La exponencial es una operación tradicionalmente cara de computar. Tanto es así que, si bien el throughput de las operaciones matriciales ha seguido aumentando en Hopper y Blackwell, el throughput de las operaciones trascendentales no ha aumentado al mismo ritmo. En Blackwell, la unidad MUFU proporciona 16 resultados/ciclo/SM para estas operaciones, lo que convierte el cálculo de la exponencial del *softmax* en uno de los posibles cuellos de botella de FlashAttention [35].

Por este motivo, FlashAttention-4 propone aumentar el throughput efectivo de la exponencial mediante una aproximación polinómica ejecutada sobre las unidades FMA [35].

Para dispositivos con *compute capability* 8.6, NVIDIA documenta un throughput máximo teórico de 16 resultados por ciclo y por SM para operaciones trascendentales aproximadas como la exponencial en base 2 [33].

Considero que en Ampere también podemos obtener un mayor throughput efectivo mediante una aproximación de la función exponencial.

La aproximación utilizada se describe a continuación.

3.14.1. Fusionar el scale con la exponencial

Se parte de

$$\max_j(\text{scale} \cdot s_{ij}), \tag{3.36}$$

donde i representa la fila de S y j recorre las columnas de dicha fila.

Definimos el máximo de la fila sin escalar como

$$m_i = \max_j(s_{ij}). \tag{3.37}$$

Como

$$\text{scale} = \frac{1}{\sqrt{d}} > 0, \tag{3.38}$$

se cumple que

$$\max_j(\text{scale} \cdot s_{ij}) = \text{scale} \cdot \max_j(s_{ij}) = \text{scale} \cdot m_i. \tag{3.39}$$

Por tanto, para cada elemento de la fila,

$$\text{scale} \cdot s_{ij} - \text{scale} \cdot m_i = \text{scale}(s_{ij} - m_i). \tag{3.40}$$

Esto permite realizar primero la resta entre el *score* y el máximo de la fila, sin escalar previamente ninguno de los dos valores. Se define entonces

$$x = \text{score} - \text{row_max} = s_{ij} - m_i. \tag{3.41}$$

Posteriormente se evalúa

$$\text{exp_poly2_scaled}(x), \tag{3.42}$$

donde la propia función incorpora el factor scale y aproxima

$$e^{\text{scale}\cdot x} = e^{\text{scale}(s_{ij}-m_i)} = e^{\text{scale}(\text{score}-\text{row_max})}. \tag{3.43}$$

3.14.2. Cálculo de $e^{\text{scale}(\text{score}-\text{row_max})}$

Utilizando la identidad

$$a^b = 2^{b\log_2(a)}, \tag{3.44}$$

para $a = e$ se obtiene

$$e^z = 2^{z\log_2(e)}. \tag{3.45}$$

Tomando

$$z = \text{scale} \cdot x, \tag{3.46}$$

donde, como se ha definido anteriormente,

$$x = \text{score} - \text{row_max}, \tag{3.47}$$

queda

$$e^{\text{scale}\cdot x} = 2^{(\text{scale}\cdot x)\log_2(e)}. \tag{3.48}$$

Como scale y $\log_2(e)$ son constantes, se define

$$K = \text{scale} \cdot \log_2(e), \tag{3.49}$$

que puede calcularse en tiempo de compilación.
A continuación, se define

$$y = xK. \tag{3.50}$$

De esta forma,

$$e^{\text{scale}\cdot x} = 2^y, \tag{3.51}$$

por lo que el problema se reduce al cálculo de

$$2^y. \tag{3.52}$$

3.14.3. Reducción de rango

Hasta este punto,

$$e^z = 2^y. \tag{3.53}$$

Sea

$$y = n + f, \tag{3.54}$$

donde

$$n = \text{round}(y), \qquad f = y - n, \qquad f \in [-0{,}5, 0{,}5]. \tag{3.55}$$

Entonces,

$$e^z = 2^{n+f} = 2^n 2^f. \tag{3.56}$$

La función 2^f es una función suave en un intervalo reducido:

$$2^{-0{,}5} \approx 0{,}7071, \qquad 2^{0{,}5} \approx 1{,}4142, \tag{3.57}$$

por lo que puede aproximarse mediante un polinomio.

3.14.4. Sollya

Sollya [36] utiliza el algoritmo de Remez [37] para obtener una aproximación polinómica de 2^f en el intervalo $f \in [-0{,}5, 0{,}5]$. La diferencia con respecto a un polinomio de Taylor es que Taylor aproxima muy bien la función cerca de un punto determinado, pero la aproximación puede empeorar conforme aumenta la distancia a dicho punto. El algoritmo de Remez, en cambio, busca minimizar el máximo error de la aproximación sobre todo el intervalo evaluado.

El resultado obtenido con una precisión de 200 bits es

$$2^f \approx 1 + f\,(0{,}70294183492660522460937 5 + f \cdot 0{,}23986406624317169189453125)\,. \tag{3.58}$$

El máximo error relativo obtenido en el intervalo $f \in [-0{,}5, 0{,}5]$ es

$$0{,}19634037\,\%. \tag{3.59}$$

Es importante fijar en Sollya el primer coeficiente del polinomio a 1. Como se determina en el Anexo D, conforme aumenta la longitud de secuencia, el caso que más tiende a repetirse es aquel en el que el máximo de la fila no cambia. En este caso, el argumento de la exponencial es 0 y, por tanto,

$$2^0 = 1. \tag{3.60}$$

Si el primer coeficiente no se fijase a 1, la aproximación podría introducir un error incluso para $f = 0$, por ejemplo aproximando 2^0 mediante un valor cercano a 0,98. Este error es especialmente relevante porque, cuando el máximo no cambia, este factor se utiliza repetidamente durante el cálculo del *softmax* online, por lo que su efecto puede acumularse conforme aumenta la longitud de secuencia. Fijar el primer coeficiente a 1 garantiza que la aproximación sea exacta en $f = 0$.

Sollya no necesita una semilla.

3.14.5. Downcast de FP32 a BF16

Nuestra exponencial utiliza un polinomio de grado 2 para aproximar el cálculo de 2^f, mientras que FlashAttention-4 estudia, entre otras, una aproximación de grado 3 [35]. Nuestra aproximación se beneficia del posterior *downcasting* a BF16, porque parte del error introducido por el polinomio puede quedar oculto por la propia cuantización de este formato.

Para calcular la diferencia entre la versión basada en `__expf()` y `poly2_exp`, se han generado 10 millones de valores distribuidos en el intervalo $[-80, 0]$. Para cada valor se calcula el resultado mediante ambas funciones y, posteriormente, ambos resultados se convierten a BF16.

Una vez realizada la conversión, se calcula la distancia entre ambos resultados en ULP. Una diferencia de 0 ULP indica que ambas versiones producen exactamente el mismo valor BF16, mientras que una diferencia de 1 ULP indica que los resultados corresponden a dos valores BF16 consecutivos.

Los resultados obtenidos son:

Diferencia	**Porcentaje**
0 ULP BF16	80,165900 %
1 ULP BF16	19,834100 %
≥ 2 ULP BF16	0,000000 %

Tabla 3.2. Distribución de la diferencia en ULP entre `__expf()` y `poly2_exp` después de convertir los resultados a BF16.

Por tanto, aproximadamente el 80 % de los resultados coinciden exactamente entre ambas versiones después de la conversión a BF16. En el 20 % restante, los resultados difieren únicamente en 1 ULP, sin observarse ningún caso con una diferencia igual o superior a 2 ULP.

3.14.6. Cálculo de 2^n

El cálculo de 2^n puede realizarse directamente mediante la representación IEEE 754 de FP32. Para valores normalizados, una potencia de dos tiene una mantisa nula y su exponente almacenado corresponde a n más el *bias* de FP32, cuyo valor es 127.

Por tanto, el patrón de bits correspondiente a 2^n puede construirse como

$$(n + 127) \ll 23, \tag{3.61}$$

donde el desplazamiento de 23 bits sitúa el exponente en su posición correspondiente dentro de la representación FP32. Finalmente, este patrón de bits se reinterpreta como un valor de tipo `float`.

Constante MAGIC

Para realizar la reducción de rango es necesario calcular

$$n = \text{round}(y). \tag{3.62}$$

En lugar de utilizar una operación explícita de redondeo, se emplea la constante

$$\text{MAGIC} = 12582912 = 1{,}5 \cdot 2^{23}. \tag{3.63}$$

El valor 2^{23} está relacionado con los 23 bits almacenados de la mantisa de FP32. En el intervalo

$$[2^{23}, 2^{24}), \tag{3.64}$$

la distancia entre dos valores FP32 consecutivos es exactamente 1. Por tanto, al sumar a y una constante situada dentro de este intervalo, la representación FP32 deja de poder conservar una parte fraccionaria menor que 1 y el resultado se redondea al entero representable más cercano.

La elección concreta

$$\text{MAGIC} = 1{,}5 \cdot 2^{23} \tag{3.65}$$

sitúa la constante en el centro del intervalo $[2^{23}, 2^{24})$. De esta forma existe margen tanto para valores positivos como negativos de y sin abandonar la región en la que la separación entre valores FP32 es 1.

Por ejemplo, para

$$y = 3{,}7, \tag{3.66}$$

se obtiene

$$t = 3{,}7 + 12582912 = 12582915{,}7. \tag{3.67}$$

Como en esta región los valores FP32 consecutivos están separados por 1, el resultado se redondea a

$$t = 12582916. \tag{3.68}$$

Al restar nuevamente la constante,

$$n = 12582916 - 12582912 = 4, \tag{3.69}$$

que coincide con

$$\text{round}(3{,}7) = 4. \tag{3.70}$$

El mismo mecanismo funciona para valores negativos. Por ejemplo,

$$y = -3{,}7 \tag{3.71}$$

produce

$$t = 12582912 - 3{,}7 = 12582908{,}3, \tag{3.72}$$

que se redondea a

$$12582908, \tag{3.73}$$

y, por tanto,

$$n = 12582908 - 12582912 = -4. \tag{3.74}$$

Se calcula entonces

$$t = y + \text{MAGIC}, \tag{3.75}$$

y posteriormente

$$n = t - \text{MAGIC}. \tag{3.76}$$

De esta forma se obtiene y redondeado al entero más cercano sin utilizar una instrucción específica de redondeo. Bajo el modo de redondeo *round-to-nearest-even* de FP32, los casos exactamente equidistantes se redondean al entero par más cercano.

En la implementación, esta operación puede fusionarse con el cálculo previo de $y = xK$, dando lugar a

$$t = \mathrm{fma}(x, K, \mathrm{MAGIC}), \tag{3.77}$$

seguido de

$$n = t - \mathrm{MAGIC}. \tag{3.78}$$

3.14.7. Validez

El criterio de validez empleado para determinar si la precisión obtenida es suficiente consiste en comprobar el resultado final del kernel frente a Torch SDPA mediante `torch.testing.assert_close`, utilizando

$$\mathrm{rtol} = 2 \times 10^{-2}, \qquad \mathrm{atol} = 2 \times 10^{-2}. \tag{3.79}$$

Todos los experimentos emplean la misma semilla, 0, para generar los datos de entrada.

Resulta complicado, dado el alcance del proyecto y los conocimientos adquiridos hasta el momento, determinar si una precisión en la que aproximadamente el 80 % de los valores de la aproximación exponencial coinciden exactamente con `__expf()` después de la conversión a BF16 y el 20 % restante difiere en 1 ULP es suficiente para cualquier escenario. Si se encadenan muchas capas o se utilizan longitudes de secuencia considerablemente mayores, los errores numéricos podrían propagarse y sería necesario estudiar con mayor detalle la precisión de la aproximación.

En este trabajo se adopta una medida conservadora basada en comprobar la validez del resultado final del kernel, sin realizar un estudio exhaustivo de propagación del error numérico, debido al alcance del proyecto. Para ello, `validez.sh` recompila el kernel con la configuración correspondiente a cada longitud de secuencia y compara su salida con Torch SDPA.

Longitud de secuencia	Error absoluto máximo	Error absoluto medio	Valores próximos (%)
8192	$4{,}8828125 \times 10^{-4}$	$3{,}9664301 \times 10^{-5}$	100 %
16384	$2{,}44140625 \times 10^{-4}$	$2{,}8437838 \times 10^{-5}$	100 %
32768	$2{,}44140625 \times 10^{-4}$	$2{,}0409740 \times 10^{-5}$	100 %
65536	$1{,}220703125 \times 10^{-4}$	$1{,}4567711 \times 10^{-5}$	100 %

Tabla 3.3. Error de la implementación CUDA respecto a Torch SDPA para las distintas longitudes de secuencia.

Resulta interesante observar que, en los experimentos realizados, tanto el error

absoluto máximo como el error absoluto medio presentan una tendencia decreciente conforme aumenta la longitud de secuencia. Dado que los valores de la matriz V se generan aleatoriamente mediante una distribución normal de media cero, las contribuciones positivas y negativas a la salida de atención pueden compensarse entre sí.

3.14.8. Análisis SASS: `__expf` frente a la aproximación por polinomio

El análisis del código SASS generado puede consultarse en Compiler Explorer (Godbolt) mediante el siguiente enlace: https://godbolt.org/z/5TWacKq47.

La descripción de las instrucciones es la siguiente:

Instrucción	Significado	Throughput
`MOV R3, 0x3e0293ee`	Mover a R3 el resultado de scale $\cdot \log_2(e)$	–
`FMNMX R0, R0, -905.0966796875, !PT`	Clamp del resultado	64
`MOV R2, 0x3e759eed`	Cargar el coeficiente 0,2398…	–
`FFMA R4, R0, R3, 12582912`	FMA con K y MAGIC	128
`FADD R3, R4, -12582912`	$n = t -$ MAGIC	128
`FFMA R3, R0, 0.12751743197441101074, -R3`	$f =$ fmaf $(x, K, -n)$	128
`FFMA R0, R3, R2, 0.70294183492660522461`	fmaf(0,2398…, f, 0,7029…)	128
`FFMA R5, R3, R0, 1`	Último FMA con p y f	128
`LEA R5, R4, R5, 0x17`	$p +$ (nbits $\ll 23$)	–

En total, se ejecutan 4 FMA, 1 máximo, 1 FADD, 2 operaciones de carga con inmediatos y una LEA.

$$T_{\text{poly}} \simeq \frac{4}{128} + \frac{1}{128} + \frac{1}{64} = \frac{7}{128} = 0{,}0547 \text{ ciclos/resultado.} \tag{3.80}$$

`__expf`

La función `__expf` calcula

$$e^z = 2^{z \log_2(e)}. \tag{3.81}$$

En el análisis SASS realizado mediante Compiler Explorer (Godbolt), la principal

diferencia observada con respecto a nuestra implementación es el cálculo de 2^y. `__expf` utiliza la instrucción `MUFU.EX2` para realizar esta operación, mientras que nuestra implementación aplica una reducción de rango, separa y en una parte entera y una parte fraccionaria mediante el uso de la constante MAGIC, y aproxima la parte fraccionaria mediante el polinomio obtenido con Sollya.

Instrucción	**Significado**	**Throughput**
`FMUL R0, R0, 0.08838834613561630249`	Escalado de x por $1/\sqrt{128}$	128
`FMUL R0, R0, 1.4426950216293334961`	Multiplicar R0 por $\log_2(e)$	128
`FSETP.GEU.AND P0, PT, R0, -126, PT`	Registro de predicado para el clamp si $x \leq -126$	–
`@!P0 FMUL R0, R0, 0.5`	Si $x \leq -126$, dividir entre 2	128
`MUFU.EX2 R5, R0`	Cálculo de 2^y	16
`@!P0 FMUL R5, R5, R5`	Si $x \leq -126$, elevar R5 al cuadrado	128

$$T_{\text{EX2}} = \frac{1}{16} = 0{,}0625 \text{ ciclos/resultado.} \tag{3.82}$$

Sin contar las MOV y las LEA, que difícilmente pueden tener un throughput inferior al de las FMUL, la aproximación presenta un mayor throughput por operación. Si se tienen en cuenta las FMUL y las instrucciones predicadas, el throughput es considerablemente peor.

`__expf` contiene dos FMUL que podrían fusionarse, pero que el compilador ha decidido no fusionar.

No hay que confundir la latencia con el throughput. El throughput es el número de resultados que se pueden obtener por ciclo y por SM para un pipeline determinado. La latencia es el tiempo que tarda una operación en producir su resultado.

NVIDIA proporciona el throughput de las instrucciones, pero no cuánto tarda cada una de ellas ni qué ocurre cuando existen dependencias entre instrucciones. Por tanto, únicamente se puede afirmar que la aproximación presenta un mayor throughput que la `expf` base.

En total, `__expf` utiliza 2 FMUL, 2 FMUL opcionales, 1 FSETP y una MUFU.

3.15. Downcast a P

El *downcasting* de S desde FP32 a P en BF16 se realiza mediante la instrucción PTX

```
cvt.rn.bf16x2.f32 %0, %1, %2.
```
(3.83)

NVIDIA no especifica el throughput de esta instrucción, por lo que no se puede determinar directamente si, al igual que ocurre con las instrucciones `shfl` utilizadas para el intercambio entre registros, presenta un throughput reducido.

3.16. Multiplicar PV

P_r, de dimensiones 16×16, multiplica cada subtile V_{ij}. El resultado se acumula en O, de dimensiones $B_r \times d$, dividido en subtiles O_j, donde cada O_j tiene dimensiones 16×16.

3.17. División por recíproco de O

En lugar de realizar la división directamente, se utiliza la intrínseca `__frcp_rn(x)`, que calcula el recíproco de x,

$$\frac{1}{x}. \tag{3.84}$$

Posteriormente, este recíproco se multiplica por cada elemento de O, sustituyendo así la división por un recíproco seguido de multiplicaciones.

3.18. Volcado de O en memoria principal

La matriz O vuelve a trasponer sus registros antes de realizar el volcado a memoria principal. De esta manera, cada *thread* puede realizar un almacenamiento vectorizado de 16 bytes en memoria.

3.19. Optimización de hiperparámetros mediante Optuna

Para balancear el número de warps, el número de CTAs que ocupan un SM, el número de buffers empleados en la estrategia multibuffer, cuánto se puede desenrollar el bucle principal sin incurrir en excesivos fallos de caché de instrucciones y si se deben utilizar accesos `cg` o `wb` para las matrices Q y O, es necesario realizar una búsqueda de parámetros.

Parámetro	N.º de valores	Valores posibles
`ctas_per_sm`	8	$1, 2, 3, 4, 5, 6, 7, 8$
`num_warp`	5	$1, 2, 4, 8, 16$
`buffer_num`	5	$2, 3, 4, 5, 6$
`uncached`	2	$0, 1$
`buffer_unroll`	16	$1, 2, 3, 4, 5, 6, 7, 8,$ $9, 10, 11, 12, 13, 14, 15, 16$
Total de combinaciones	**6400**	$8 \times 5 \times 5 \times 2 \times 16$

A priori, los parámetros mencionados pueden parecer pocos. Sin embargo, probar las 6400 combinaciones es un proceso costoso. Se pueden considerar únicamente aquellas combinaciones que sean posibles, teniendo en cuenta tanto el número máximo de warps por SM como el número máximo de CTAs, en función de la SRAM ocupada por el kernel para un determinado valor de `buffer_num`.

3.19.1. Poda del espacio de búsqueda

Antes de realizar la búsqueda mediante Optuna, se eliminan las configuraciones que no pueden ejecutarse debido a las restricciones hardware de la GPU.

En primer lugar, se comprueba el número de threads por bloque. La condición de poda es

$$32 \cdot \texttt{warps_per_block} > 1024. \tag{3.85}$$

Sin embargo, el máximo número de warps considerado es 16, por lo que el máximo número de threads por bloque es

$$32 \cdot 16 = 512. \tag{3.86}$$

Por tanto, ninguna de las 6400 combinaciones iniciales se elimina mediante este filtro.

A continuación, se comprueba el número máximo de threads que pueden residir simultáneamente en un SM:

$$32 \cdot \texttt{warps_per_block} \cdot \texttt{ctas_per_sm} > 1536. \tag{3.87}$$

Esta condición equivale a

$$\texttt{warps_per_block} \cdot \texttt{ctas_per_sm} > 48. \tag{3.88}$$

Existen 7 pares de valores de `warps_per_block` y `ctas_per_sm` que incumplen esta restricción.

Para cada uno de estos pares todavía existen 5 posibles valores de `buffer_num`, 2

valores de `uncached` y 16 valores de `buffer_unroll`. Por tanto, cada par representa

$$5 \cdot 2 \cdot 16 = 160 \tag{3.89}$$

trials, por lo que se eliminan

$$7 \cdot 160 = 1120 \tag{3.90}$$

configuraciones. El espacio de búsqueda se reduce así de 6400 a

$$6400 - 1120 = 5280 \tag{3.91}$$

trials.

Finalmente, se tiene en cuenta el consumo de shared memory por SM. La condición utilizada es

$$(8 \text{ KiB} \cdot \texttt{buffer_num} + 1 \text{ KiB}) \cdot \texttt{ctas_per_sm} > 100 \text{ KiB}. \tag{3.92}$$

Tras aplicar previamente la restricción de threads por SM, existen 88 combinaciones de `buffer_num`, `ctas_per_sm` y `warps_per_block` que incumplen esta condición.

Cada una de estas combinaciones todavía puede asociarse con 2 valores de `uncached` y 16 valores de `buffer_unroll`, es decir,

$$2 \cdot 16 = 32 \tag{3.93}$$

trials por combinación. Por tanto, este filtro elimina

$$88 \cdot 32 = 2816 \tag{3.94}$$

trials adicionales.

En total, se eliminan

$$1120 + 2816 = 3936 \tag{3.95}$$

de las 6400 configuraciones iniciales, quedando finalmente

$$6400 - 3936 = \boxed{2464} \tag{3.96}$$

configuraciones válidas[1].

Si se tienen 2464 trials y se supone un tiempo generoso de compilación y profiling de 30 s por trial,

$$2464 \text{ trials} \times 30 \text{ s/trial} = 73920 \text{ s}. \tag{3.97}$$

[1]El repositorio conserva el cálculo original del consumo de memoria compartida. La corrección realizada únicamente modifica el recuento de configuraciones no ejecutables y la estimación del tiempo de búsqueda, sin afectar a los resultados experimentales obtenidos.

$$73920 \text{ s} \times \frac{1 \text{ h}}{3600 \text{ s}} = 20{,}53 \text{ h}. \tag{3.98}$$

Un tiempo de 20,53 h para un rango de parámetros tan reducido supone un coste excesivo e innecesario. Por ello, es necesario realizar una búsqueda informada.

En el proyecto se ha decidido realizar esta búsqueda mediante Optuna y el sampler `TPESampler (multivariate=True)`, utilizando la semilla 0, 200 trials en total, de los cuales los 100 primeros corresponden a la fase inicial de muestreo.

Esta idea puede extrapolarse para ajustar, si fuera necesario, el *warp coarsening* e incluso determinar qué distribución de datos corresponde a cada gráfica en una configuración multi-GPU.

Capítulo 4

Análisis de resultados

4.1. Cualidades del sistema

Los experimentos se realizaron sobre una máquina alquilada mediante Vast.ai, con Ubuntu 22.04.5 LTS y kernel 6.8.0-59-generic. La máquina estaba equipada con un procesador AMD Ryzen Threadripper PRO 3995WX y 24 GiB de memoria RAM. Durante los experimentos, el entorno exponía 30 CPUs lógicas, con un único hilo por núcleo.

La GPU utilizada fue una NVIDIA GeForce RTX 3090 con 24 GiB de memoria, arquitectura Ampere y *compute capability* 8.6. La GPU dispone de 82 SMs.

El entorno software empleado incluye CUDA 13.0, PyTorch 2.13.0 con soporte para CUDA 13.0, Triton 3.7.1 y cuDNN 9.2. Para el análisis de rendimiento se utilizó NVIDIA Nsight Compute 2025.3.1. La compilación se realizó con GCC 11.4.0, G++ 11.4.0 y CMake 4.4.3.

Para más detalles sobre el proyecto, los resultados obtenidos, las dudas surgidas durante el desarrollo o cualquier aspecto no tratado en este apartado, se puede consultar el Anexo F.

4.2. Comandos NCU

Para realizar el perfilado con NVIDIA Nsight Compute se utilizaron una serie de opciones comunes a todas las implementaciones. La opción `--set full` activa el conjunto completo de secciones y métricas predefinidas por Nsight Compute, mientras que `--apply-rules no` desactiva la ejecución automática de reglas de análisis.

Para reducir la variabilidad entre ejecuciones se utilizó `--clock-control base`, fijando las frecuencias empleadas durante el perfilado. La opción `-f` permite sobrescribir informes `.ncu-rep` existentes.

Como los kernels se ejecutan desde Python y PyTorch, se añadió `--target-processes all` para incluir también los procesos hijo. Además, el perfilado se delimitó mediante rangos NVTX utilizando `--nvtx` y `--nvtx-include`, con rangos distintos para CUDA,

Triton, Torch SDPA y cuDNN SDPA.

Finalmente, la opción `-o` se utiliza para indicar el nombre y la ruta del informe generado por Nsight Compute.

4.2.1. Por qué utilizamos NCU para medir el rendimiento

Para medir el rendimiento utilizamos NVIDIA Nsight Compute en lugar de ejecutar el kernel un gran número de veces y calcular el tiempo medio. Esta decisión se debe a varios motivos:

1. NCU realiza, cuando son necesarios, distintos *replays* del kernel para recopilar las métricas solicitadas. Además, vacía las cachés entre *replays* y permite fijar las frecuencias de la GPU mediante `--clock-control base`. Esto permite obtener unas condiciones de ejecución más reproducibles entre las distintas implementaciones [16].

2. El tiempo de ejecución se analiza conjuntamente con métricas como el *throughput*, los fallos de caché, la ocupación o los distintos motivos de parada de los *warps*. Obtener tanto las métricas microarquitecturales como la duración del kernel mediante NCU permite relacionarlas bajo unas mismas condiciones de ejecución, evitando introducir un segundo procedimiento de medición con condiciones potencialmente diferentes.

3. Ejecutar un kernel un gran número de veces consecutivas modifica las condiciones bajo las que se realiza la medición. Las cachés pueden quedar precargadas y las frecuencias de la GPU pueden variar durante las ejecuciones. En este caso estaríamos midiendo principalmente el comportamiento del kernel en régimen estable, mientras que NCU permite comparar las distintas implementaciones bajo unas condiciones controladas y comunes.

4. Debido a la naturaleza de la máquina utilizada (alquilada y compartida), las mediciones realizadas mediante CUDA Events no proporcionaban resultados fiables. Los kernels de 8K, cuyos tiempos de ejecución no alcanzaban el milisegundo, llegaban a registrar tiempos cercanos al segundo, además de presentar una elevada desviación estándar.

5. En una ejecución real se procesan múltiples cabezas, *batches* y se encadenan múltiples capas. Si se mide el tiempo ejecutando repetidamente el mismo kernel dentro de un bucle `for`, los resultados pueden resultar optimistas debido a los *cache hits* producidos por reutilizar una y otra vez los mismos datos.

   Una vez procesados los datos correspondientes a una capa, estos no suelen volver a utilizarse de la misma forma. De la operación de atención, principalmente la salida O continúa hacia las siguientes operaciones, mientras que Q, K y V se volverán a calcular para la siguiente capa. Por tanto, mantener artificialmente

los mismos datos en caché mediante ejecuciones consecutivas puede representar peor el comportamiento esperado del kernel dentro de un modelo completo.

4.3. Longitud de secuencia 8192

4.3.1. Ciclos, frecuencia y tiempo estimado

Implementación	Frecuencia SM (GHz)	Ciclos	Tiempo (μs)
CUDA	1,3923	1 270 992	911,872
Triton	1,3933	1 258 610	902,624
Torch SDPA – split-KV	1,3909	1 002 523	719,840
Torch SDPA – combine	1,3787	48 946	35,456
Torch SDPA – total	–	1 051 469	755,296
cuDNN SDPA	1,3906	1 158 729	832,416

Tabla 4.1. Frecuencia del SM, ciclos transcurridos y tiempo de ejecución para una longitud de secuencia de 8192.

Observando las frecuencias de la Tabla 4.1, podemos ver que la versión cuDNN SDPA toma menos ciclos SM que las versiones CUDA y Triton, lo que se traduce en una mejora de rendimiento con respecto a las mismas. A su vez, la versión Torch parece ser más rápida que cuDNN si sumamos ambas partes.

La versión Torch SDPA emplea un algoritmo distinto a FlashAttention, denominado *split-KV*. Sabemos que el algoritmo es *split-KV* porque NCU reporta los nombres de las funciones que lo componen como `flash_fwd_splitkv_kernel` y `flash_fwd_splitkv_combine_kernel`. El objetivo de esta memoria no es describir dicho algoritmo, y se plantea que puedan existir algoritmos posiblemente más eficientes para el cálculo de la atención, como puede ser *split-KV*. Es por este motivo que *split-KV* no recibe un análisis detallado de qué puede hacer o cómo puede hacerlo apoyado en las tablas de esta longitud de secuencia.

4.3.2. Throughput pipelines

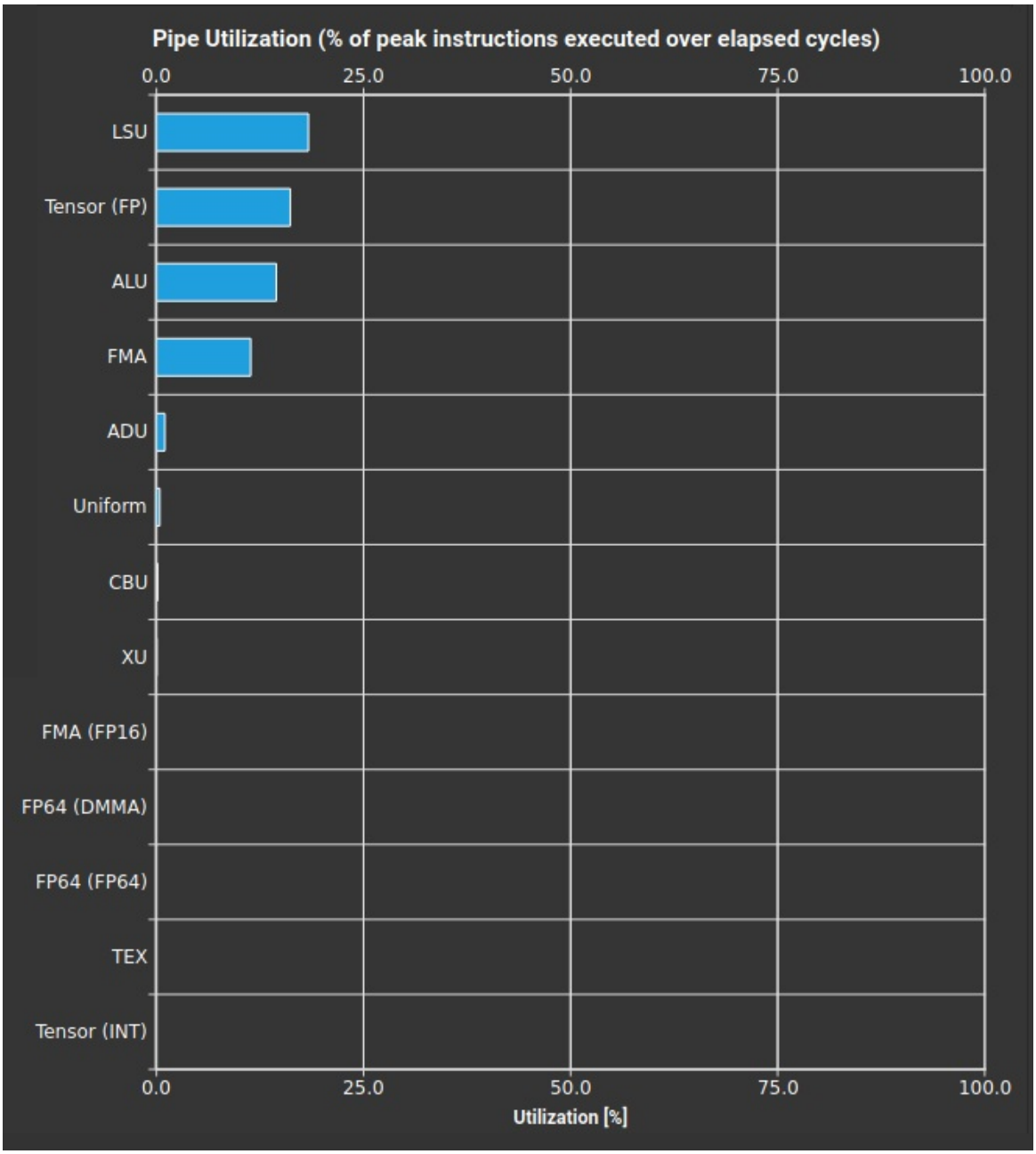


Figura 4.1. Throughput de los pipelines de ejecución para CUDA con 8K, expresado como porcentaje del máximo sostenido durante los ciclos transcurridos.

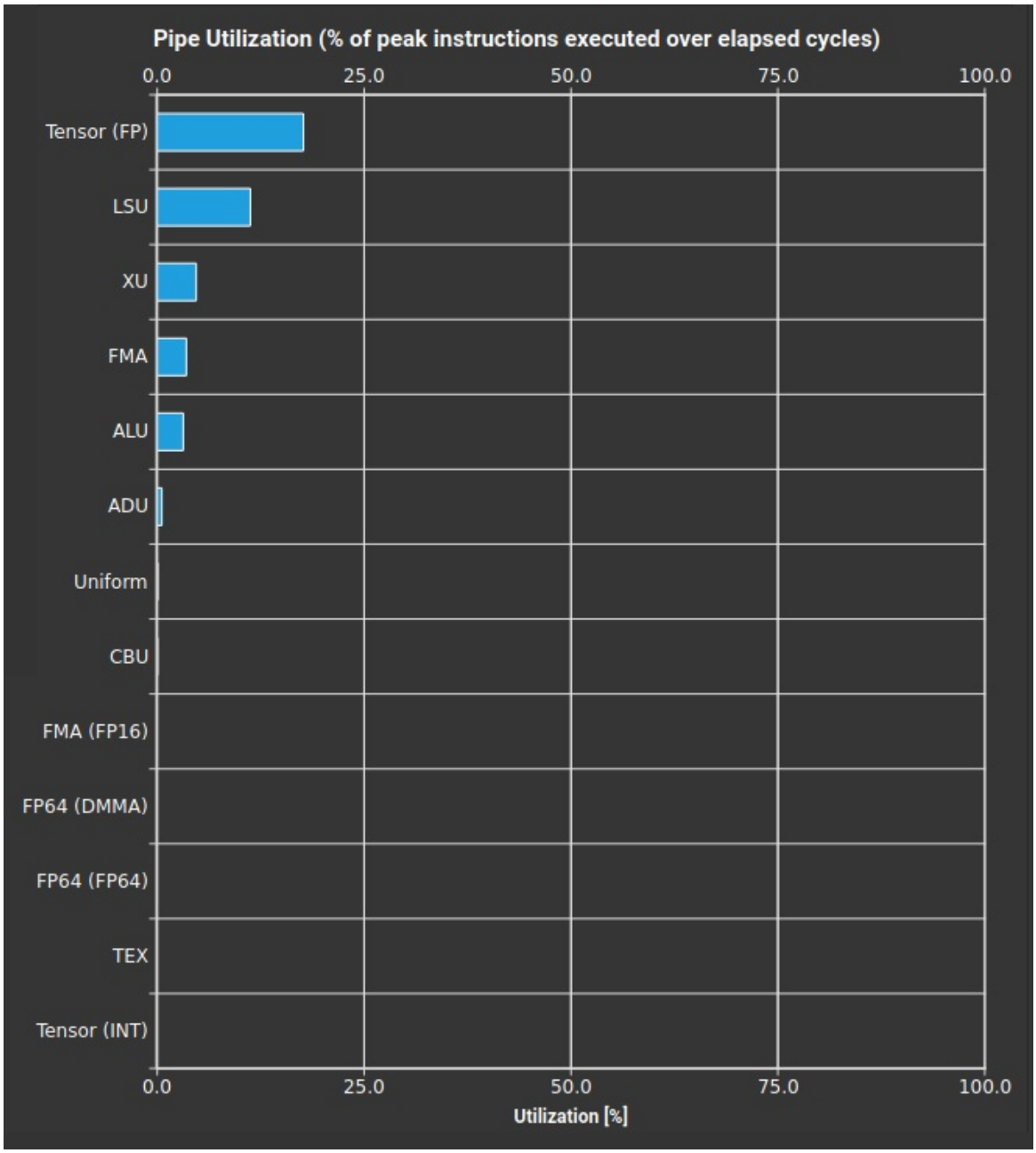


Figura 4.2. Throughput de los pipelines de ejecución para cuDNN con 8K, expresado como porcentaje del máximo sostenido durante los ciclos transcurridos.

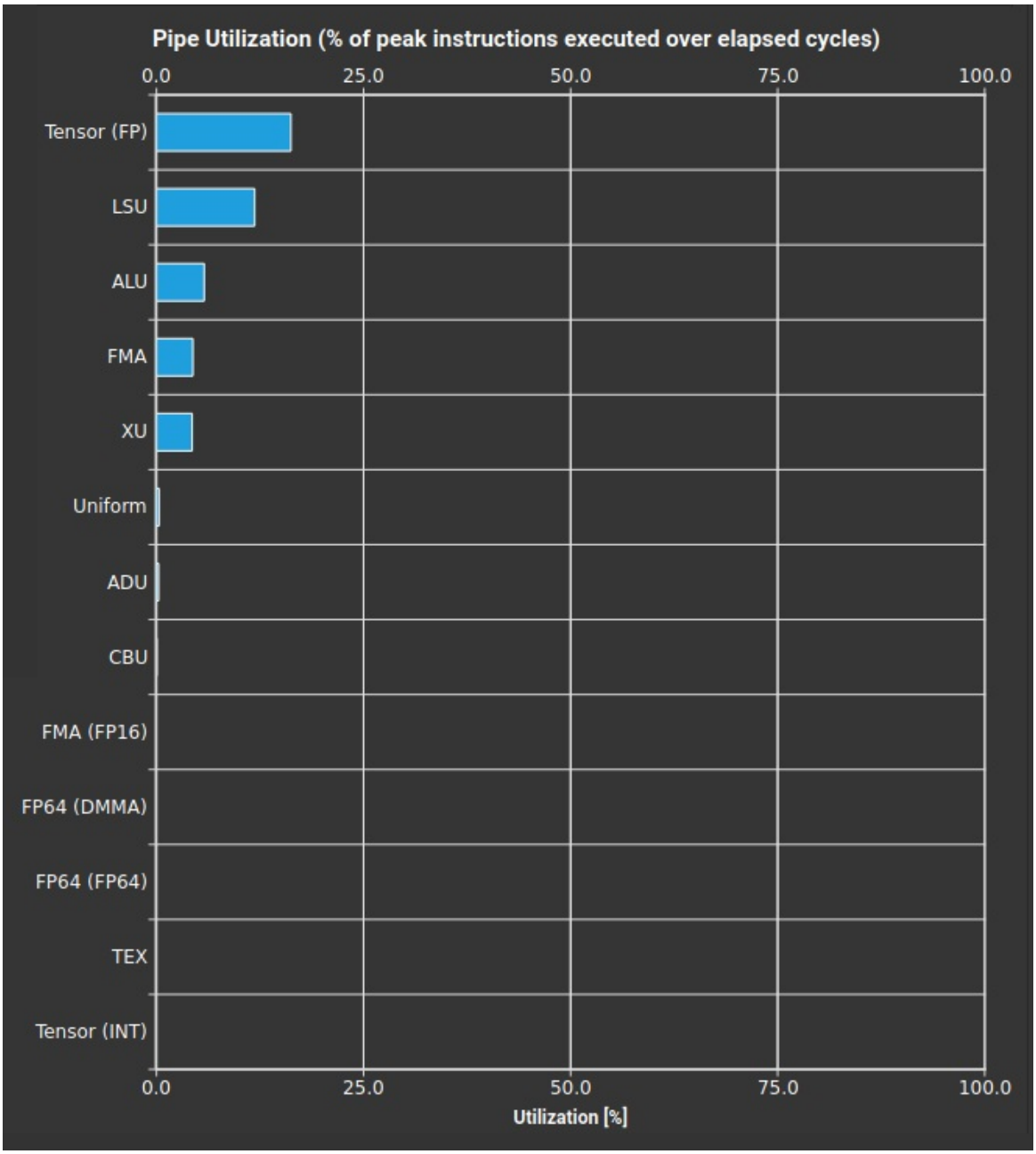


Figura 4.3. Throughput de los pipelines de ejecución para Triton con 8K, expresado como porcentaje del máximo sostenido durante los ciclos transcurridos.

Pipeline	Operaciones ejecutadas
FMA	Operaciones aritméticas FP32 (suma, multiplicación y multiplicación-acumulación) y determinadas operaciones enteras.
ALU	Aritmética entera, operaciones lógicas y desplazamientos de bits.
XU	Funciones matemáticas especiales y conversiones de tipos de datos.
FP16	Operaciones aritméticas de media precisión (FP16).
FP64	Operaciones aritméticas de doble precisión (FP64).
Tensor (FP)	Multiplicaciones y acumulaciones matriciales en FP16, BF16 y TF32.
Tensor (INT)	Multiplicaciones y acumulaciones matriciales enteras (INT8 e INT4).
LSU	Instrucciones de carga, almacenamiento e intercambio de datos.
TEX	Operaciones de acceso y muestreo de texturas.
ADU	Operaciones de direccionamiento y determinadas operaciones de control.
CBU	Instrucciones de control de flujo, saltos y convergencia.
Uniform	Operaciones sobre datos uniformes entre los hilos de un warp.

Tabla 4.2. Tipos de operaciones ejecutadas por los pipelines de la NVIDIA RTX 3090 (Ampere, SM 8.6).

Las Figuras 4.1, 4.2 y 4.3 muestran el throughput de los distintos pipelines de ejecución para una longitud de secuencia de 8K, comparando las implementaciones CUDA, cuDNN y Triton, respectivamente. La figura 4.2 describe el % de uso de cada pipeline de la GPU. Los % de uso se calculan de acuerdo a la formula:

$$U_{\text{pipe}}(\,\%) = \frac{N_{\text{inst}}}{C_{\text{elapsed}} \cdot R_{\text{peak}}} \cdot 100 \tag{4.1}$$

Donde

- N_{inst}: número de instrucciones de warp ejecutadas por el pipeline.
- C_{elapsed}: número de ciclos transcurridos durante la ejecución del kernel.
- R_{peak}: throughput máximo sostenido del pipeline, expresado en instrucciones de warp por ciclo.

Las barras azules constituyen el porcentaje de instrucciones emitidas a un pipeline determinado sobre el total de instrucciones que podía realizar durante la duración total de nuestro kernel.

Podemos apreciar que ninguno de los pipelines consigue una utilización mayor al 25 % en ninguno de los casos. El kernel CUDA deja de utilizar completamente el pipeline XU, pero presenta mayor throughput de los pipelines FMA y ALU en comparación con cuDNN y Triton.

Que el throughput sea menor al 25 % puede indirectamente decir que estamos desaprovechando los recursos de cómputo, pero no refleja la realidad del kernel.

En el caso del kernel cuDNN, el pipeline de las exponenciales (XU) tiene un ratio de utilización bajo, pero esto no impide que no haya contención en el pipeline si todos los warps de un SM piden utilizarlo al mismo tiempo. Simplemente indica que hay un % teórico de ciclos durante la ejecución del kernel en el que puede recibir instrucciones y no lo hace.

Dado que nuestro kernel no puede ejecutar suficientes instrucciones para llegar al 100 % de utilización en todos los pipelines, que debe ocurrir en códigos sin accesos a memoria y sin dependencias entre instrucciones, surge la pregunta: ¿Y si fusionamos más kernels? El compilador puede reordenar más instrucciones y el paralelismo a nivel de warps permite aprovechar los pipelines disponibles. No siempre es directo, y menos en arquitecturas de deep learning en las que hay fuertes dependencias entre las capas, pero puede ser una opción para otro tipo de dominios.

Este tema hace tambalear el enfoque CUDA de warps y SIMT: hasta ahora, el paralelismo a nivel de pipelines se debía exclusivamente a un intercalado de warps (si hay un warp que use un pipeline distinto a uno ocupado, se encola y hay dos pipelines activos simultáneamente). El problema es que la tendencia en los últimos años es reducir el número de warps por SM, que dificulta o impide aprovechar este paralelismo.

Una posible solución sería emplear CUDA streams, que permiten la ejecución concurrente de CTAs de distintos kernels en una misma SM. Quizás el problema sea la baja libertad de programación: no podemos decir “El kernel A, que es intenso en enteros, y el kernel B, intenso en FMAs, deben ejecutarse concurrentemente en el mismo SM”, y ni la máquina ni el compilador pueden determinar que ese caso se dé de manera estática.

Tampoco tenemos —ninguna— garantía de que aprovechar mejor los pipelines pueda resultar en una mejora de rendimiento. La bajada de la frecuencia por parte del procesador para mitigar el sobrecalentamiento ligado al uso intensivo de las unidades de cómputo está presente también en GPUs. Podemos pensar que no es problema nuestro y sí de la refrigeración o del fabricante, pero rara vez podemos cambiar los planos enteros de un clúster para subsanarlo, por lo que es un problema con el que tenemos que lidiar de manera directa.

Estas gráficas se incluyen de nuevo en 64K. El repositorio del proyecto (Anexo F) contiene estos datos y más en la carpeta ‘profile/reports-images/‘.

4.3.3. Ocupancia

Implementación	CTAs	Warps/CTA	Warps totales
CUDA	256	2	512
Triton	64	8	512
cuDNN SDPA	64	8	512

Tabla 4.3. Tamaño del grid y número de warps lanzados para una longitud de secuencia de 8192.

De acuerdo con la Tabla 4.3, podemos ver que las versiones CUDA, Triton y cuDNN lanzan el mismo número de warps, pero, si bien Triton y cuDNN lanzan 64 CTAs, CUDA lanza 256.

Implementación	Registros/thread	SRAM/CTA (KiB)
CUDA	168	25
Triton	199	97
cuDNN SDPA	209	67

Tabla 4.4. Uso de registros y SRAM por bloque para una longitud de secuencia de 8192.

Si atendemos al número de registros de la Tabla 4.4, podemos ver que la versión CUDA es la versión que menos registros consume por thread. En términos de SRAM, el kernel CUDA emplea 24 KiB de memoria, que se corresponden con 3 buffers de 8192 bytes de memoria y 1kb reservado.

La cifra de registros empleados por CUDA (168) dista de lo que habíamos calculado previamente (120). Es importante recalcar que no necesariamente es malo: Optuna optimiza el número mínimo de CTAs que deben funcionar en un SM con `__launch_bounds__`, y nvcc elige un reparto de registros acorde. Si nvcc determina que puede mantener registros de indexado porque el número de CTAs/SM se lo permite, no usarlos sería un error. La cuenta de registros (120) se calcula porque nos permite identificar si ocurren *spills* o si es necesario más paralelismo para reducir el número de registros/warp.

Implementación	Ocup. teórica (%)	Ocup. alcanzada (%)	Warps activos/SM
CUDA	16,67	11,74	5,63
Triton	16,67	16,54	7,94
cuDNN SDPA	16,67	16,66	8,00

Tabla 4.5. Ocupancia teórica, ocupancia alcanzada y número medio de warps activos por SM para una longitud de secuencia de 8192.

Las métricas de la Tabla 4.5 pueden arrojar pistas sobre el rendimiento de la aplicación. Si atendemos a las versiones Triton, cuDNN y CUDA, todas ellas tienen la misma ocupancia teórica, pero solo Triton y cuDNN tienen una ocupancia alcanzada cercana al $16,67\,\%$, en comparación con CUDA, que alcanza un $11,74\,\%$.

La cifra de warps activos/SM indica que Triton y cuDNN tienen aproximadamente 2,3 warps más por SM. No se puede concluir todavía que el kernel CUDA sea más lento simplemente debido a una menor ocupancia; $T_\text{cuDNN} \approx 832\,\mu s$ y $T_\text{Triton} \approx 903\,\mu s$ aun teniendo prácticamente la misma ocupancia. Sin embargo, sí se puede concluir que, para esta longitud de secuencia, CUDA no lanza un número de warps suficiente para aprovechar al máximo la ocupancia teórica. Esto es interesante, porque dichos kernels lanzan la misma cantidad de warps totales.

La explicación reside en que CUDA lanza 256 CTAs, mientras que cuDNN y Triton lanzan 64. cuDNN y Triton usan menos SMs; ambos están limitados a 1 CTA por SM, utilizando como máximo 64 SMs. CUDA puede mantener hasta 4 CTAs por SM, siendo la memoria compartida el factor limitante. Con un grid de 256 CTAs para 82 SMs, algunos SMs pueden quedarse con una carga de trabajo desigual, porque $82 \cdot 4 > 256$. No se puede asumir que los CTAs se asignen a un SM hasta alcanzar su ocupancia máxima, porque en ese caso $64 \cdot 4 = 256$ y la ocupancia estaría cerca del máximo.

Entonces, la ocupancia / uso de SMs no nos sirve de momento como evidencia porque, si bien es cierto que CUDA tiene menor ocupancia y usa más SMs, empata con Triton y es más lenta que cuDNN.

4.3.4. Cantidad de instrucciones ejecutadas

Implementación	Instrucciones ejecutadas
CUDA	99 973 632
Triton	48 423 936
cuDNN SDPA	35 971 584

Tabla 4.6. Número de instrucciones ejecutadas para una longitud de secuencia de 8192.

De acuerdo con la Tabla 4.6, salta a la vista que CUDA emplea aproximadamente 99 millones de instrucciones, más del doble que Triton y más del doble que cuDNN. Aun así, CUDA consigue un tiempo de ejecución parecido al de Triton, y no es el doble de peor que cuDNN. El motivo de que Triton pueda rendir de la misma manera que CUDA puede deberse a que las operaciones que compute tengan un menor throughput. Lo analizamos en las siguientes tablas.

Implementación	Cantidad de instrucciones	Sobre el total (%)
CUDA	2 179 072	2,18
Triton	524 288	1,08
cuDNN SDPA	264 192	0,73

Tabla 4.7. Número de instrucciones de tipo `SHFL` y porcentaje sobre el total para una longitud de secuencia de 8192.

De acuerdo con la tabla 4.7, CUDA ejecuta más instrucciones SHFL que las versiones Triton y cuDNN.

Implementación	Cantidad de instrucciones	Sobre el total (%)
CUDA	7 346 688	7,35
Triton	5 604 352	11,57
cuDNN SDPA	5 082 112	14,13

Tabla 4.8. Número de instrucciones de carga/almacenamiento y porcentaje sobre el total para una longitud de secuencia de 8192.

De acuerdo con la tabla 4.8, CUDA ejecuta más instrucciones de carga y almacenamiento que las versiones Triton y cuDNN.

Implementación	Cantidad de instrucciones	Sobre el total (%)
CUDA	49 837 056	49,85
Triton	34 968 576	72,21
cuDNN SDPA	26 253 312	72,98

Tabla 4.9. Número de instrucciones de punto flotante y porcentaje sobre el total para una longitud de secuencia de 8192.

De acuerdo con la tabla 4.9, CUDA ejecuta más instrucciones de punto flotante que las versiones Triton y cuDNN.

Implementación	Cantidad de instrucciones	Sobre el total (%)
CUDA	8 388 608	8,39
Triton	8 388 608	17,32
cuDNN SDPA	8 388 608	23,32

Tabla 4.10. Número de instrucciones Tensor Core (`HMMA`) y porcentaje sobre el total para una longitud de secuencia de 8192.

De acuerdo con la tabla 4.10, CUDA ejecuta exactamente las mismas operaciones matriciales que las versiones Triton y cuDNN.

Implementación	Cantidad de instrucciones	Sobre el total (%)
CUDA	0	0,00
Triton	2 228 224	4,60
cuDNN SDPA	2 228 224	6,19

Tabla 4.11. Número de instrucciones exponenciales (`MUFU.EX2`) y porcentaje sobre el total para una longitud de secuencia de 8192.

De acuerdo con la tabla 4.11, CUDA no ejecuta ninguna operación exponencial en comparación con Triton y cuDNN.

Implementación	Cantidad de instrucciones	Sobre el total (%)
CUDA	31 311 872	31,32
Triton	2 984 960	6,16
cuDNN SDPA	2 320 384	6,45

Tabla 4.12. Número de instrucciones enteras y porcentaje sobre el total para una longitud de secuencia de 8192.

De acuerdo con la tabla 4.12, CUDA ejecuta aproximadamente 10.5 veces más operaciones sobre enteros en comparación con Triton y 13.5 con cuDNN.

Llegados a este punto, no podemos simplemente afirmar que CUDA ejecuta más instrucciones que Triton y cuDNN, pero que las instrucciones de estos últimos tienen más latencia que las de CUDA. CUDA ejecuta más instrucciones de los principales tipos que cuDNN y Triton.

Un dato a remarcar es que todas las implementaciones ejecutan exactamente la misma cantidad de instrucciones matriciales. La conclusión es que CUDA realiza

mucho más trabajo de indexación y acceso a datos que Triton y cuDNN para la misma cantidad de operaciones tensoriales.

4.3.5. Memoria

Implementación	Lecturas DRAM (MiB)	Escrituras DRAM (MiB)	Total DRAM (MiB)
CUDA	6,09	1,95	8,05
Triton	6,02	1,91	7,93
cuDNN SDPA	6,02	1,94	7,96

Tabla 4.13. Volumen de datos leído y escrito en DRAM para una longitud de secuencia de 8192.

De acuerdo con la Tabla 4.13, todos los kernels acceden a una cantidad de datos semejante, siendo el kernel CUDA el que más datos carga y almacena en memoria. Sin embargo, la diferencia no es lo suficientemente grande como para afirmar que un kernel pueda rendir más que otro debido al tráfico de memoria DRAM.

Implementación	Lecturas L2 (MiB)	Escrituras L2 (MiB)	Total L2 (MiB)	Hit rate L2 (%)
CUDA	1028,00	2,00	1030,00	99,39
Triton	258,00	2,00	260,00	97,65
cuDNN SDPA	258,00	2,00	260,00	97,72

Tabla 4.14. Volumen de datos leído y escrito en la caché L2 y tasa de aciertos para una longitud de secuencia de 8192.

La Tabla 4.14 sí arroja datos interesantes de rendimiento. El kernel CUDA accede al cuádruple de MiB de memoria en comparación con Triton y cuDNN, mientras que Triton y cuDNN acceden exactamente a la misma cantidad. Un alto *hit rate* se debe en gran medida a la longitud de secuencia.

$$3 \text{ matrices} \cdot 128 \text{ elementos} \cdot 2 \text{ bytes/BF16} \cdot 8192 = 6\,291\,456 \text{ bytes} = 6 \text{ MiB}.$$

Estos 6 MiB coinciden con la capacidad de la caché L2 de la RTX 3090, por lo que el hardware puede mantener gran parte de estos datos en L2. Para longitudes de secuencia mayores, el tamaño de las matrices supera la capacidad de la caché, por lo que podría aumentar el número de fallos de caché.

Implementación	Wavefronts teóricos	Wavefronts ejecutados	Conflictos de banco	Conflictos (%)
CUDA	20 004 687	20 006 228	1 541	0,0077
Triton	19 698 265	19 699 012	747	0,0038
cuDNN SDPA	17 371 392	17 371 392	0	0,0000

Tabla 4.15. Número de wavefronts teóricos y ejecutados en memoria compartida, conflictos de banco y proporción de conflictos para una longitud de secuencia de 8192.

De acuerdo con la tabla 4.15, CUDA ejecuta más wavefronts que Triton y que cuDNN. Los conflictos de banco ocurren en CUDA y Triton, pero son despreciables en comparación con el total de wavefronts ejecutados.

Implementación	Evict first (sectores)	Hit rate (%)	Evict last (sectores)	Hit rate (%)
CUDA	5 783	99,64	33 619 968	99,61
Triton	5 584	99,55	0	–
cuDNN SDPA	6 615	84,47	0	–

Tabla 4.16. Número de sectores accedidos en L2 mediante las políticas *evict first* y *evict last* para una longitud de secuencia de 8192.

La Tabla 4.16 evidencia que, si bien todas las versiones usan el *hint evict first*, solo el kernel CUDA emplea el *hint evict last*. Este factor se puede relacionar con el ligero aumento del *hit rate* del kernel CUDA en comparación con Triton y cuDNN.

4.3.6. Throughput, sincronizaciones, warps activos

Implementación	Ciclos con una instrucción emitida (%)
CUDA	28,10
Triton	15,14
cuDNN SDPA	12,20

Tabla 4.17. Porcentaje de ciclos en los que se emitió una instrucción para una longitud de secuencia de 8192. En NCU, esta métrica corresponde a `smsp__issue_active.avg.pct_of_peak_sustained_active`.

De acuerdo con la Tabla 4.17, podemos ver uno de los principales motivos por los que el kernel CUDA es capaz de conseguir su rendimiento a pesar de generar

aproximadamente 4 veces más tráfico de memoria L2 y ejecutar el doble de instrucciones que los kernels de Triton y cuDNN. Es cierto que ejecuta el doble de instrucciones, pero también es cierto que el porcentaje de ciclos en los que se emite una instrucción es aproximadamente el doble, es consistente con un mayor paralelismo a nivel de instrucción. La pregunta es por qué puede ocurrir esto si el número total de warps es el mismo.

Implementación	Barrier	Math Pipe Throttle	Wait	Short Scoreboard	Issue Active (%)
CUDA	0,14	2,68	1,33	0,21	28,10
Triton	0,75	7,98	2,01	0,68	15,14
cuDNN SDPA	0,50	12,46	1,44	0,10	12,20

Tabla 4.18. Número medio de warp-ciclos de *stall* por instrucción emitida para diferentes causas de *stall*, junto con el porcentaje de ciclos con emisión, para una longitud de secuencia de 8192.

Para responder a esa pregunta, primero tenemos que interpretar los datos de la Tabla 4.18. Los datos de NCU son ambiguos. Por ejemplo, la descripción asociada a `smsp__average_warps_issue_stalled_math_pipe_throttle_per_issue_active.ratio` es:

> `average # of warps resident per issue cycle, waiting for an execution pipe to be available`
>
> `(This ratio metric represents the value expressed as a ratio across all sub-unit instances)`

La pregunta es: si la medida representa la media de warps residentes esperando a que una *execution pipe* esté disponible, ¿por qué la fila de cuDNN tiene un valor de 12,46 si solo puede mantener 8 warps residentes por SM?

La forma de obtener ese número es:

$$R_{\text{stall}} = \frac{\text{warp-ciclos acumulados en Math Pipe Throttle}}{\text{ciclos con emisión}}. \tag{4.2}$$

Para cuDNN, el valor de *Math Pipe Throttle* es 12,46, por lo que:

$$12{,}46 = \frac{W_{\text{math}}}{C_{\text{issue}}}. \tag{4.3}$$

Como *Issue Active* es 12,20 %, se cumple que:

$$C_{\text{issue}} = 0{,}1220\, C_{\text{active}}. \tag{4.4}$$

Sustituyendo en la ecuación anterior:

$$12{,}46 = \frac{W_{\text{math}}}{0{,}1220\, C_{\text{active}}}. \tag{4.5}$$

Despejando W_{math}:

$$W_{\text{math}} = 12{,}46 \cdot 0{,}1220 \cdot C_{\text{active}} \approx 1{,}52\, C_{\text{active}}. \tag{4.6}$$

Por tanto:

$$\frac{W_{\text{math}}}{C_{\text{active}}} \approx 1{,}52. \tag{4.7}$$

Es decir, hay de media 1,52 warps que se encuentran en estado de *Math Pipe Throttle* por ciclo activo del SMSP.

Si dividimos entre los 2 warps activos por scheduler de cuDNN:

$$P_{\text{math}} = \frac{1{,}52}{2} = 0{,}76. \tag{4.8}$$

Esto significa que aproximadamente el 76 % de los warp-ciclos activos de cuDNN se encuentran en estado de *Math Pipe Throttle*. Dicho de otra forma, un warp activo pasa, de media, aproximadamente el 76 % de sus ciclos de residencia esperando a que esté disponible la *execution pipe* requerida.

Implementación	**Barrier (%)**	**Math Pipe Throttle (%)**	**Wait (%)**	**Short Scoreboard (%)**
CUDA	2,37	46,34	22,96	3,65
Triton	5,72	60,51	15,20	5,18
cuDNN SDPA	3,06	76,05	8,76	0,61

Tabla 4.19. Porcentaje de warp-ciclos activos asociados a diferentes causas de *stall* para una longitud de secuencia de 8192.

La interpretación de la Tabla 4.19 es que tanto la versión CUDA como Triton y cuDNN presentan una proporción relativamente baja de warp-ciclos activos asociados a barreras (`__syncthreads()`) y *Short Scoreboard* (principalmente operaciones de memoria compartida).

Sin embargo, en *Math Pipe Throttle*, CUDA presenta un 46,34 % de los warp-ciclos activos, Triton un 60,52 % y cuDNN un 76,05 %. CUDA presenta, sin embargo, un valor mayor en *Wait*, con un 22,96 %, que representa la proporción de warp-ciclos activos en los que un warp espera a que se resuelva una dependencia de ejecución de latencia fija.

4.3.7. Conclusión y análisis para longitud de secuencia 8192

Para esta longitud de secuencia podemos apreciar:

1- En primer lugar, ninguno de los kernels llega a usar al 100 % la GPU. Esto lo podemos ver en que el número de CTAs lanzados por los kernels Triton y cuDNN ni siquiera usa todas las SMs, quedándose en 64. Similarmente, CUDA lanza más CTAs pero su ocupancia real acaba resultando menor. Esto hace que las diferencias de rendimiento no tengan un motivo tan visible.

2- La fortaleza principal del kernel CUDA es enmascarar la latencia con paralelismo a nivel de instrucción. Mientras que el kernel CUDA lanza cuatro veces más transferencias de memoria L2 y computa el doble de instrucciones que Triton, su rendimiento en ciclos y en tiempo es parejo. Durante la etapa de desarrollo, este tipo de *trade-offs* es difícil de calcular: ¿cómo sabemos cuándo un kernel va a ser capaz de esconder la latencia a cambio de un mayor nivel de paralelismo?

3- El kernel cuDNN queda por encima de Triton, que queda a su vez por encima de CUDA.

4- El número de HMMAs es idéntico. Esto indica que el trabajo principal realizado mediante Tensor Cores es el mismo, por lo que las diferencias aparecen en el trabajo auxiliar y en la forma de planificar y alimentar ese cálculo.

5- Los conflictos de bancos de memoria compartida son despreciables en los tres kernels, por lo que no constituyen un cuello de botella relevante para esta longitud de secuencia.

6- cuDNN presenta un *Math Pipe Throttle* mayor que CUDA y Triton y, aun así, es el kernel más rápido. Por tanto, un mayor porcentaje de *stall* en una categoría concreta no implica necesariamente un peor rendimiento global.

Para mejorar el kernel CUDA podemos investigar si, aumentando la fase de exploración de Optuna, encontramos una nueva cuenca que permita sacrificar ocupancia a cambio de utilizar mejor los SMs. Otro tipo de optimizaciones requeriría reescrituras de varias líneas de código del kernel y aumentar el número de parámetros que determinar con Optuna. Con un conjunto de datos tan pequeño, no podemos afirmar "si hago esto, esto va más rápido". Eliminar instrucciones auxiliares o reestructurar el kernel puede, por ejemplo, aumentar el uso de registros o memoria compartida y reducir el número de warps residentes, por lo que no existe ninguna certeza de que el kernel vaya a ser más rápido.

4.4. Longitud de secuencia 16384

4.4.1. Ciclos, frecuencia y tiempo estimado

Implementación	Frecuencia SM (GHz)	Ciclos	Tiempo (μs)
CUDA	1,3928	5 032 443	3609,344
Triton	1,3936	4 972 286	3565,408
Torch SDPA	1,3934	4 685 154	3360,064
cuDNN SDPA	1,3915	4 594 015	3299,200

Tabla 4.20. Frecuencia del SM, ciclos transcurridos y tiempo de ejecución para una longitud de secuencia de 16384.

De acuerdo con la Tabla 4.20, vemos que las frecuencias son lo suficientemente similares como para considerar válidas las métricas obtenidas. El kernel cuDNN es el que emplea un menor número de ciclos, seguido de Torch SDPA, Triton y, por último, CUDA.

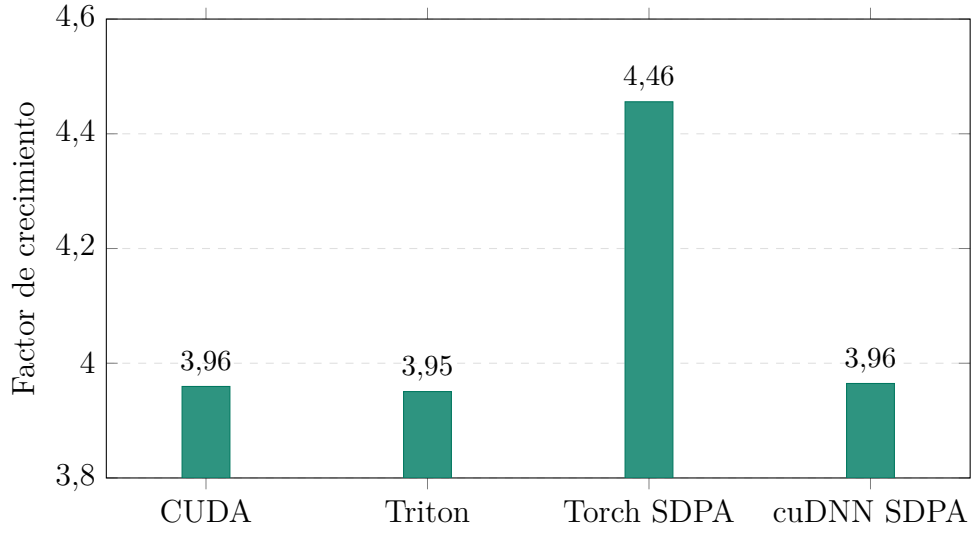


Figura 4.4. Factor de crecimiento del número de ciclos al aumentar la longitud de secuencia de 8192 a 16384.

Si comparamos cómo crece el número de ciclos al aumentar la longitud de secuencia de 8192 a 16384 como se muestra en la Figura 4.4, observamos que CUDA, Triton y cuDNN multiplican su número de ciclos aproximadamente por cuatro. CUDA pasa de 1 270 992 a 5 032 443 ciclos, Triton de 1 258 610 a 4 972 286 y cuDNN de 1 158 729 a 4 594 015. Torch SDPA presenta un crecimiento mayor, pasando de 1 051 469 a 4 685 154 ciclos, aproximadamente 4,46 veces más.

Torch SDPA, además, abandona *split-KV* para esta longitud de secuencia. Esto sugiere que, a partir de cierto tamaño, el overhead introducido por *split-KV* deja de

compensar las ventajas que aporta, por lo que Torch pasa a emplear otra estrategia.

4.4.2. Ocupancia

Implementación	CTAs	Warps/CTA	Warps totales
CUDA	256	4	1 024
Triton	128	8	1 024
Torch SDPA	128	4	512
cuDNN SDPA	128	8	1 024

Tabla 4.21. Tamaño del grid y número de warps lanzados para una longitud de secuencia de 16384.

En la ocupancia también observamos cambios en el número de warps lanzados por las versiones CUDA, Triton y cuDNN SDPA. Todas pasan a emplear 1024 warps, pero CUDA sigue compensando la mitad de warps por CTA con un mayor número de CTAs. Torch SDPA, sin embargo, lanza únicamente la mitad de warps.

Implementación	Registros/thread	SRAM/CTA (KiB)
CUDA	168	25
Triton	199	97
Torch SDPA	255	49
cuDNN SDPA	209	67

Tabla 4.22. Uso de registros y SRAM por bloque para una longitud de secuencia de 16384.

Si atendemos a los registros y SRAM utilizados en la Tabla 4.22, podemos ver que CUDA mantiene el mismo uso de recursos, con 168 registros por thread y 25 KiB de SRAM por CTA. Triton también mantiene 199 registros por thread y 97 KiB de SRAM por CTA, mientras que cuDNN mantiene 209 registros por thread y 67 KiB de SRAM por CTA.

La cifra de 255 registros por thread en Torch SDPA es alarmante. Esta cifra puede indicar que se han utilizado exactamente 255 registros, o que se han requerido más de 255 y se han producido *register spills*. Si es lo segundo, sería recomendable investigar si, reduciendo el trabajo por warp, podemos reducir el número de registros y evitar los *spills* sin perder rendimiento.

Implementación	Ocup. teórica (%)	Ocup. alcanzada (%)	Warps activos/SM
CUDA	25,00	19,95	9,58
Triton	16,67	16,59	7,96
Torch SDPA	16,67	12,53	6,01
cuDNN SDPA	16,67	16,66	8,00

Tabla 4.23. Ocupancia teórica, ocupancia alcanzada y número medio de warps activos por SM para una longitud de secuencia de 16384.

La ocupancia de la Tabla 4.23 experimenta cambios en CUDA. Alcanza ahora una ocupancia teórica del 25 % y una ocupancia real cercana al 20 %, mientras que anteriormente eran aproximadamente del 16 % y 12 %, respectivamente. Además, consigue un mayor número de warps activos por SM. La implementación de Torch SDPA presenta una ocupancia real relativamente baja respecto a la teórica, mientras que Triton y cuDNN se acercan especialmente al máximo teórico.

Este cambio en la ocupancia de CUDA se debe al reúso de datos a nivel de CTA. Anteriormente, para disponer de 4 warps eran necesarios 2 CTAs de 2 warps, que requerían en total 2×25 KiB de SRAM. Ahora, un único CTA con 4 warps sigue necesitando 25 KiB de SRAM, lo que permite mantener más warps residentes por SM. Anteriormente, CUDA veía su ocupancia limitada principalmente por la SRAM.

4.4.3. Cantidad de instrucciones ejecutadas

Implementación	Instrucciones ejecutadas
CUDA	367 601 664
Triton	193 054 720
Torch SDPA	181 199 360
cuDNN SDPA	143 246 336

Tabla 4.24. Número de instrucciones ejecutadas para una longitud de secuencia de 16384.

Como se observa en la Tabla 4.24, CUDA sigue ejecutando aproximadamente el doble de instrucciones que las demás implementaciones.

Implementación	Cantidad de instrucciones	Sobre el total 16384 (%)	Sobre el total 8192 (%)
CUDA	8 552 448	2,33	2,18
Triton	2 097 152	1,09	1,08
Torch SDPA	2 101 248	1,16	–
cuDNN SDPA	1 052 672	0,73	0,73

Tabla 4.25. Número de instrucciones de tipo `SHFL` y porcentaje sobre el total para una longitud de secuencia de 16384.

Como se observa en la Tabla 4.25, la versión CUDA realiza aproximadamente cuatro veces más instrucciones `SHFL` que Triton y Torch SDPA, y hasta ocho veces más que cuDNN SDPA.

Implementación	Cantidad de instrucciones	Sobre el total 16384 (%)	Sobre el total 8192 (%)
CUDA	25 179 136	6,85	7,35
Triton	22 349 824	11,58	11,57
Torch SDPA	16 837 120	9,29	–
cuDNN SDPA	20 256 768	14,14	14,13

Tabla 4.26. Número de instrucciones de carga/almacenamiento y porcentaje sobre el total para una longitud de secuencia de 16384.

De acuerdo con la Tabla 4.26, Torch SDPA es la implementación que menos instrucciones de carga/almacenamiento realiza, seguida de cuDNN SDPA, Triton y CUDA.

Implementación	Cantidad de instrucciones	Sobre el total 16384 (%)	Sobre el total 8192 (%)
CUDA	199 288 832	54,21	49,85
Triton	139 667 456	72,35	72,21
Torch SDPA	145 762 816	80,44	–
cuDNN SDPA	104 935 424	73,26	72,98

Tabla 4.27. Número de instrucciones de punto flotante y porcentaje sobre el total para una longitud de secuencia de 16384.

De acuerdo con la Tabla 4.27, cuDNN realiza un menor número de instrucciones de punto flotante, seguido de Triton, Torch SDPA y CUDA.

Implementación	Cantidad de instrucciones	Sobre el total 16384 (%)	Sobre el total 8192 (%)
CUDA	33 554 432	9,13	8,39
Triton	33 554 432	17,38	17,32
Torch SDPA	33 554 432	18,52	–
cuDNN SDPA	33 554 432	23,42	23,32

Tabla 4.28. Número de instrucciones Tensor Core (`HMMA`) y porcentaje sobre el total para una longitud de secuencia de 16384.

Según la información mostrada en la Tabla 4.28, observamos que todos los kernels siguen emitiendo la misma cantidad de instrucciones matriciales. De nuevo, la conclusión es que cuDNN necesita menos trabajo auxiliar, si tenemos en cuenta el número total de instrucciones, para realizar las mismas operaciones tensoriales.

Implementación	Cantidad de instrucciones	Sobre el total 16384 (%)	Sobre el total 8192 (%)
CUDA	0	0,00	0,00
Triton	8 912 896	4,62	4,60
Torch SDPA	9 435 136	5,21	–
cuDNN SDPA	8 912 896	6,22	6,19

Tabla 4.29. Número de instrucciones exponenciales (`MUFU.EX2`) y porcentaje sobre el total para una longitud de secuencia de 16384.

En cuanto al número de operaciones exponenciales de la Tabla 4.29, CUDA sigue sin lanzar ninguna operación, como consecuencia de la aproximación exponencial utilizada, mientras que Triton y cuDNN realizan exactamente el mismo número de estas operaciones. Torch SDPA es la implementación que más operaciones exponenciales computa.

Implementación	Cantidad de instrucciones	Sobre el total 16384 (%)	Sobre el total 8192 (%)
CUDA	104 787 968	28,51	31,32
Triton	11 737 088	6,08	6,16
Torch SDPA	9 332 224	5,15	–
cuDNN SDPA	8 966 144	6,26	6,45

Tabla 4.30. Número de instrucciones enteras y porcentaje sobre el total para una longitud de secuencia de 16384.

En cuanto al número de operaciones sobre enteros, observamos en la Tabla 4.30 que CUDA lanza aproximadamente diez veces más operaciones que el resto de im-

plementaciones. Comparado con las aproximadamente 15 veces anteriores, CUDA sigue computando muchas operaciones sobre enteros, pero ha reducido su proporción, probablemente debido al aumento del número de warps por CTA.

En este punto, las implementaciones presentan una situación similar a la anterior. CUDA computa un mayor número de instrucciones que el resto de versiones, pero su tiempo de ejecución no es mucho mayor. La única diferencia notable hasta el momento es que CUDA ha conseguido reducir la proporción de instrucciones sobre enteros.

4.4.4. Memoria

Implementación	Local loads	Local stores	Spilling overhead (%)
CUDA	0	0	0,00
Triton	0	0	0,00
Torch SDPA	512	512	100,00
cuDNN SDPA	0	0	0,00

Tabla 4.31. Accesos a memoria local asociados a *register spilling* para una longitud de secuencia de 16384.

De acuerdo con la Tabla 4.31, podemos ver que solo la implementación Torch SDPA provoca *spills*. Probablemente, si Torch consigue reducir estos *spills* mediante reutilización de registros, recomputación de operaciones, etc., podría acercarse bastante a la implementación cuDNN. Si esto no es posible, convendría investigar si realizar los *spills* sobre SRAM permite aumentar el rendimiento.

Implementación	Lecturas DRAM (MiB)	Escrituras DRAM (MiB)	Total DRAM (MiB)
CUDA	27,07	5,36	32,43
Triton	20,02	4,53	24,56
Torch SDPA	16,79	4,05	20,84
cuDNN SDPA	20,04	4,52	24,56

Tabla 4.32. Volumen de datos leído y escrito en DRAM para una longitud de secuencia de 16384.

De acuerdo con la Tabla 4.32, CUDA sigue requiriendo más bytes de memoria en comparación con Triton, cuDNN y Torch SDPA. Torch SDPA es el kernel que realiza menos lecturas de DRAM.

El aspecto principal de este kernel es que las tres matrices dejan de caber en memoria L2 al mismo tiempo:

$$16384 \times 256 \times 3 = 12\,582\,912 \text{ bytes} = 12 \text{ MiB} > 6 \text{ MiB}. \tag{4.9}$$

Esto implica que, a diferencia de la longitud de secuencia 8192, no todas las matrices pueden permanecer simultáneamente en la caché L2, por lo que el reúso de datos pasa a tener una mayor importancia.

Implementación	Lecturas L2 (MiB)	Escrituras L2 (MiB)	Total L2 (MiB)	Hit rate L2 (%)
CUDA	2063,39	4,00	2067,39	98,69
Triton	1028,00	4,00	1032,00	98,07
Torch SDPA	1028,01	4,12	1032,13	98,37
cuDNN SDPA	1028,00	4,00	1032,00	98,05

Tabla 4.33. Volumen de datos leído y escrito en la caché L2 y tasa de aciertos para una longitud de secuencia de 16384.

La Tabla 4.33 subraya una mejora del kernel CUDA en comparación con la versión de longitud de secuencia 8192: el kernel reduce a aproximadamente el doble la proporción de accesos a L2 con respecto a Triton y Torch SDPA. El *hit rate* sigue siendo el más elevado en CUDA. El motivo principal es que, al tener más warps por CTA, se reutilizan más los datos de la L2. Este reúso sigue siendo menor que el que pueden hacer Triton o cuDNN, y lo notamos en el doble de transferencias L2 o en los bytes extra leídos de DRAM, pero se compensa en cierta medida con un *hit rate* ligeramente más elevado en L2.

Implementación	Wavefronts teóricos	Wavefronts ejecutados	Conflictos de banco	Conflictos (%)
CUDA	79 521 184	79 525 006	3 822	0,0048
Triton	78 619 341	78 620 969	1 628	0,0021
Torch SDPA	54 203 725	54 205 339	1 614	0,0030
cuDNN SDPA	69 280 226	69 280 280	54	0,0001

Tabla 4.34. Número de wavefronts teóricos y ejecutados en memoria compartida, conflictos de banco y proporción de conflictos para una longitud de secuencia de 16384.

En la Tabla 4.34 se puede apreciar que todos los kernels incurren en conflictos de banco. Estos conflictos pueden deberse a determinados patrones de acceso a SRAM que no son completamente libres de conflictos, ya sea por los accesos realizados explícitamente por el kernel o por instrucciones generadas por el compilador. Sin embargo, la proporción de conflictos es extremadamente reducida en todas las implementaciones, por lo que no parece constituir un factor relevante para el rendimiento.

Implementación	Evict first (sectores)	Evict last (sectores)
CUDA	5 873	67 481 204
Triton	12 512	0
Torch SDPA	6 075	0
cuDNN SDPA	11 555	0

Tabla 4.35. Número de sectores accedidos en L2 mediante las políticas *evict first* y *evict last* para una longitud de secuencia de 16384.

En la Tabla 4.35 se vuelve a apreciar que CUDA es la única implementación que utiliza *hints* `evict_last`. De hecho, este comportamiento se mantiene en las secciones posteriores para el resto de longitudes de secuencia.

4.4.5. Throughput, sincronizaciones, warps activos

Implementación	Ciclos con una instrucción emitida (%)	En 8192 (%)
CUDA	28,42	28,10
Triton	15,17	15,14
Torch SDPA	14,90	–
cuDNN SDPA	12,20	12,20

Tabla 4.36. Porcentaje de ciclos en los que se emitió una instrucción para una longitud de secuencia de 16384, junto con los valores obtenidos para 8192. En NCU, esta métrica corresponde a `smsp__issue_active.avg.pct_of_peak_sustained_active`.

De acuerdo con la Tabla 4.36, podemos ver que el porcentaje de ciclos en los que se emite una instrucción se mantiene en cuDNN, en Triton aumenta ligeramente un 0,03 %, una diferencia despreciable, y en CUDA aumenta un 0,32 %, consistente con el aumento de la ocupancia teórica y de la ocupancia alcanzada.

Implementación	Barrier (%)	Math Pipe Throttle (%)	Wait (%)	Short Scoreboard (%)
CUDA	7,92	56,01	13,81	2,89
Triton	5,92	60,45	15,22	5,18
Torch SDPA	8,00	63,79	13,13	0,43
cuDNN SDPA	3,03	76,38	8,79	0,61

Tabla 4.37. Porcentaje de warp-ciclos activos asociados a diferentes causas de *stall* para una longitud de secuencia de 16384.

La Tabla 4.37 es interesante. Triton y cuDNN no cambian apenas las métricas de la Tabla 4.19, pero el kernel CUDA aumenta el número de warp-ciclos esperando en barreras hasta casi un 8 %, aumentan los *throttles* en el *Math Pipe* hasta un 56 %, disminuyen las esperas por dependencias hasta un 13,81 % y las dependencias por *scoreboard* un 2,89 %. Una mayor cantidad de warps por CTA hace que una mayor proporción de warp-ciclos quede limitada por el *Math Pipe Throttle* y es consistente con la reducción relativa de las instrucciones de indexado observada anteriormente, a costa de un incremento en los costes de sincronización.

4.4.6. Conclusión y análisis para longitud de secuencia 16384

El análisis para la longitud de secuencia 16384 es similar al análisis para la longitud de secuencia 8192.

1- Si bien los kernels Triton y cuDNN alcanzan una mayor ocupancia respecto a su ocupancia teórica, el kernel CUDA mantiene una ocupancia alcanzada por debajo de la ocupancia teórica. CUDA mejora la ocupancia alcanzada al instanciar una mayor cantidad de warps por CTA, pero se mantiene por debajo del nuevo techo teórico establecido en un 25 %. CUDA alcanza aproximadamente un 20 %, por lo que no está aprovechando al máximo la ocupancia que puede llegar a tener.

2- El número de accesos de CUDA a memoria L2 pasa de ser aproximadamente el cuádruple que en Triton y cuDNN a ser el doble. Este cambio es consistente con el uso del doble de warps por CTA y, por tanto, con un mayor reúso de datos en SRAM. Esto sugiere que, cuanto más larga sea la longitud de secuencia, más puede compensar aumentar el reúso de datos en SRAM. A pesar de realizar más accesos a L2, sigue siendo el kernel que mayor *hit rate* tiene.

3- Los conflictos de banco aparecen ya en todos los kernels. Resulta llamativo que también ocurra en la implementación cuDNN, implementación oficial de NVIDIA. En cualquier caso, son muy pocos conflictos como para que constituyan un problema prioritario de rendimiento.

4- La implementación Torch SDPA sufre de *register spills*, aunque consigue una latencia cercana a cuDNN. Convendría investigar más a fondo si se pueden evitar

para conseguir una mejora en rendimiento. cuDNN sigue siendo la implementación con menor tiempo de ejecución y menor número de ciclos.

5- La implementación CUDA reduce la proporción de instrucciones sobre enteros de aproximadamente un 31 % a un 28 %. Aumenta el número de *stalls* debidos a barreras hasta un 7,92 %, aumenta los *stalls* asociados a *Math Pipe Throttle* hasta un 56,01 % y reduce los *stalls* asociados a *Wait* hasta un 13,81 % y a *Short Scoreboard* hasta un 2,89 %.

4.5. Longitud de secuencia 32768

4.5.1. Ciclos, frecuencia y tiempo estimado

Implementación	Frecuencia SM (GHz)	Ciclos	Tiempo (μs)
CUDA	1,3923	17 071 075	12246,976
Triton	1,3934	19 834 796	14222,240
Torch SDPA	1,3933	18 503 341	13269,600
cuDNN SDPA	1,3927	18 314 980	13139,744

Tabla 4.38. Frecuencia del SM, ciclos transcurridos y tiempo de ejecución para una longitud de secuencia de 32768.

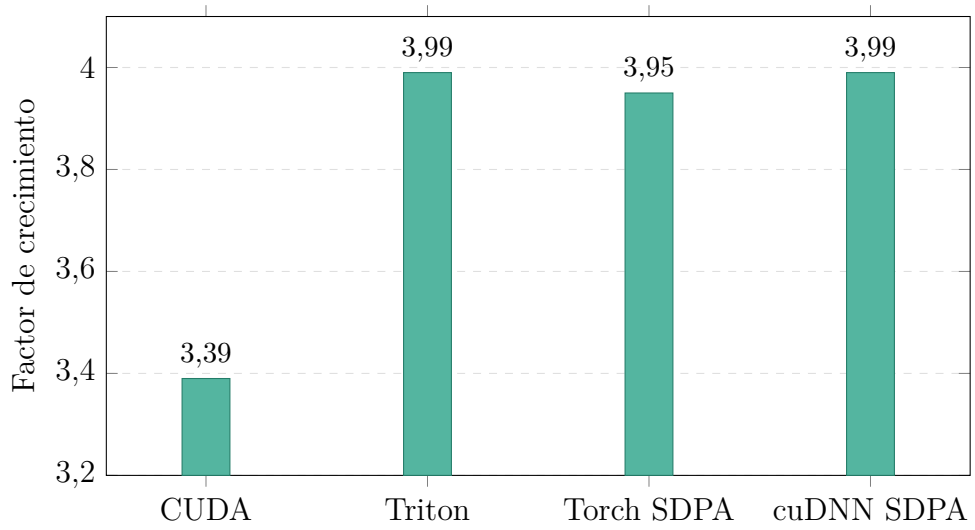


Figura 4.5. Factor de crecimiento del número de ciclos al aumentar la longitud de secuencia de 16384 a 32768.

De acuerdo con la Tabla 4.38, la frecuencia se vuelve a mantener constante, por lo que las métricas se pueden considerar válidas. Observamos que las versiones Torch SDPA y cuDNN SDPA tienen un rendimiento similar, Triton es la versión con peor rendimiento y CUDA la versión más rápida.

Si atendemos a la Figura 4.5, podemos observar que las versiones Triton, Torch SDPA y cuDNN SDPA tardan aproximadamente el cuádruple de ciclos. Al duplicar la longitud de secuencia, estas implementaciones requieren aproximadamente cuatro veces más ciclos, por lo que podemos empezar a observar un coste computacional cuadrático en función de la longitud de secuencia en el mecanismo de atención.

Resulta llamativo que la versión CUDA solo haya escalado aproximadamente $3,4\times$. En las siguientes secciones intentaremos investigar por qué aparece esta mejora de rendimiento al pasar de 16k a 32k, que no estaba presente en el paso de 8k a 16k.

4.5.2. Ocupancia

Implementación	CTAs	Warps/CTA	Warps totales
CUDA	512	4	2 048
Triton	256	8	2 048
Torch SDPA	256	4	1 024
cuDNN SDPA	256	8	2 048

Tabla 4.39. Tamaño del grid y número de warps lanzados para una longitud de secuencia de 32768.

De acuerdo con la Tabla 4.39, las relaciones entre CTAs y warps/CTA se mantienen con respecto a 16k: CUDA lanza la mitad de warps por CTA que Triton y cuDNN SDPA, pero sigue lanzando el doble de CTAs. Torch SDPA lanza la mitad de CTAs que CUDA, pero la misma cantidad de warps por CTA. Todo esto se traduce en la mitad de warps totales que el resto de implementaciones.

Implementación	Registros/thread	SRAM/CTA (KiB)
CUDA	168	33
Triton	199	97
Torch SDPA	255	49
cuDNN SDPA	209	67

Tabla 4.40. Uso de registros y SRAM por bloque para una longitud de secuencia de 32768.

De acuerdo con la Tabla 4.40, el único cambio observado es que CUDA pasa a consumir 33 KiB de datos por CTA. La explicación más plausible es la siguiente:

Cuando hay una longitud de secuencia mayor, la probabilidad de encontrar un dato en la caché L2 disminuye. Que esta probabilidad sea menor influye directamente en la latencia media de las transferencias de memoria. Por tanto, para solapar mejor cómputo y transferencia, es necesario aumentar el número de buffers.

Implementación	Ocup. teórica (%)	Ocup. alcanzada (%)	Warps activos/SM
CUDA	25,00	22,07	10,59
Triton	16,67	16,71	8,02
Torch SDPA	16,67	14,56	6,99
cuDNN SDPA	16,67	16,66	8,00

Tabla 4.41. Ocupancia teórica, ocupancia alcanzada y número medio de warps activos por SM para una longitud de secuencia de 32768.

La ocupancia teórica de la Tabla 4.41 se mantiene en todos los kernels con respecto a 16k, Tabla 4.23, pero la ocupancia alcanzada sube aproximadamente 2,1 puntos en CUDA, una décima en Triton, incluso ligeramente por encima de la teórica, lo que deduzco es una imprecisión despreciable del profiler, y aproximadamente 2 puntos en Torch SDPA. La implementación CUDA sigue sin alcanzar el techo de ocupancia, pero esta mejora coincide con el aumento de rendimiento observado.

4.5.3. Cantidad de instrucciones ejecutadas

Implementación	Instrucciones ejecutadas
CUDA	1 451 810 816
Triton	770 936 832
Torch SDPA	724 157 440
cuDNN SDPA	571 705 344

Tabla 4.42. Número de instrucciones ejecutadas para una longitud de secuencia de 32768.

La cantidad de instrucciones ejecutadas deja de ser noticia en la Tabla 4.42: CUDA realiza aproximadamente 2,5 veces más instrucciones que cuDNN y el doble que Torch SDPA y Triton. Este test muestra que la relación

$$\text{más instrucciones} \Rightarrow \text{menor rendimiento} \tag{4.10}$$

no es necesariamente cierta si se consigue explotar suficiente paralelismo a nivel de instrucción. Antes podíamos afirmar que la implementación CUDA rendía por debajo del resto de implementaciones y que podía deberse, en parte, a la gran cantidad de instrucciones realizadas. Para esta longitud de secuencia, la implementación que más instrucciones realiza es la más rápida, invalidando esa afirmación.

Implementación	Cantidad de instrucciones	Sobre el total 32768 (%)	Sobre el total 16384 (%)
CUDA	33 882 112	2,33	2,33
Triton	8 388 608	1,09	1,09
Torch SDPA	8 396 800	1,16	1,16
cuDNN SDPA	4 202 496	0,74	0,73

Tabla 4.43. Número de instrucciones de tipo `SHFL` y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384.

La relación de instrucciones `SHFL` de la Tabla 4.43 se mantiene para todas las implementaciones con respecto a 16k.

Implementación	Cantidad de instrucciones	Sobre el total 32768 (%)	Sobre el total 16384 (%)
CUDA	100 689 920	6,94	6,85
Triton	89 264 128	11,58	11,58
Torch SDPA	67 228 672	9,28	9,29
cuDNN SDPA	80 883 712	14,15	14,14

Tabla 4.44. Número de instrucciones de carga/almacenamiento y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384.

La relación de instrucciones de carga y almacenamiento disminuye aproximadamente una décima en CUDA, mientras que en el resto de implementaciones se mantiene prácticamente constante, como se observa en la Tabla 4.44.

Implementación	Cantidad de instrucciones	Sobre el total 32768 (%)	Sobre el total 16384 (%)
CUDA	797 036 544	54,90	54,21
Triton	558 256 128	72,41	72,35
Torch SDPA	583 029 760	80,51	80,44
cuDNN SDPA	419 586 048	73,39	73,26

Tabla 4.45. Número de instrucciones de punto flotante y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384.

La relación de instrucciones de punto flotante presenta poca variación, como se observa en la Tabla 4.45.

Implementación	Cantidad de instrucciones	Sobre el total 32768 (%)	Sobre el total 16384 (%)
CUDA	134 217 728	9,24	9,13
Triton	134 217 728	17,41	17,38
Torch SDPA	134 217 728	18,53	18,52
cuDNN SDPA	134 217 728	23,48	23,42

Tabla 4.46. Número de instrucciones Tensor Core (`HMMA`) y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384.

El número de instrucciones Tensor Core sigue siendo el mismo entre las distintas implementaciones, como se observa en la Tabla 4.46.

Implementación	Cantidad de instrucciones	Sobre el total 32768 (%)	Sobre el total 16384 (%)
CUDA	0	0,00	0,00
Triton	35 651 584	4,62	4,62
Torch SDPA	37 744 640	5,21	5,21
cuDNN SDPA	35 651 584	6,24	6,22

Tabla 4.47. Número de instrucciones exponenciales (`MUFU.EX2`) y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384.

La proporción de instrucciones exponenciales se mantiene muy similar con respecto a 16k, como se observa en la Tabla 4.47.

Implementación	Cantidad de instrucciones	Sobre el total 32768 (%)	Sobre el total 16384 (%)
CUDA	418 977 792	28,86	28,51
Triton	46 542 848	6,04	6,08
Torch SDPA	37 014 528	5,11	5,15
cuDNN SDPA	35 233 792	6,16	6,26

Tabla 4.48. Número de instrucciones enteras y porcentaje sobre el total para una longitud de secuencia de 32768, junto con el porcentaje obtenido para 16384.

La proporción de instrucciones sobre enteros se mantiene muy similar con respecto a 16k, variando aproximadamente tres décimas en CUDA y una décima en cuDNN, como se observa en la Tabla 4.48.

No existen cambios apreciables en la proporción de instrucciones con respecto a 16k que justifiquen por sí solos la mejora en el tiempo de ejecución de CUDA. Hasta ahora, los únicos cambios observados son el aumento de la ocupancia alcanzada de CUDA y el incremento del número de buffers por CTA.

4.5.4. Memoria

Implementación	Lecturas DRAM (MiB)	Escrituras DRAM (MiB)	Total DRAM (MiB)
CUDA	113,77	9,68	123,45
Triton	72,07	9,67	81,74
Torch SDPA	71,68	9,96	81,64
cuDNN SDPA	72,04	9,63	81,67

Tabla 4.49. Volumen de datos leído y escrito en DRAM para una longitud de secuencia de 32768.

En términos brutos, el volumen de datos transferidos desde memoria principal aumenta. La versión CUDA realiza aproximadamente 40 MiB más de lecturas desde DRAM, mientras que el número de escrituras se mantiene prácticamente idéntico entre las implementaciones.

Implementación	Aumento lecturas DRAM (%)	Aumento escrituras DRAM (%)	Aumento total DRAM (%)
CUDA	320,28	80,60	280,67
Triton	259,99	113,47	232,82
Torch SDPA	326,92	145,93	291,75
cuDNN SDPA	259,48	113,05	232,53

Tabla 4.50. Aumento porcentual del volumen de datos leído y escrito en DRAM al pasar de una longitud de secuencia de 16384 a 32768.

Añadiendo más perspectiva, la versión CUDA es cierto que es la implementación que más transferencias de memoria realiza, como se observa en la Tabla 4.49, pero no es la que peor escala con respecto a 16k. La que peor escala es Torch SDPA, necesitando un 291 % más de transferencias de memoria, algo por encima de CUDA con un 280 %. Triton y cuDNN SDPA parecen empatar en el escalado.

Implementación	Local loads	Local stores
CUDA	0	0
Triton	0	0
Torch SDPA	1024	1024
cuDNN SDPA	0	0

Tabla 4.51. Accesos a memoria local para una longitud de secuencia de 32768.

Torch SDPA aumenta el número de *spills* con respecto a 16k, pasando de 512 a 1024 accesos de cada tipo. Este aumento es problemático y podría estar relacionado con el peor escalado de sus accesos a memoria en comparación con CUDA.

Implementación	Lecturas L2 (MiB)	Escrituras L2 (MiB)	Total L2 (MiB)	Hit rate L2 (%)
CUDA	8253,07	8,00	8261,07	98,62
Triton	4104,00	8,00	4112,00	98,25
Torch SDPA	4104,02	8,25	4112,27	98,26
cuDNN SDPA	4104,00	8,00	4112,00	98,25

Tabla 4.52. Volumen de datos leído y escrito en la caché L2 y tasa de aciertos para una longitud de secuencia de 32768.

Los accesos a memoria L2 siguen siendo el doble en CUDA y esta sigue conservando el *hit rate* más alto, como se observa en la Tabla 4.52.

Al igual que con la cantidad de instrucciones, la afirmación

$$\text{más transferencias de memoria} \Rightarrow \text{menor rendimiento} \tag{4.11}$$

tampoco se cumple necesariamente. No es solo que el kernel que realiza más instrucciones sea el que mejor rinde, sino que además es el kernel que más transferencias de memoria realiza. Esto pone de manifiesto que una mayor ocupancia y un mayor paralelismo pueden compensar tanto una mayor cantidad de instrucciones como un mayor tráfico de memoria.

4.5.5. Throughput, sincronizaciones, warps activos

Implementación	Ciclos con una instrucción emitida 32768 (%)	Ciclos con una instrucción emitida 16384 (%)
CUDA	29,04	28,42
Triton	15,24	15,17
Torch SDPA	15,21	14,90
cuDNN SDPA	12,19	12,20

Tabla 4.53. Porcentaje de ciclos en los que se emitió una instrucción para longitudes de secuencia de 32768 y 16384. En NCU, esta métrica corresponde a `smsp__issue_active.avg.pct_of_peak_sustained_active`.

De acuerdo con la Tabla 4.53, el kernel CUDA aumenta el número de ciclos en los que emite una instrucción en 0,6 puntos, Triton en 0,07 y Torch SDPA en 0,3 puntos. Los aumentos en CUDA y Torch SDPA son consistentes con el aumento de la ocupancia alcanzada.

Implementación	Barrier (%)	Math Pipe Throttle (%)	Wait (%)	Short Scoreboard (%)
CUDA	8,53	57,66	12,48	2,99
Triton	5,89	60,67	15,26	5,19
Torch SDPA	10,00	64,24	11,58	0,34
cuDNN SDPA	3,01	76,53	8,80	0,61

Tabla 4.54. Porcentaje de warp-ciclos activos asociados a diferentes causas de *stall* para una longitud de secuencia de 32768.

Si miramos el porcentaje de ciclos de *stall* de la Tabla 4.54, vemos que los principales cambios con respecto a 16k, Tabla 4.37, son los siguientes: Torch SDPA aumenta aproximadamente un 2 % los ciclos que pasa en *stall* como consecuencia de barreras, mientras que CUDA aumenta aproximadamente un 1 %. En CUDA, los *stalls* de tipo *Wait* disminuyen un 1,4 %, mientras que los asociados a *Math Pipe Throttle* aumentan un 1,6 %. En Torch SDPA, los *stalls* asociados a *Math Pipe Throttle* aumentan aproximadamente un 0,5 %.

4.5.6. Conclusión y análisis para longitud de secuencia 32768

El análisis para esta longitud de secuencia es el siguiente:

1- CUDA ha aumentado en uno el número de buffers por CTA (8 KiB). La explicación más plausible es que los accesos a memoria hayan aumentado su latencia media como consecuencia del aumento de la longitud de secuencia y de una mayor presión sobre la memoria caché.

2- Torch SDPA y CUDA han aumentado su ocupancia alcanzada. Aunque ninguno ha alcanzado su ocupancia teórica, CUDA ha multiplicado su número de ciclos aproximadamente $3,4\times$, frente a casi $4\times$ en el resto de kernels. Esto ha permitido a CUDA situarse por delante del resto de versiones.

3- El kernel CUDA invalida, para esta longitud de secuencia, las afirmaciones

$$\text{más instrucciones} \Rightarrow \text{menor rendimiento} \tag{4.12}$$

y

$$\text{más transferencias de memoria} \Rightarrow \text{menor rendimiento.} \tag{4.13}$$

4- Los kernels CUDA y Torch SDPA disminuyen el porcentaje de *stalls* debidos a dependencias de datos y aumentan los *stalls* debidos a barreras y a *Math Pipe Throttle*.

4.6. Longitud de secuencia 65536

4.6.1. Ciclos, frecuencia y tiempo estimado

Implementación	Frecuencia SM (GHz)	Ciclos	Tiempo (μs)
CUDA	1,3927	62 398 567	44760,224
Triton	1,3939	69 489 910	49824,032
Torch SDPA	1,3933	64 334 031	46135,552
cuDNN SDPA	1,3930	64 045 490	45939,616

Tabla 4.55. Frecuencia del SM, ciclos transcurridos y tiempo de ejecución para una longitud de secuencia de 65536.

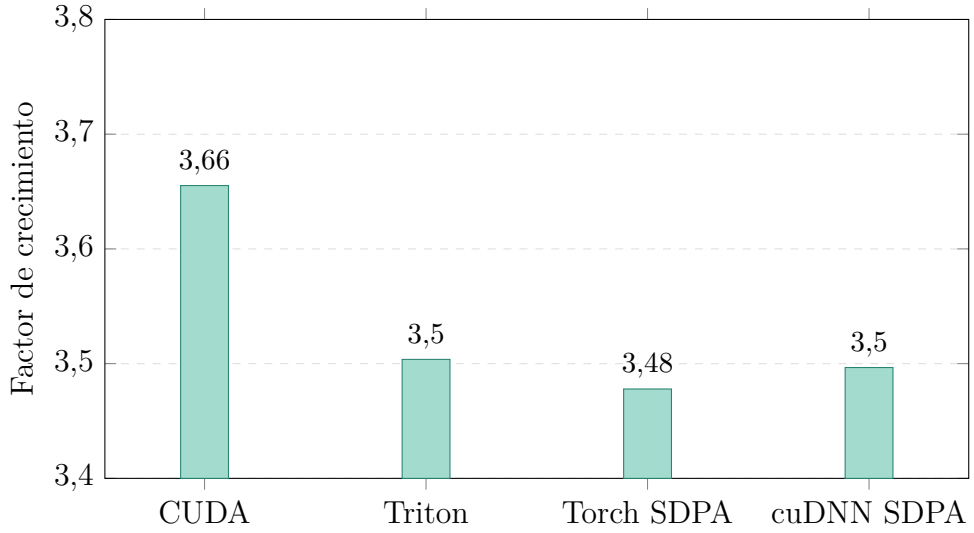


Figura 4.6. Factor de crecimiento del número de ciclos al aumentar la longitud de secuencia de 32768 a 65536.

De acuerdo con la Tabla 4.55, de nuevo, la frecuencia es pareja, por lo que las mediciones se pueden considerar válidas. CUDA, Torch SDPA y cuDNN SDPA quedan prácticamente empatados, y Triton queda claramente por debajo, necesitando aproximadamente 5 ms más que CUDA. CUDA tiene una ligera ventaja pero, dado que no hemos ejecutado múltiples veces el kernel, no podemos afirmar que en múltiples ejecuciones la ventaja se mantenga.

Comparado con 32K, la diferencia entre las versiones CUDA y cuDNN/Torch SDPA sigue siendo aproximadamente 1 ms: es cierto que el porcentaje es más pequeño, pero es un porcentaje más pequeño sobre una cantidad más grande.

Si miramos el escalado de la Figura 4.6, vemos que Triton, Torch SDPA y cuDNN SDPA reducen el factor cercano a 4× observado anteriormente y escalan aproximadamente $3,5\times$. CUDA escala $3,66\times$, por lo que en este salto escala peor que el resto de

implementaciones. De nuevo, al duplicar la longitud de secuencia, el número de ciclos aumenta entre aproximadamente $3,5\times$ y $4\times$, evidenciando un coste computacional aproximadamente cuadrático con respecto a la longitud de secuencia.

4.6.2. Throughput pipelines

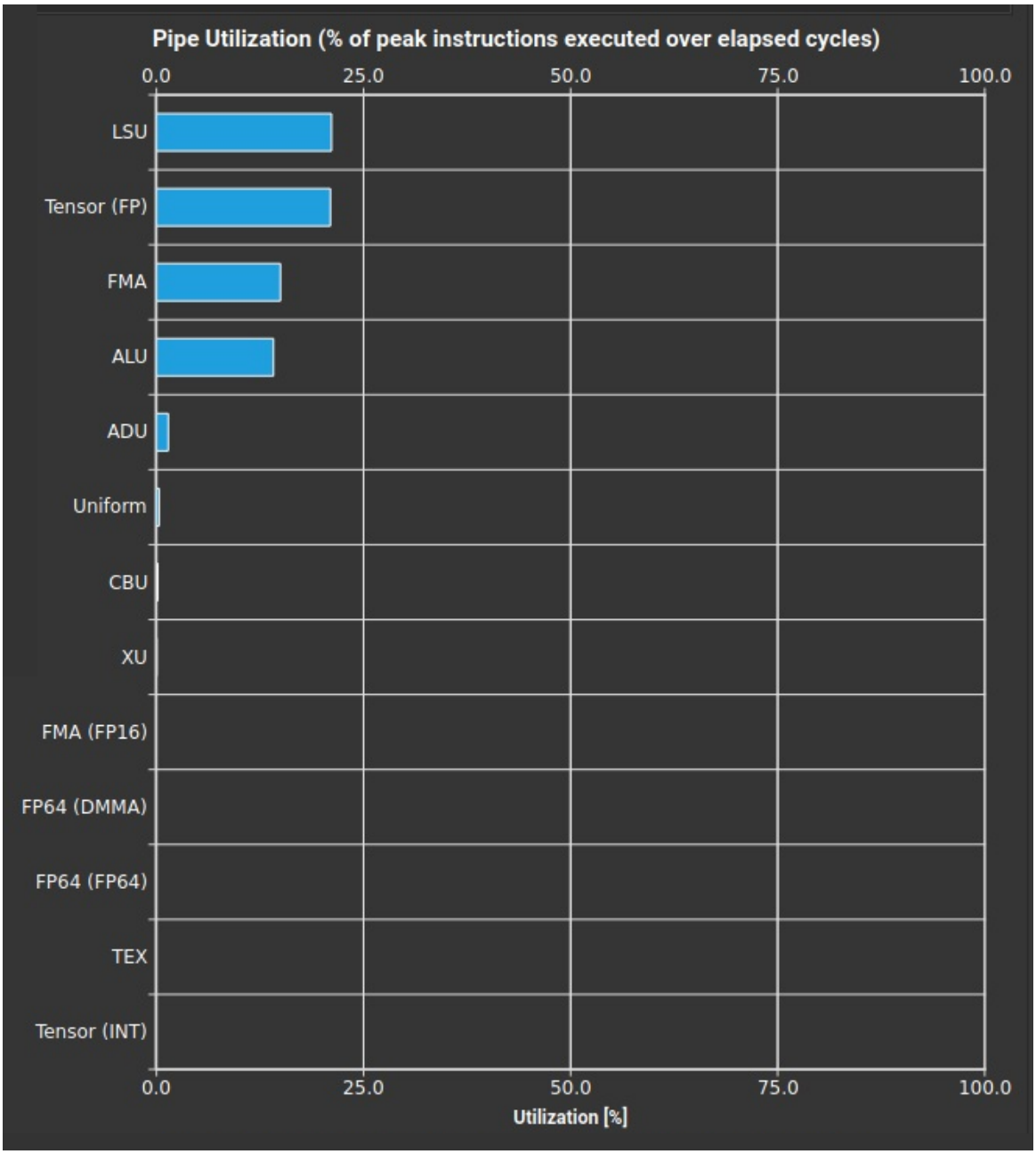


Figura 4.7. Throughput de los pipelines de ejecución para CUDA con 64K.

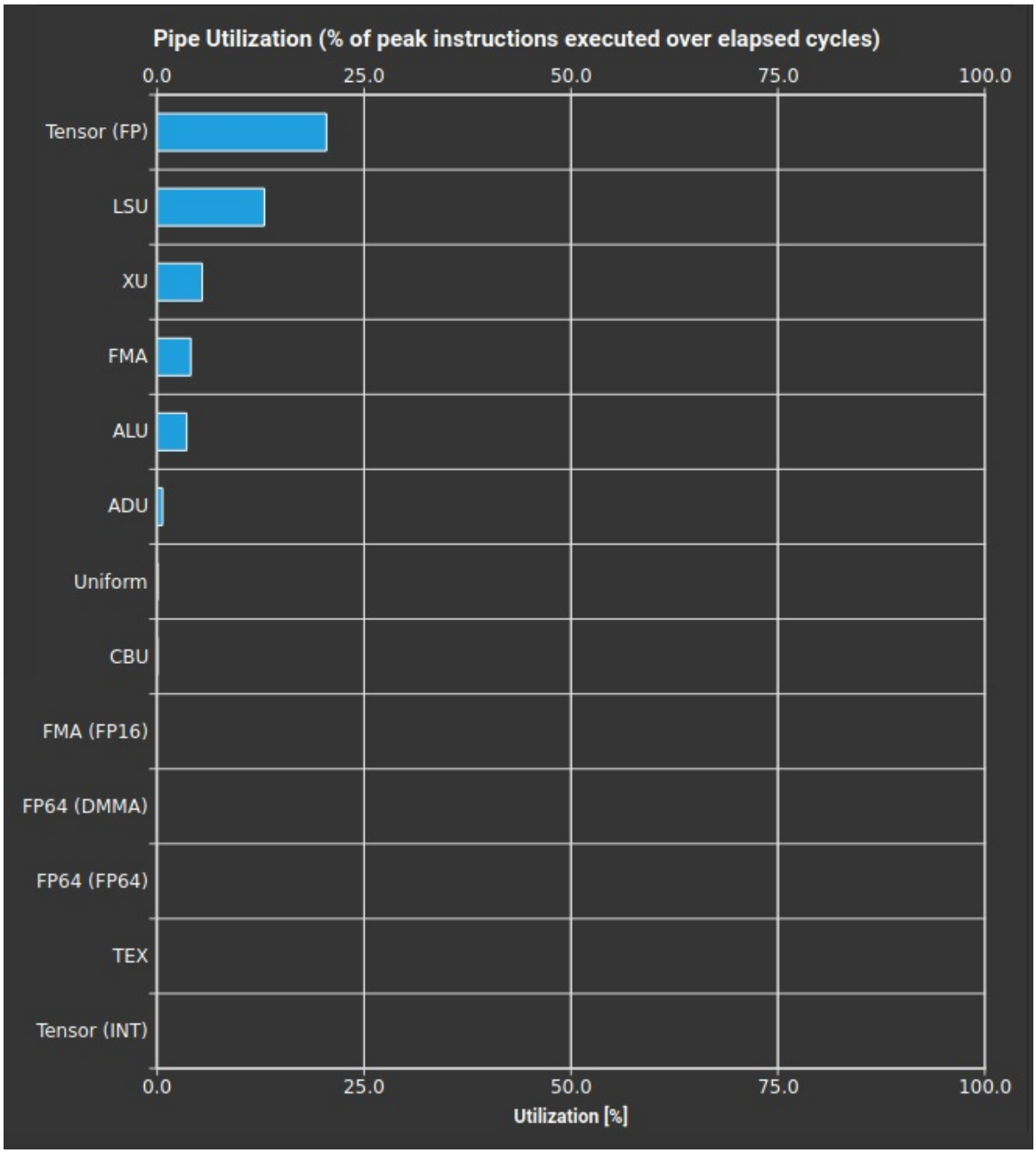


Figura 4.8. Throughput de los pipelines de ejecución para cuDNN con 64K.

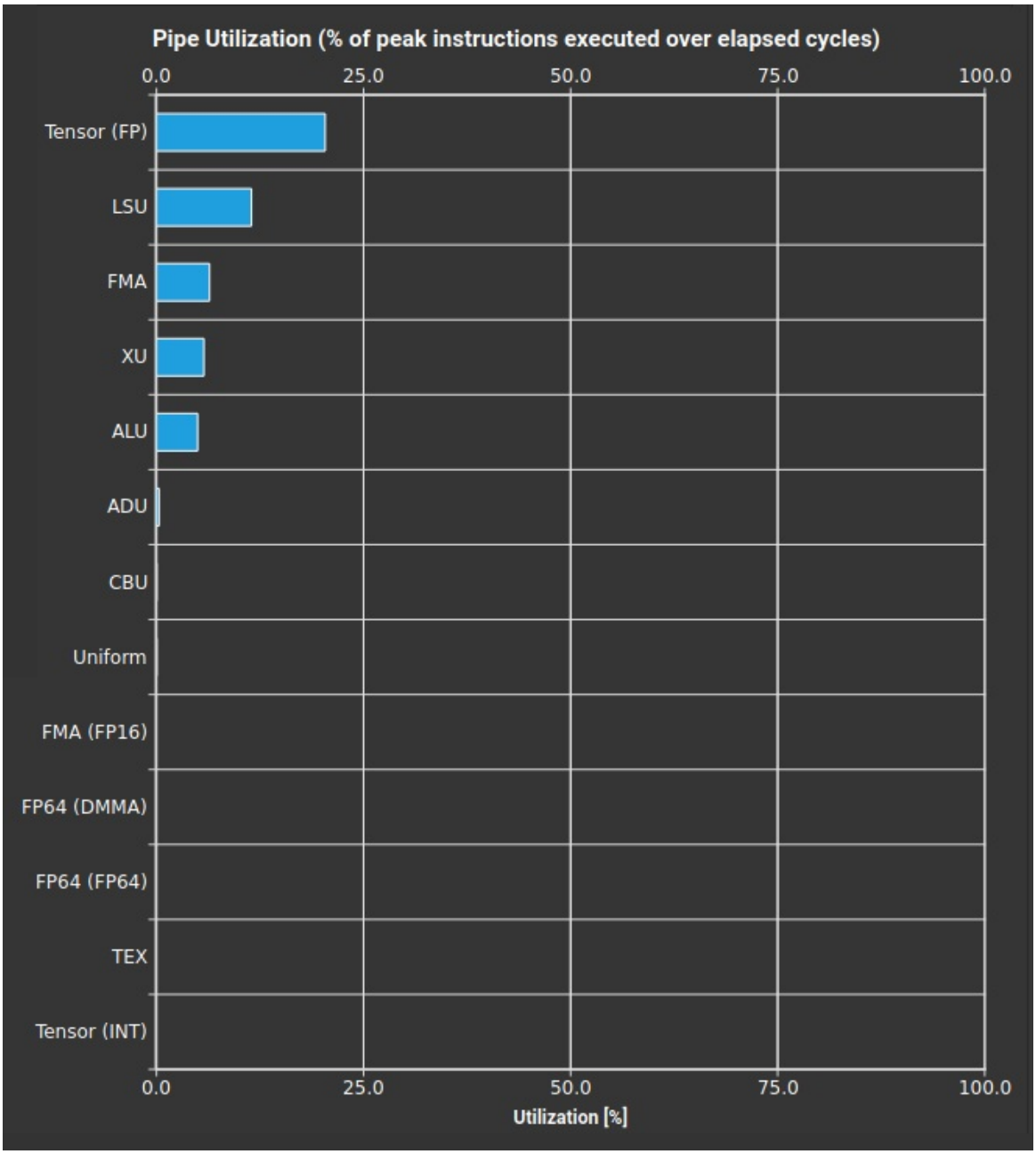


Figura 4.9. Throughput de los pipelines de ejecución para SDPA con 64K.

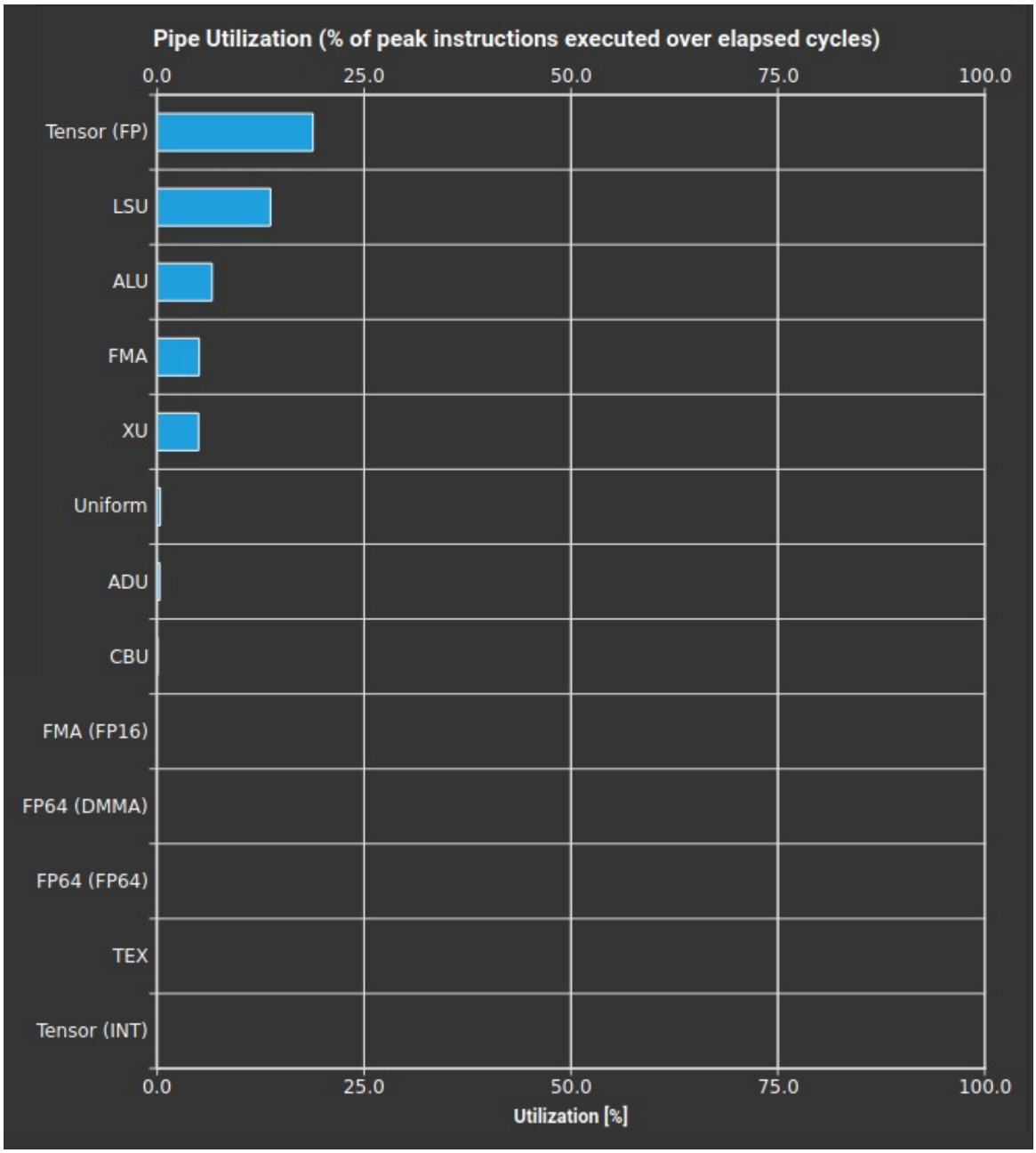


Figura 4.10. Throughput de los pipelines de ejecución para Triton con 64K.

Las Figuras 4.7, 4.8, 4.9 y 4.10 muestran el throughput de los distintos pipelines de ejecución para una longitud de secuencia de 64K, comparando las implementaciones CUDA, cuDNN, SDPA y Triton, respectivamente. Podemos apreciar que ninguno de los kernels consigue superar el 25 % de throughput, a pesar de presentar una mayor ocupación y procesar una mayor cantidad de datos. Sin embargo, en el caso del kernel CUDA, observamos un mayor throughput en los pipelines LSU, Tensor, ALU y FMA.

4.6.3. Ocupancia

Implementación	CTAs	Warps/CTA	Warps totales
CUDA	1 024	4	4 096
Triton	512	8	4 096
Torch SDPA	512	4	2 048
cuDNN SDPA	512	8	4 096

Tabla 4.56. Tamaño del grid y número de warps lanzados para una longitud de secuencia de 65536.

La relación entre CTAs y warps por CTA se mantiene con respecto a 32k, como se observa en la Tabla 4.56.

Implementación	Registros/thread	SRAM/CTA (KiB)
CUDA	168	33
Triton	199	97
Torch SDPA	255	49
cuDNN SDPA	209	67

Tabla 4.57. Uso de registros y SRAM por bloque para una longitud de secuencia de 65536.

El número de registros por *thread* y la cantidad de SRAM consumida se mantienen idénticos con respecto a 32k, como se observa en la Tabla 4.57.

Implementación	Ocup. teórica (%)	Ocup. alcanzada (%)	Warps activos/SM
CUDA	25,00	23,49	11,28
Triton	16,67	16,69	8,01
Torch SDPA	16,67	15,69	7,53
cuDNN SDPA	16,67	16,66	8,00

Tabla 4.58. Ocupancia teórica, ocupancia alcanzada y número medio de warps activos por SM para una longitud de secuencia de 65536.

La ocupancia aumenta aproximadamente un punto y medio en CUDA y sigue quedando por debajo de la ocupancia teórica, por lo que todavía existe margen para aumentar la ocupancia. Torch SDPA también aumenta aproximadamente un punto su ocupancia, como se observa en la Tabla 4.58.

4.6.4. Cantidad de instrucciones ejecutadas

Implementación	Instrucciones ejecutadas
CUDA	5 814 046 720
Triton	3 081 183 232
Torch SDPA	2 895 349 760
cuDNN SDPA	2 284 261 376

Tabla 4.59. Número de instrucciones ejecutadas para una longitud de secuencia de 65536.

Como se observa en la Tabla 4.59, CUDA ejecuta aproximadamente 1,9× más instrucciones que Triton, 2,0× más que Torch SDPA y 2,5× más que cuDNN SDPA.

Implementación	Cantidad de instrucciones	Sobre el total 65536 (%)	Sobre el total 32768 (%)
CUDA	134 873 088	2,32	2,33
Triton	33 554 432	1,09	1,09
Torch SDPA	33 570 816	1,16	1,16
cuDNN SDPA	16 793 600	0,74	0,74

Tabla 4.60. Número de instrucciones de tipo `SHFL` y porcentaje sobre el total para una longitud de secuencia de 65536.

El porcentaje de instrucciones de tipo `SHFL` prácticamente no varía con respecto a 32k, como se observa en la Tabla 4.60.

Implementación	Cantidad de instrucciones	Sobre el total 65536 (%)	Sobre el total 32768 (%)
CUDA	402 706 432	6,93	6,94
Triton	356 786 176	11,58	11,58
Torch SDPA	268 675 072	9,28	9,28
cuDNN SDPA	323 248 128	14,15	14,15

Tabla 4.61. Número de instrucciones de carga/almacenamiento y porcentaje sobre el total para una longitud de secuencia de 65536.

El porcentaje de instrucciones de carga y almacenamiento prácticamente no varía con respecto a 32k, como se observa en la Tabla 4.61.

Implementación	Cantidad de instrucciones	Sobre el total 65536 (%)	Sobre el total 32768 (%)
CUDA	3 187 908 608	54,83	54,90
Triton	2 232 197 120	72,45	72,41
Torch SDPA	2 332 076 032	80,55	80,51
cuDNN SDPA	1 678 032 896	73,46	73,39

Tabla 4.62. Número de instrucciones de punto flotante y porcentaje sobre el total para una longitud de secuencia de 65536.

El porcentaje de instrucciones de punto flotante prácticamente no varía con respecto a 32k, como se observa en la Tabla 4.62.

Implementación	Cantidad de instrucciones	Sobre el total 65536 (%)	Sobre el total 32768 (%)
CUDA	536 870 912	9,23	9,24
Triton	536 870 912	17,42	17,41
Torch SDPA	536 870 912	18,54	18,53
cuDNN SDPA	536 870 912	23,50	23,48

Tabla 4.63. Número de instrucciones Tensor Core (`HMMA`) y porcentaje sobre el total para una longitud de secuencia de 65536.

El porcentaje de instrucciones matriciales prácticamente no varía con respecto a 32k, como se observa en la Tabla 4.63.

Implementación	Cantidad de instrucciones	Sobre el total 65536 (%)	Sobre el total 32768 (%)
CUDA	0	0,00	0,00
Triton	142 606 336	4,63	4,62
Torch SDPA	150 986 752	5,21	5,21
cuDNN SDPA	142 606 336	6,24	6,24

Tabla 4.64. Número de instrucciones exponenciales (`MUFU.EX2`) y porcentaje sobre el total para una longitud de secuencia de 65536.

El porcentaje de instrucciones exponenciales prácticamente no varía con respecto a 32k, como se observa en la Tabla 4.64.

Implementación	Cantidad de instrucciones	Sobre el total 65536 (%)	Sobre el total 32768 (%)
CUDA	1 717 129 216	29,53	28,86
Triton	185 360 384	6,02	6,04
Torch SDPA	147 429 376	5,09	5,11
cuDNN SDPA	139 673 600	6,11	6,16

Tabla 4.65. Número de instrucciones enteras y porcentaje sobre el total para una longitud de secuencia de 65536.

El porcentaje de instrucciones enteras prácticamente no varía con respecto a 32k, como se observa en la Tabla 4.65.

En cuanto a la cuenta de instrucciones, estamos en una situación similar a la observada para una longitud de secuencia de 32k. Los porcentajes relativos de los distintos tipos de instrucciones no varían de forma significativa.

4.6.5. Memoria

Implementación	Lecturas DRAM (MiB)	Escrituras DRAM (MiB)	Total DRAM (MiB)
CUDA	439,10	17,55	456,65
Triton	241,56	17,61	259,18
Torch SDPA	249,89	17,99	267,88
cuDNN SDPA	240,60	17,49	258,10

Tabla 4.66. Volumen de datos leído y escrito en DRAM para una longitud de secuencia de 65536.

La versión CUDA transfiere aproximadamente 200 MiB más de memoria que las versiones de Triton, Torch SDPA y cuDNN SDPA, como se observa en la Tabla 4.66.

Implementación	8k → 16k (%)	16k → 32k (%)	32k → 64k (%)
CUDA	302,86	280,67	269,91
Triton	209,71	232,82	217,08
Torch SDPA	–	291,75	228,12
cuDNN SDPA	208,54	232,53	216,03

Tabla 4.67. Incremento porcentual del volumen total transferido a DRAM al duplicar la longitud de secuencia.

Si miramos la Tabla 4.67, vemos que, si bien Triton y cuDNN escalan aproximadamente un 200 % al doblar la longitud de secuencia, con un pico al pasar de 16k a 32k, la versión CUDA empieza realizando un 300 % de tráfico adicional de 8k a 16k, pero este incremento disminuye posteriormente hasta aproximadamente un 280 % de 16k a 32k y un 270 % de 32k a 64k. Esto puede ser una buena señal si, al seguir aumentando la longitud de secuencia, el crecimiento de los accesos a memoria continúa reduciéndose.

Implementación	Local loads	Local stores
CUDA	0	0
Triton	0	0
Torch SDPA	2048	2048
cuDNN SDPA	0	0

Tabla 4.68. Accesos a memoria local asociados a register spilling para una longitud de secuencia de 65536.

En la Tabla 4.51 habíamos dicho que el peor escalado de Torch en accesos a memoria DRAM podía deberse a los register spills. Sin embargo, aun con el doble de local loads y local stores, como se describe en la Tabla 4.68, el paso de 32k a 64k vuelve a un 200 % en comparación con el 291 % observado de 16k a 32k, invalidando esa hipótesis.

Implementación	Lecturas L2 (MiB)	Escrituras L2 (MiB)	Total L2 (MiB)	Hit rate L2 (%)
CUDA	32996,63	16,00	33012,63	98,67
Triton	16400,00	16,00	16416,00	98,45
Torch SDPA	16400,03	16,50	16416,53	98,47
cuDNN SDPA	16400,00	16,00	16416,00	98,53

Tabla 4.69. Volumen de datos leído y escrito en la caché L2 y tasa de aciertos para una longitud de secuencia de 65536.

La proporción de lecturas de L2 y el *hit rate* se mantienen prácticamente idénticos

con respecto a 32k, como se observa en la Tabla 4.69.

4.6.6. Throughput, sincronizaciones, warps activos

Implementación	Ciclos con una instrucción emitida 65536 (%)	Ciclos con una instrucción emitida 32768 (%)
CUDA	29,60	29,04
Triton	15,19	15,24
Torch SDPA	15,38	15,21
cuDNN SDPA	12,19	12,19

Tabla 4.70. Porcentaje de ciclos en los que se emitió una instrucción para una longitud de secuencia de 65536, comparado con 32768. En NCU, esta métrica corresponde a `smsp__issue_active.avg.pct_of_peak_sustained_active`.

De acuerdo con la información expresada en la Tabla 4.70, CUDA aumenta aproximadamente 0,6 puntos porcentuales el porcentaje de ciclos en los que consigue emitir una instrucción, mientras que Torch SDPA aumenta aproximadamente 0,2 puntos. Este aumento coincide, de nuevo, con la mejora de ocupancia observada al aumentar la longitud de secuencia.

Implementación	Barrier (%)	Math Pipe Throttle (%)	Wait (%)	Short Scoreboard (%)
CUDA	8,96	58,15	12,02	2,75
Triton	5,86	60,67	15,27	5,20
Torch SDPA	10,91	64,43	10,89	0,31
cuDNN SDPA	3,02	76,58	8,81	0,61

Tabla 4.71. Porcentaje de warp-ciclos activos asociados a diferentes causas de *stall* para una longitud de secuencia de 65536.

Si comparamos la Tabla 4.71 con la Tabla 4.54, podemos ver que CUDA aumenta aproximadamente 0,4 puntos el porcentaje de *stalls* debido a barreras y 0,5 puntos el porcentaje debido a *Math Pipe Throttle*, mientras que disminuye aproximadamente 0,5 puntos el porcentaje asociado a dependencias de datos y 0,2 puntos el asociado a *Short Scoreboard*. Torch SDPA, por su parte, aumenta aproximadamente 0,9 puntos el porcentaje de *stalls* debido a barreras y disminuye aproximadamente 0,7 puntos el asociado a dependencias de datos.

4.6.7. Conclusión y análisis de los kernels

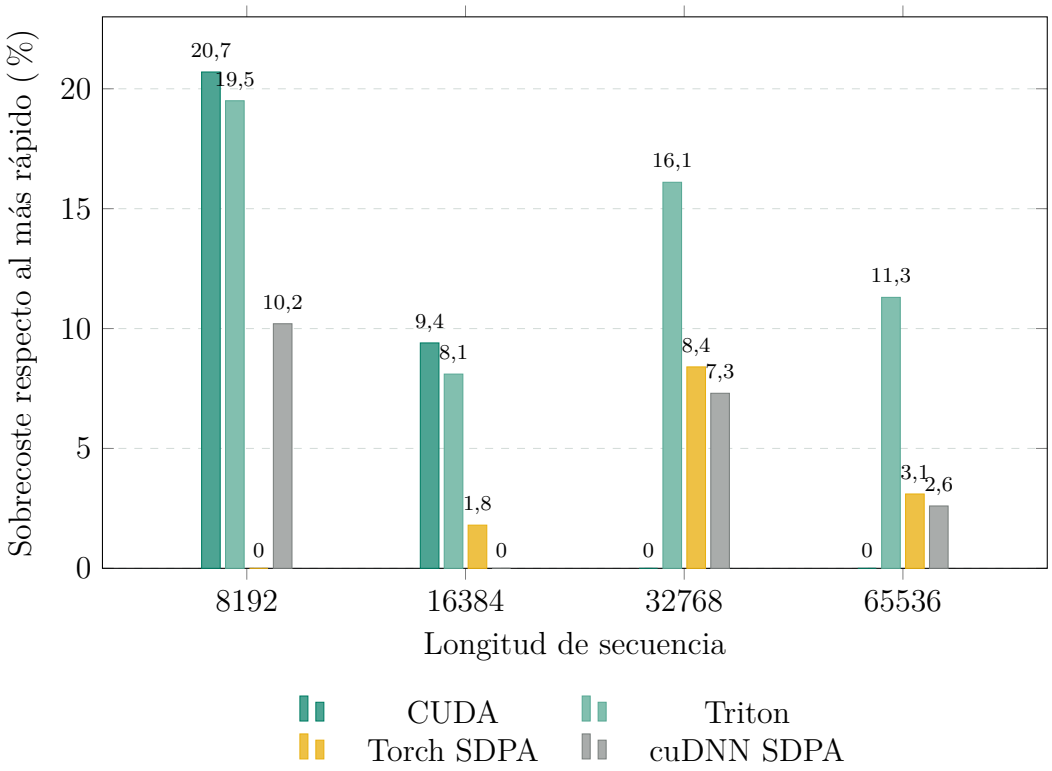


Figura 4.11. Sobrecoste temporal de cada implementación con respecto a la implementación más rápida para cada longitud de secuencia. Un valor de 0 % indica la implementación con menor tiempo de ejecución para dicha longitud.

De acuerdo con la Figura 4.11, para una longitud de secuencia de 8192, las versiones CUDA y Triton tardan aproximadamente un 20 % más que Torch SDPA, mientras que cuDNN tarda alrededor de un 10 % más. Las versiones CUDA y Triton quedan prácticamente empatadas.

Si aumentamos la longitud de secuencia a 16k, se reduce la brecha y cuDNN sobrepasa a Torch SDPA como la versión más rápida. Torch SDPA queda un $1,8\,\%$ por debajo de cuDNN, Triton un $8,1\,\%$ y CUDA un $9,4\,\%$.

Para una longitud de secuencia de 32k, la versión CUDA pasa a ser la más rápida. cuDNN y Torch SDPA quedan un $7,3\,\%$ y un $8,4\,\%$ por debajo de CUDA, respectivamente, mientras que Triton queda un $16,1\,\%$ por debajo.

Para una longitud de secuencia de 64k, la versión CUDA sigue siendo la más rápida. Triton queda un 11 % por debajo, mientras que Torch SDPA y cuDNN quedan prácticamente empatados alrededor de un 3 % por debajo de CUDA.

Para longitudes de secuencia mayores, es posible que Optuna decida cambiar el número de buffers por CTA, así como el número de warps. Con ello podría reducir los accesos extra que realiza CUDA con respecto al resto de versiones o aumentar el paralelismo del kernel. Además, si al aumentar la longitud de secuencia continúa la tendencia observada, el crecimiento de los accesos a memoria principal podría seguir

reduciéndose desde el 270 % observado entre 32k y 64k.

También podría ocurrir que CUDA y Torch SDPA siguiesen aumentando su ocupancia alcanzada y acercándose a su ocupancia teórica, consiguiendo así una mayor capacidad para ocultar latencias mediante un mayor paralelismo a nivel de warps.

Los kernels de Torch SDPA presentan accesos a memoria local asociados a *register spilling*, por lo que sería conveniente estudiar si estos *spills* pueden reducirse o evitarse.

Capítulo 5

Conclusiones

En primer lugar, se han cumplido los objetivos propuestos: se ha implementado el algoritmo FlashAttention forward tanto en CUDA (O1) como en Triton (O3), se han justificado las decisiones ligadas a la implementación CUDA (O2) y, por último, se han comparado ambas implementaciones con las funciones de referencia de Torch y NVIDIA cuDNN (O4).

Longitud	**CUDA**	**Triton**	**Torch SDPA**	**cuDNN SDPA**
8192	0,912	0,903	**0,755**	0,832
16384	3,609	3,565	3,360	**3,299**
32768	**12,247**	14,222	13,270	13,140
65536	**44,760**	49,824	46,136	45,940

Tabla 5.1. Comparación de los tiempos de ejecución (ms) de las distintas implementaciones en función de la longitud de secuencia. Se destaca en negrita el menor tiempo para cada longitud.

Los resultados obtenidos se especifican en la Tabla 5.1. En 8k Torch es la implementación más rápida, en 16k, cuDNN, y en 32k y 64k nuestra implementación CUDA.

Puede parecer que la implementación de Triton ha quedado en último lugar en dos de las cuatro longitudes de secuencia analizadas y que, por tanto, no merece la pena utilizarla. No es cierto; al contrario. La experiencia durante este TFG me ha demostrado que escribir la implementación Triton requirió considerablemente menos tiempo de desarrollo durante este proyecto que la implementación CUDA.

La primera vez que se trabaja con instrucciones como `ldmatrix`, técnicas de *swizzling* o con el reparto de los datos de la matriz A de `mma.sync` entre los diferentes *warps*, el proceso resulta costoso. Como ocurre con cualquier tecnología, conforme se adquiere experiencia resulta más sencillo implementar nuevas optimizaciones. Sin embargo, la

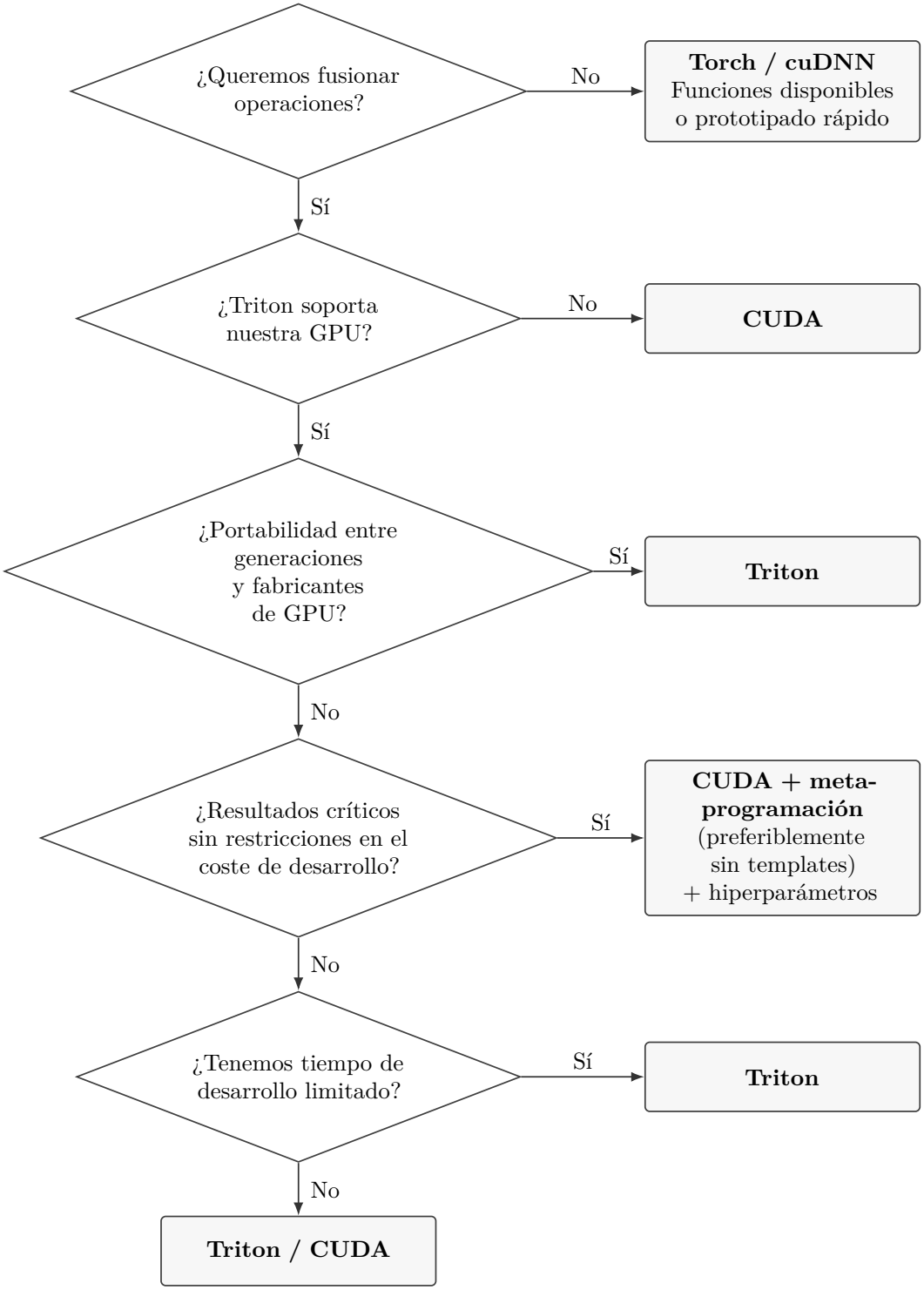


Figura 5.1. Diagrama de decisión para la elección entre Torch, cuDNN, Triton y CUDA en función de los requisitos del proyecto.

curva de aprendizaje inicial, especialmente en CUDA moderno, es elevada.

A partir de la experiencia adquirida durante el desarrollo, si tuviese que desarrollar una biblioteca seguiría una estrategia similar: implementaría primero los kernels en Triton, de forma que se dispusiese rápidamente de una versión funcional y con un rendimiento competitivo, y posteriormente trasladaría a CUDA aquellas partes más críticas en las que el rendimiento adicional justificase el mayor coste de desarrollo.

El ciclo de optimización con Optuna también presenta un problema de escalabilidad. Actualmente, un proceso de Python selecciona los parámetros, reconfigura el proyecto con CMake y recompila antes de evaluar cada configuración. Si aumenta el tamaño de la biblioteca o el número de parámetros a explorar, el tiempo total de búsqueda aumenta necesariamente, porque cada configuración requiere una nueva compilación.

NVRTC permite evitar recompilar todo el proyecto y compilar únicamente los kernels necesarios en tiempo de ejecución. Combinándolo con CUPTI para recoger directamente las métricas de rendimiento, se puede construir un ciclo de autotuning mucho más rápido e integrado.

El autotuner de Triton también escala mal cuando aumenta el número de configuraciones. Si se prueban todas las combinaciones posibles, el número de ejecuciones crece como un producto cartesiano. Con una sola GPU, una búsqueda guiada como la realizada con Optuna resulta más eficiente que recorrer exhaustivamente todo el espacio de configuraciones.

El reparto de tareas por *warp* y optimizaciones como el número de buffers, su tamaño y forma o los factores de *unroll* deben expresarse como hiperparámetros. Cada decisión que tomamos en CUDA tiene un coste, y ese coste acaba reflejándose en el tiempo de ejecución del kernel. Podemos medir el coste global una vez ejecutado, pero no resulta sencillo asignar de antemano un coste concreto a cada decisión.

Por ejemplo, para una longitud de secuencia pequeña probablemente sea más barato paralelizar únicamente una dimensión. Sin embargo, para una longitud de secuencia de 1024, ¿sigue compensando? ¿A partir de qué tamaño empieza a compensar otra distribución? ¿Cuántos *warps* deben participar? No disponemos de una función de coste sencilla que permita responder directamente a estas preguntas.

Por ello, un kernel no debe entenderse únicamente como el conjunto de operaciones que debe ejecutar, sino principalmente como la ruta que siguen los datos y las múltiples decisiones locales de almacenamiento, movimiento y cómputo que se toman durante esa ruta. La optimización de estos hiperparámetros permite minimizar el coste global medido del kernel y constituye una forma más fiable de encontrar una buena configuración que intentar determinar manualmente el coste aislado de cada decisión.

Anexos

Anexo A

Correspondencia entre magnitudes y métricas de NVIDIA Nsight Compute

Las tablas del Capítulo 4 presentan las métricas utilizando nombres descriptivos para facilitar su lectura. Este anexo recoge los identificadores de NVIDIA Nsight Compute (NCU) utilizados para obtener dichas magnitudes y las tablas de la memoria en las que aparecen. Algunas magnitudes se obtienen directamente de una métrica de NCU, mientras que otras se calculan a partir de una o varias métricas.

A.1. Tiempo y configuración de lanzamiento

Magnitud	Métrica NCU	Obtención	Tablas del Capítulo 4
Frecuencia del SM	`gpc__cycles_elapsed.avg.per_second`	Conversión a GHz	Tablas 4.1, 4.20, 4.38 y 4.55
Ciclos transcurridos	`gpc__cycles_elapsed.max`	Directa	Tablas 4.1, 4.20, 4.38 y 4.55
Tiempo de ejecución	`gpu__time_duration.sum`	Conversión a μs	Tablas 4.1, 4.20, 4.38 y 4.55
Número de CTAs	`launch__grid_size`	Directa	Tablas 4.3, 4.21, 4.39 y 4.56
Warps por CTA	`launch__block_size`	Derivada	Tablas 4.3, 4.21, 4.39 y 4.56
Warps totales	`launch__grid_size`, `launch__block_size`	Derivada	Tablas 4.3, 4.21, 4.39 y 4.56
Registros por thread	`launch__registers_per_thread`	Directa	Tablas 4.4, 4.22, 4.40 y 4.57
Memoria compartida por CTA	`launch__shared_mem_per_block`	Conversión a KiB	Tablas 4.4, 4.22, 4.40 y 4.57

Tabla A.1. Correspondencia entre las magnitudes de tiempo, lanzamiento y recursos y las métricas de NCU.

Las dos magnitudes marcadas como derivadas se calculan a partir del tamaño del bloque: el número de warps por CTA se obtiene dividiendo el número de threads por CTA entre 32, y el número total de warps se obtiene multiplicando este valor por el número de CTAs del grid.

A.2. Ocupancia

Magnitud	Métrica NCU	Tablas del Capítulo 4
Ocupancia teórica	`sm__maximum_warps_per_active_cycle_pct`	Tablas 4.5, 4.23, 4.41 y 4.58
Ocupancia alcanzada	`sm__warps_active.avg.pct_of_peak_sustained_active`	Tablas 4.5, 4.23, 4.41 y 4.58
Warps activos por SM	`sm__warps_active.avg.per_cycle_active`	Tablas 4.5, 4.23, 4.41 y 4.58
Warps activos por SMSP	`smsp__warps_active.avg.per_cycle_active`	Utilizada en la derivación de las Tablas 4.19, 4.37, 4.54 y 4.71

Tabla A.2. Correspondencia entre las magnitudes de ocupancia y las métricas de NCU.

A.3. Instrucciones ejecutadas

Magnitud	Métrica NCU	Instancia	Tablas del Capítulo 4
Instrucciones totales	`smsp__inst_executed.sum`	–	Tablas 4.6, 4.24, 4.42 y 4.59
SHFL	`sass__inst_executed_per_opcode`	`SHFL`	Tablas 4.7, 4.25, 4.43 y 4.60
Carga/ almacenamiento	`sass__inst_executed_per_opcode_category`	`Load/ Store`	Tablas 4.8, 4.26, 4.44 y 4.61
Punto flotante	`sass__inst_executed_per_opcode_category`	`Floating Point`	Tablas 4.9, 4.27, 4.45 y 4.62
Tensor Core	`sass__inst_executed_per_opcode`	`HMMA`	Tablas 4.10, 4.28, 4.46 y 4.63
Exponencial hardware	`sass__inst_executed_per_opcode`	`MUFU.EX2`	Tablas 4.11, 4.29, 4.47 y 4.64
Enteros	`sass__inst_executed_per_opcode_category`	`Integer`	Tablas 4.12, 4.30, 4.48 y 4.65

Tabla A.3. Correspondencia entre las categorías de instrucciones estudiadas y las métricas SASS de NCU.

A.4. Register spilling

Magnitud	Métrica NCU	Tablas del Capítulo 4
Spills a memoria local	`sass__inst_executed_register_spilling_mem_local`	Tablas 4.31 y 4.51
Lecturas por spilling en memoria local	`sass__inst_executed_register_spilling_mem_local_op_read`	Tablas 4.31 y 4.51
Escrituras por spilling en memoria local	`sass__inst_executed_register_spilling_mem_local_op_write`	Tablas 4.31 y 4.51

Tabla A.4. Métricas utilizadas para identificar accesos asociados al register spilling.

A.5. DRAM y caché L2

Magnitud	Métrica NCU	Obtención	Tablas del Capítulo 4
Lecturas DRAM	`dram__bytes_read.sum`	Bytes a MiB	Tablas 4.13, 4.32, 4.49 y 4.66; derivada en 4.50 y 4.67
Escrituras DRAM	`dram__bytes_write.sum`	Bytes a MiB	Tablas 4.13, 4.32, 4.49 y 4.66; derivada en 4.50 y 4.67
Lecturas L2	`lts__t_sectors_srcunit_tex_op_read.sum`	Sectores ×32 B	Tablas 4.14, 4.33, 4.52 y 4.69
Escrituras L2	`lts__t_sectors_srcunit_tex_op_write.sum`	Sectores ×32 B	Tablas 4.14, 4.33, 4.52 y 4.69
Hit rate L2	`lts__t_sector_hit_rate.pct`	Directa	Tablas 4.14, 4.33, 4.52 y 4.69

Tabla A.5. Correspondencia entre las magnitudes de tráfico de memoria y las métricas de NCU.

A.6. Memoria compartida y políticas de caché

Magnitud	Métrica NCU	Tablas del Capítulo 4
Wavefronts ejecutados	`l1tex__data_pipe_lsu_wavefronts_mem_shared.sum`	Tablas 4.15 y 4.34
Wavefronts ideales	`memory_l1_wavefronts_shared_ideal`	Tablas 4.15 y 4.34
Conflictos de banco	`l1tex__data_bank_conflicts_pipe_lsu_mem_shared.sum`	Tablas 4.15 y 4.34
`evict_first` hits	`lts__t_sectors_srcunit_tex_evict_first_lookup_hit.sum`	Tablas 4.16 y 4.35
`evict_first` misses	`lts__t_sectors_srcunit_tex_evict_first_lookup_miss.sum`	Tablas 4.16 y 4.35
`evict_last` hits	`lts__t_sectors_srcunit_tex_evict_last_lookup_hit.sum`	Tablas 4.16 y 4.35
`evict_last` misses	`lts__t_sectors_srcunit_tex_evict_last_lookup_miss.sum`	Tablas 4.16 y 4.35

Tabla A.6. Métricas utilizadas para estudiar memoria compartida y políticas de eviction de L2.

A.7. Emisión de instrucciones y motivos de stall

Magnitud	Métrica NCU	Tablas del Capítulo 4
Ciclos con emisión	`smsp__issue_active.avg.pct_of_peak_sustained_active`	Tablas 4.17, 4.36, 4.53 y 4.70; también interviene en la derivación de 4.19, 4.37, 4.54 y 4.71
Barrier	`smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio`	Tabla 4.18; derivada en las Tablas 4.19, 4.37, 4.54 y 4.71
Math Pipe Throttle	`smsp__average_warps_issue_stalled_math_pipe_throttle_per_issue_active.ratio`	Tabla 4.18; derivada en las Tablas 4.19, 4.37, 4.54 y 4.71
Wait	`smsp__average_warps_issue_stalled_wait_per_issue_active.ratio`	Tabla 4.18; derivada en las Tablas 4.19, 4.37, 4.54 y 4.71
Short Scoreboard	`smsp__average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio`	Tabla 4.18; derivada en las Tablas 4.19, 4.37, 4.54 y 4.71

Tabla A.7. Métricas utilizadas para estudiar la emisión de instrucciones y los principales motivos de stall.

Anexo B

Qué quiere decir la fórmula de la atención

La atención parte de una idea sencilla: para actualizar la representación de una posición de una secuencia, el modelo calcula qué otras posiciones contienen información útil. Para una única cabeza y sin máscara, la atención escalada producto punto se expresa como

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V, \tag{B.1}$$

donde Q, K y V representan las matrices de consultas, claves y valores, respectivamente, y d es la dimensión de la cabeza.

Antes de calcular la atención, la representación de cada token se proyecta a tres espacios distintos. Las consultas Q representan qué está buscando cada posición; las claves K representan qué tipo de información ofrece cada posición; y los valores V contienen la información que se transmitirá si una posición recibe peso de atención. Esta separación permite que una posición busque información en un espacio distinto de aquel en el que la ofrece.

El producto QK^T compara cada consulta con todas las claves. Si se considera una consulta q_i y una clave k_j, su producto escalar puede escribirse como

$$q_i \cdot k_j = \|q_i\| \, \|k_j\| \cos\theta. \tag{B.2}$$

Esta identidad muestra la relación con la similitud coseno: cuando dos vectores apuntan en direcciones parecidas, el coseno del ángulo es alto y el producto escalar tiende a ser mayor. Sin embargo, la atención no calcula únicamente similitud semántica en el espacio original de *embeddings*. Los tokens se transforman previamente mediante proyecciones entrenables, por lo que el producto escalar mide compatibilidad en espacios aprendidos de consulta y clave.

La división por $\sqrt{d}$ evita que las puntuaciones crezcan excesivamente al aumentar la dimensión de la cabeza. Sin este escalado, los valores de entrada al *softmax* podrían presentar magnitudes elevadas y producir distribuciones demasiado concentradas. El

escalado estabiliza la normalización y facilita el entrenamiento.

El *softmax* convierte las puntuaciones en pesos positivos que suman uno en cada fila. Posteriormente, esos pesos multiplican a V. La salida de una posición es, por tanto, una combinación ponderada de los vectores de valor. Una posición con un peso alto influye de forma importante en la salida, mientras que una posición con un peso pequeño apenas contribuye.

En atención causal se introduce además una matriz de máscara M:

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + M\right)V. \tag{B.3}$$

Para un modelo autoregresivo, $M_{ij} = 0$ cuando $j \leq i$ y toma un valor equivalente a $-\infty$ cuando $j > i$. De esta forma, la posición i no puede asignar probabilidad a posiciones futuras. Esta restricción es propia del caso autoregresivo; la implementación evaluada en este TFG utiliza atención no causal para aislar el comportamiento del kernel denso y mantener una configuración común entre todas las rutas comparadas.

La formulación anterior también permite entender por qué la operación resulta costosa. Para una secuencia de longitud N, la matriz de puntuaciones contiene N^2 elementos por cabeza. Una implementación eficiente no debe limitarse a acelerar los productos matriciales: también debe reducir escrituras intermedias, mantener estabilidad numérica y organizar los datos de forma que la GPU pueda sostener un elevado paralelismo.

Anexo C

NVIDIA Blackwell (SM100 y SM103)

Las GPU NVIDIA Blackwell orientadas a centros de datos constituyen una generación posterior a Ampere y amplían de forma importante los mecanismos disponibles para organizar el movimiento de datos y el cómputo.

Con respecto a Ampere, arquitectura sobre la que se desarrolla la implementación CUDA de este trabajo, Blackwell incorpora y amplía diferentes mecanismos destinados a mejorar el movimiento de datos, la utilización de los Tensor Cores y la capacidad de solapar distintas etapas de un kernel. Entre ellos destacan los siguientes:

- **Warp specialization.** La especialización de warps consiste en asignar funciones diferentes a distintos warps de un mismo bloque. Aunque la técnica no es exclusiva de Blackwell, las nuevas operaciones asíncronas de las generaciones más recientes permiten explotarla de forma especialmente efectiva. Por ejemplo, algunos warps pueden actuar como productores encargados de iniciar transferencias de memoria mediante TMA, mientras que otros actúan como consumidores ejecutando operaciones sobre los Tensor Cores mediante instrucciones `tcgen05.mma`. Otros warps pueden encargarse de etapas adicionales del algoritmo, como el cálculo del softmax. De esta forma, las distintas fases del kernel pueden ejecutarse de manera solapada y formar un pipeline productor-consumidor.

- **Tensor Memory (TMEM).** En Ampere, los acumuladores producidos por las instrucciones MMA se almacenan directamente en registros. Blackwell introduce Tensor Memory, una memoria dedicada a las operaciones de los Tensor Cores. En SM100, esta memoria se organiza como una matriz de 128 filas y 512 columnas por CTA, donde cada posición contiene 32 bits, lo que supone un total de 256 KiB. Los resultados de las operaciones `tcgen05.mma` pueden almacenarse y acumularse directamente en TMEM, reduciendo la presión sobre el fichero de registros. Posteriormente, estos datos pueden transferirse a registros cuando sea necesario realizar operaciones adicionales, como las correspondientes al epílogo.

- **Tensor Memory Accelerator (TMA).** TMA es un mecanismo hardware

para realizar transferencias asíncronas de datos entre memoria global y memoria compartida. A diferencia de las operaciones utilizadas en Ampere, el programador puede describir la disposición de un tensor mediante un *tensor map*, especificando sus dimensiones, strides y otras propiedades. El hardware se encarga entonces de calcular las direcciones necesarias para realizar la transferencia. TMA también permite aplicar directamente patrones de *swizzling* durante la copia a memoria compartida, evitando que el programador tenga que calcular manualmente estas transformaciones y facilitando la reducción de conflictos entre bancos de memoria.

- **Thread Block Clusters y Distributed Shared Memory.** Los *Thread Block Clusters*, introducidos originalmente en Hopper y soportados también por Blackwell, permiten agrupar varios bloques de hilos dentro de un mismo cluster. Los bloques pertenecientes a este cluster pueden acceder a la memoria compartida de otros bloques mediante un espacio denominado *Distributed Shared Memory* (DSM). Esto permite realizar lecturas, escrituras y operaciones atómicas sobre la memoria compartida de bloques que pueden estar ejecutándose en distintos Streaming Multiprocessors, evitando en determinados algoritmos la necesidad de utilizar memoria global como mecanismo intermedio de comunicación. Aunque este acceso resulta considerablemente más eficiente que intercambiar los datos a través de memoria global, acceder a la memoria compartida de otro SM presenta una latencia superior a la de la memoria compartida local.

- **MBarrier.** Las *mbarriers* constituyen un mecanismo de sincronización especialmente útil para implementar pipelines productor-consumidor junto con operaciones asíncronas como TMA. Una barrera se inicializa indicando el número esperado de participantes. Los hilos notifican su llegada a la barrera y, en el caso de operaciones asíncronas, también puede asociarse a ella una cantidad de datos pendientes de transferir. La fase de la barrera se considera completada únicamente cuando se han producido las llegadas esperadas y han finalizado todas las transacciones asociadas.

  Este mecanismo permite implementar de forma eficiente esquemas de *multibuffering*. Por ejemplo, cada buffer puede disponer de una barrera que indique que contiene datos listos para ser consumidos y otra que indique que puede volver a ser utilizado por el productor. Antes de leer un buffer, el consumidor espera a que finalice su barrera de datos disponibles; una vez procesados los datos, señala que el buffer puede reutilizarse. De forma análoga, el productor espera a que el buffer esté libre antes de iniciar una nueva transferencia. De este modo, movimiento de datos y cómputo pueden solaparse sin necesidad de sincronizar globalmente todos los warps.

- **Tensor Cores de quinta generación (TCGen05).** Blackwell introduce una nueva generación de Tensor Cores accesible mediante las instrucciones `tcgen05`. A diferencia de las instrucciones MMA utilizadas en Ampere, estas operaciones están diseñadas para trabajar conjuntamente con Tensor Memory y

memoria compartida. Los acumuladores se almacenan directamente en Tensor Memory, mientras que los operandos pueden obtenerse de memoria compartida y, en determinadas variantes, también de Tensor Memory. Esto permite realizar operaciones matriciales de mayores dimensiones sin mantener los acumuladores en el fichero general de registros, reduciendo así la presión sobre estos. Además, incorporan soporte hardware para nuevos formatos numéricos y operaciones de *block scaling*, permitiendo realizar productos matriciales de muy baja precisión de forma eficiente.

Estas características modifican considerablemente la forma en la que pueden estructurarse kernels de alto rendimiento con respecto a Ampere. En particular, permiten separar de forma más clara el movimiento de datos y el cómputo, reducir la presión sobre los registros y construir pipelines asíncronos en los que diferentes unidades hardware trabajan simultáneamente.

C.1. Una nota sobre Blackwell

Ampere tiene la ventaja de que las GPU de escritorio y de servidor comparten gran parte de las instrucciones utilizadas en este trabajo. En Blackwell no se da este caso de la misma manera: `sm_100` posee, por ejemplo, las instrucciones `tcgen05`, mientras que `sm_120` no dispone de ellas.

Conseguir acceso a una B200 para realizar un perfilado con NCU es muy complicado a día de hoy. NCU requiere permisos especiales para acceder a los contadores de rendimiento de la GPU [16], permisos que muchos servicios de alquiler de GPU no ofrecen. Además, en los servicios considerados durante este trabajo la disponibilidad de GPU Blackwell de escritorio era limitada, y conseguir acceso a modelos como la B200 resultaba todavía más complicado.

Sin embargo, Blackwell se incluye aquí como una forma de mostrar el estado del arte actual y la dirección seguida por las arquitecturas más recientes de NVIDIA. Para más información sobre las características de Blackwell, se puede consultar [7].

Anexo D

Evitar cómputos innecesarios

En la computación hay dos maneras principales de reducir cómputos:

1. Dar una aproximación, que es lo que hacemos con la exponencial.
2. No realizar la computación.

Lógicamente, el segundo caso es más rápido que el primero, que a su vez es más rápido que realizar la operación precisa.

En el cómputo de α es posible evitar calcular la exponencial en cada iteración del bucle. Si los valores se consideran variables aleatorias independientes e idénticamente distribuidas procedentes de una distribución continua, la probabilidad de que el valor n sea un nuevo máximo respecto a los $n-1$ valores anteriores es exactamente

$$P\left(X_n > \text{máx}(X_1, \ldots, X_{n-1})\right) = \frac{1}{n}. \tag{D.1}$$

Esto se debe a que existen n valores y solamente uno de ellos puede ser el máximo.

Conforme se procesan elementos, n aumenta. Con solo 10 valores procesados, existe un 10 % de probabilidad de que el décimo sea un nuevo máximo. Con 1000 valores procesados, esta probabilidad se reduce al 0,1 %.

Por tanto, la probabilidad de encontrar un nuevo máximo se reduce rápidamente conforme avanza la secuencia.

El problema es que el modelo de ejecución de NVIDIA no permite al programador un tratamiento de los saltos tan explícito como otras arquitecturas.

Como optimización básica, se pueden utilizar los atributos `[[likely]]` y `[[unlikely]]` para indicar al compilador cuál de los caminos se espera que sea más frecuente. Para longitudes de secuencia grandes, el caso más probable es que el máximo no cambie, aunque durante las primeras iteraciones sea considerablemente más probable encontrar nuevos máximos.

Una optimización más compleja requeriría conocer con mayor detalle cómo trata la GPU los saltos. Por ejemplo, sería interesante estudiar si los saltos introducen dependencias de ejecución, si existe algún mecanismo de ejecución especulativa, si

se aplica algún tipo de predicción estática del salto o si el compilador sustituye directamente determinadas ramas por predicación.

Como comparación, en el Cell Broadband Engine era posible proporcionar al procesador un *hint* sobre un salto futuro. Si existía un número suficiente de ciclos entre el *hint* y el salto, el procesador podía utilizar esta información antes de alcanzar la instrucción correspondiente [38].

La idea sería aplicar un principio similar: después de calcular el nuevo máximo, no calcular inmediatamente α, sino realizar primero trabajo independiente, como parte del cálculo de P, dejando suficiente distancia entre el cálculo de la condición y el punto en el que se necesita conocer el camino tomado. Actualmente no parece existir un mecanismo explícito de este tipo accesible al programador en GPUs NVIDIA, pero podría constituir una línea interesante de evolución del hardware.

Otra optimización consiste en aprovechar el cierto *unrolling* existente en el bucle. Se puede sacar un número determinado de iteraciones fuera del bucle y tratar las primeras iteraciones, donde encontrar un nuevo máximo es más probable, de forma distinta a las posteriores. De esta forma podrían utilizarse distintas indicaciones `[[likely]]` y `[[unlikely]]` según la posición dentro de la secuencia. El número de iteraciones de cada grupo podría tratarse como otro hiperparámetro.

Anexo E

Caché L2

No tengo tan claro que la caché L2 sea necesariamente beneficiosa para este tipo de kernel. Actualmente, K y V se reutilizan entre distintas matrices Q_r y son comunes a los warps de un mismo CTA mediante SRAM, pero deben volver a obtenerse desde L2 cuando son utilizados por CTAs ejecutados en distintos SM.

Esto puede convertirse en un problema al aumentar el número de SMs y la longitud de secuencia. No solo existirían más SMs accediendo simultáneamente a la caché, sino que también aumenta la posibilidad de que distintos SMs se encuentren procesando iteraciones diferentes de K_i y V_i. En ese caso, la reutilización temporal entre SMs disminuye y podría aumentar la presión sobre la L2 y sobre la memoria principal.

Las matrices Q_r se cargan una única vez para cada bloque, por lo que su reutilización mediante L2 es limitada. En el caso de K y V, una arquitectura alternativa podría cargarlas una única vez desde memoria principal y distribuirlas posteriormente entre los procesadores mediante una interconexión *on-chip*, evitando que cada SM tenga que obtener de nuevo los mismos datos a través de la jerarquía de caché.

Una posibilidad sería una estructura de procesadores sin una caché L2 tradicional para este tráfico. Todos los procesadores podrían acceder a memoria principal para cargar Q_r, mientras que uno o varios dispondrían de un mayor ancho de banda o menor latencia para cargar K_i y V_i. Una vez procesado un bloque K_i/V_i, este podría transferirse directamente al siguiente procesador mediante una interconexión *on-chip*, sin volver a pasar por memoria principal.

Esto podría organizarse como un anillo o mediante una topología más compleja, formando efectivamente un pipeline de procesadores por el que los bloques de K y V se desplazan mientras cada procesador mantiene y procesa sus propios bloques de Q_r.

Anexo F

Repositorio del proyecto

La totalidad del proyecto, incluyendo los archivos `.ncu-rep`, las características de la máquina utilizada, los resultados de Optuna y el código fuente, se encuentra disponible en:

https://github.com/angelMDUex/TFG-FlashAttention.git

Los elementos fundamentales del repositorio son:

1. `profile.sh`: realiza el perfilado de las distintas implementaciones para cada longitud de secuencia utilizando las configuraciones definidas en `CMakeUserPresets.json`.
2. `environment_report`: obtiene las métricas y características básicas de la máquina utilizada durante los experimentos.
3. `validez.sh`: comprueba la validez de la implementación CUDA para cada longitud de secuencia. Para ello, recompila el código utilizando el *preset* correspondiente de `CMakeUserPresets.json` antes de ejecutar cada prueba.

También se incluye, a modo de ejemplo, el archivo `CMakeUserPresets.json` utilizado durante los experimentos, que puede servir como plantilla para reproducir las distintas configuraciones.

La optimización de hiperparámetros mediante Optuna puede ejecutarse con:

```
uv run python src/py/tfg_fa/cuda_optimizer.py
```

Bibliografía

[1] W. A. Wulf and S. A. McKee, "Hitting the memory wall: Implications of the obvious," *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20–24, 1995. Disponible en: https://doi.org/10.1145/216585.216588.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017. Disponible en: https://arxiv.org/abs/1706.03762.

[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022. Disponible en: https://arxiv.org/abs/2205.14135.

[4] T. Tran, "Flash attention on rtx 5090." https://gau-nernst.github.io/fa-5090/, 2025.

[5] S. Li, "Flash attention from scratch." https://lubits.ch/flash/, Aug. 2025.

[6] A. Armbruster, "How to write a fast matrix multiplication from scratch with tensor cores." https://alexarmbr.github.io/2024/08/10/How-To-Write-A-Fast-Matrix-Multiplication-From-Scratch-With-Tensor-Cores.html, 2024.

[7] MLC Community, "Modern gpu programming for mlsys." https://mlc.ai/modern-gpu-programming-for-mlsys/, 2026.

[8] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning." https://arxiv.org/abs/2307.08691, 2023.

[9] W. mei W. Hwu, D. B. Kirk, and I. E. Hajj, *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 4 ed., 2022. Disponible en: https://www.sciencedirect.com/book/9780323912310/programming-massively-parallel-processors.

[10] NVIDIA Corporation, *CUDA C++ Best Practices Guide*. NVIDIA Corporation, 2026. Section: Coalesced Access to Global Memory. Disponible en: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/.

[11] NVIDIA Corporation, *CUDA Programming Guide*, 2026. Section 2.3.4.2: Shared Memory Access Patterns. Disponible en: https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html#shared-memory-access-patterns.

[12] NVIDIA Corporation, *CUDA Programming Guide*, 2026. Section 1.2.2.1: Thread Blocks and Grids. Disponible en: https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/programming-model.html#thread-blocks-and-grids.

[13] NVIDIA Corporation, *CUDA Programming Guide*, 2026. Section 2.3.7: Kernel Launch and Occupancy. Disponible en: https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html#kernel-launch-and-occupancy.

[14] NVIDIA Corporation, "Cuda programming guide: Asynchronous data copies." https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html, 2026. Producer-consumer and double-buffering examples. Consultado el 25 de septiembre de 2026.

[15] NVIDIA Corporation, *CUDA C++ Programming Guide*, 2025. CUDA Toolkit 13.0, Sections 7 and 8.2.3: Hardware Implementation and Multiprocessor Level. Disponible en: https://docs.nvidia.com/cuda/archive/13.0.0/cuda-c-programming-guide/index.html.

[16] NVIDIA Corporation, *Nsight Compute Profiling Guide*, 2026. Disponible en: https://docs.nvidia.com/nsight-compute/ProfilingGuide/.

[17] NVIDIA Corporation, "Parallel thread execution isa, version 9.0." https://docs.nvidia.com/cuda/archive/13.0.0/parallel-thread-execution/, 2025. CUDA Toolkit 13.0.

[18] NVIDIA Corporation, *CUDA Programming Guide*, 2026. Section 1.2.2.2: Warps and SIMT. Disponible en: https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/programming-model.html#warps-and-simt.

[19] NVIDIA Corporation, *CUDA Programming Guide*, 2026. Section 5.4.9.1: `#pragma unroll`. Disponible en: https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html.

[20] A. Fog, *Optimizing Software in C++: An Optimization Guide for Windows, Linux and Mac Platforms*, 2026. Section 7.13: Loops; subsection Loop unrolling. Disponible en: https://www.agner.org/optimize/optimizing_cpp.pdf.

[21] NVIDIA Corporation, "Cuda programming guide." https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html#local-memory, 2026. Section 2.3.3.4: Local Memory.

[22] NVIDIA Corporation, *CUDA Programming Guide*, 2026. Section 5.1: Compute Capabilities. Disponible en: https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/compute-capabilities.html.

[23] NVIDIA Corporation, "Nvidia ampere gpu architecture tuning guide." https://docs.nvidia.com/cuda/ampere-tuning-guide/, 2026. Consultado el 25 de septiembre de 2026.

[24] NVIDIA Corporation, *CUDA C++ Best Practices Guide*, 2026. Section 13.2: Branch Predication. Disponible en: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#branch-predication.

[25] NVIDIA Corporation, *CUDA Programming Guide*, 2026. Section 2.3.3: GPU Device Memory Spaces; Sections 2.3.3.2 and 2.3.3.3: Shared Memory and Registers. Disponible en: https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html#gpu-device-memory-spaces.

[26] D. Shanmughan, "How to improve cuda kernel performance with shared memory register spilling." https://developer.nvidia.com/blog/?p=105101, Aug. 2025. NVIDIA Technical Blog.

[27] NVIDIA Corporation, "Cuda programming guide: Shared-memory bank swizzling." https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html, 2026. Consultado el 25 de septiembre de 2026.

[28] Optuna Developers, "optuna.samplers.tpesampler — optuna documentation." https://optuna.readthedocs.io/en/stable/reference/samplers/generated/optuna.samplers.TPESampler.html, 2026.

[29] J.-M. Muller, "On the definition of ulp(x)," Tech. Rep. RR-5504, INRIA, Feb. 2005. Disponible en: https://inria.hal.science/inria-00070503v1/file/RR2005-09.pdf.

[30] Triton Project, "Triton — compatibility." https://github.com/triton-lang/triton#compatibility, 2026. Supported hardware includes NVIDIA GPUs and AMD GPUs.

[31] OpenAI, "Introducing triton: Open-source gpu programming for neural networks." https://openai.com/index/triton/, July 2021.

[32] Triton, "triton.autotune — triton documentation." https://triton-lang.org/main/python-api/generated/triton.autotune.html, 2026.

[33] NVIDIA, "Cuda c++ best practices guide." https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#throughput-of-native-arithmetic-instructions, 2026.

[34] NVIDIA Corporation, “Efficient gemm in cuda.” https://github.com/NVIDIA/cutlass/blob/v4.1.0/media/docs/cpp/efficient_gemm.md, 2025. CUTLASS, versión 4.1.0.

[35] T. Zadouri, M. Hoehnerbach, J. Shah, T. Liu, V. Thakkar, and T. Dao, “Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling,” in *Proceedings of Machine Learning and Systems*, 2026. Disponible en: https://proceedings.mlsys.org/paper_files/paper/2026/hash/ae8b0b5838ba510daff1198474e7b984-Abstract-Conference.html.

[36] S. Chevillard, M. Joldeş, and C. Lauter, “Sollya: An environment for the development of numerical codes,” in *Mathematical Software – ICMS 2010*, vol. 6327 of *Lecture Notes in Computer Science*, pp. 28–31, Springer, 2010.

[37] Sollya Developers, *Sollya User’s Manual: remez*, 2026. Command `remez`: minimax polynomial approximation using Remez’ algorithm. Disponible en: https://www.sollya.org/sollya-2.9/help.php?name=remez.

[38] International Business Machines Corporation, Sony Computer Entertainment Incorporated, and Toshiba Corporation, *Cell Broadband Engine Programming Handbook*, 1.1 ed., Apr. 2007. Section 24.3.3: Branch Hints. Disponible en: https://informatica.uv.es/docencia/iiguia/asignatu/2000/AAC/ps3/documentacion/CBE_Handbook_v1.1_24APR2007_pub.pdf.